Deep Reinforcement Learning

Mihir Rao

Contents

1 Introduction and Course Overview **5**
- 1.1 From supervised learning to reinforcement learning . . . 5
- 1.2 Why study deep reinforcement learning? . . . 6
- 1.3 Beyond learning from reward . . . 6
- 1.4 How do we build intelligent machines? . . . 7

2 Supervised Learning of Behaviors **8**
- 2.1 Terminology and notation . . . 8
- 2.2 Behavioral cloning and distributional shift . . . 8
- 2.3 A bit of theory . . . 9
- 2.4 Addressing the problem in practice . . . 10
- 2.5 DAgger . . . 12
- 2.6 Cost and reward functions . . . 12

3 PyTorch Tutorial **13**
- 3.1 Why PyTorch . . . 13
- 3.2 Tensors . . . 13
- 3.3 Devices . . . 14
- 3.4 Gradients . . . 14
- 3.5 The training loop and networks . . . 15
- 3.6 Best practices . . . 16

4 Introduction to Reinforcement Learning **17**
- 4.1 Markov decision processes . . . 17
- 4.2 The goal of reinforcement learning . . . 17
- 4.3 The anatomy of a reinforcement learning algorithm . . . 18
- 4.4 Value functions . . . 19
- 4.5 Types of algorithms . . . 20
- 4.6 Tradeoffs between algorithms . . . 20

5 Policy Gradients **22**
- 5.1 Direct policy differentiation . . . 22
- 5.2 Understanding policy gradients . . . 23
- 5.3 Reducing variance . . . 23
- 5.4 Off-policy policy gradients . . . 24
- 5.5 Implementing policy gradients . . . 25
- 5.6 Natural policy gradient . . . 26

6 Actor-Critic Algorithms **28**
- 6.1 Improving the policy gradient . . . 28
- 6.2 Value function fitting and policy evaluation . . . 28
- 6.3 From evaluation to actor-critic . . . 29
- 6.4 Actor-critic design decisions . . . 31

6.5 Critics as baselines . . . 32
6.6 Eligibility traces and n-step returns . . . 33

7 Value Function Methods **35**
7.1 Can we omit the policy gradient? . . . 35
7.2 Dynamic programming . . . 35
7.3 Fitted value iteration and Q-iteration . . . 36
7.4 From Q-iteration to Q-learning . . . 37
7.5 Value functions in theory . . . 37

8 Deep RL with Q-Functions **40**
8.1 Replay buffers . . . 40
8.2 Target networks . . . 40
8.3 A general view of Q-learning . . . 41
8.4 Improving Q-learning . . . 42
8.5 Q-learning with continuous actions . . . 43
8.6 Implementation tips and examples . . . 44

9 Advanced Policy Gradients **46**
9.1 Policy gradient as policy iteration . . . 46
9.2 Bounding the distribution change . . . 47
9.3 Policy gradients with constraints . . . 48
9.4 Natural gradient . . . 48

10 Optimal Control and Planning **50**
10.1 Model-based reinforcement learning . . . 50
10.2 The objective . . . 50
10.3 Stochastic optimization . . . 51
10.4 Monte Carlo tree search . . . 51
10.5 Trajectory optimization with derivatives . . . 52
10.6 Linear case: LQR . . . 52
10.7 LQR for stochastic and nonlinear systems . . . 54

11 Model-Based Reinforcement Learning **56**
11.1 Learning a model and planning through it . . . 56
11.2 Uncertainty in model-based RL . . . 57
11.3 Uncertainty-aware neural network models . . . 57
11.4 Planning with uncertainty . . . 58
11.5 Model-based RL with images . . . 59

12 Model-Based Policy Learning **61**
12.1 Backpropagating into the policy . . . 61
12.2 Model-free learning with a model . . . 62
12.3 Dyna-style algorithms . . . 63
12.4 Multi-step models and successor representations . . . 64

13 Exploration (Part 1) **67**
13.1 The exploration problem . . . 67
13.2 Bandits . . . 68
13.3 Three classes of exploration methods . . . 68
13.4 Optimistic exploration in deep RL . . . 69
13.5 More novelty-seeking exploration . . . 70
13.6 Posterior sampling in deep RL . . . 71
13.7 Information gain in deep RL . . . 71

14 Exploration (Part 2) **73**
14.1 Unsupervised learning of diverse behaviors . . . 73
14.2 Information theory . . . 73
14.3 Learning without a reward function by reaching goals . . . 73
14.4 State distribution matching . . . 74
14.5 Is state coverage a good exploration objective? . . . 75
14.6 Covering the space of skills . . . 75

15 Offline Reinforcement Learning (Part 1) **77**
15.1 Why offline reinforcement learning? . . . 77
15.2 Why is offline RL hard? . . . 78
15.3 Batch RL via importance sampling . . . 79
15.4 Batch RL via linear fitted value functions . . . 80

16 Offline Reinforcement Learning (Part 2) **82**
16.1 Policy constraint methods . . . 82
16.2 Implicit Q-learning . . . 83
16.3 Conservative Q-learning . . . 84
16.4 Model-based offline RL . . . 85
16.5 Which algorithm, and why offline RL . . . 86

17 Reinforcement Learning Theory Basics **88**
17.1 What questions do we ask? . . . 88
17.2 Sample complexity of a simple model-based algorithm . . . 88
17.3 Analyzing fitted Q-iteration . . . 91

18 Variational Inference and Generative Models **93**
18.1 Latent variable models . . . 93
18.2 Variational inference . . . 94
18.3 Amortized variational inference . . . 95
18.4 Variational inference in deep RL . . . 96

19 Connection between Inference and Control **98**
19.1 Optimal control as a model of human behavior . . . 98
19.2 Control as inference . . . 99
19.3 Control as variational inference . . . 100
19.4 Algorithms for RL as inference . . . 102
19.5 Example methods . . . 104

20 Inverse Reinforcement Learning **105**
20.1 Why learn rewards? . . . 105
20.2 Learning the reward function . . . 106
20.3 Approximations in high dimensions . . . 107
20.4 IRL and GANs . . . 108

21 RL with Sequence Models **110**
21.1 Beyond MDPs . . . 110
21.2 Which methods handle partial observability? . . . 110
21.3 State space models and history states . . . 111
21.4 RL and language models . . . 112
21.5 RL from human feedback . . . 112
21.6 Multi-step RL with language models . . . 113

22 Meta-Learning and Transfer Learning **115**
22.1 Transfer learning . . . 115

22.2 Forward transfer . . . 115
22.3 Multi-task transfer . . . 116
22.4 Meta-learning . . . 117
22.5 Meta-RL with recurrent policies . . . 118
22.6 Gradient-based meta-learning . . . 118
22.7 Meta-RL as a POMDP . . . 119
22.8 Three perspectives on meta-RL . . . 120

23 Challenges and Open Problems **121**
23.1 A brief review . . . 121
23.2 Challenges in deep RL . . . 121
23.3 RL as an engineering tool . . . 122
23.4 RL and the real world . . . 123
23.5 RL as "universal" learning . . . 123
23.6 Back to the bigger picture . . . 124

Lecture 1: Introduction and Course Overview

1.1 From supervised learning to reinforcement learning

Consider a robot arm with a monocular RGB camera that must pick objects out of a bin. One option is to understand the problem and design a solution by hand. Another is to set it up as a machine learning problem: collect images labeled with grasp locations (x, y, z) and train a supervised model. A third is to let the robots attempt grasps, record each image, grasp, and outcome in {success, failure}, and use reinforcement learning to improve the grasping policy from that self-collected experience.

Recent advances in AI (image generation, chatbots, protein design) rest on one idea: fit a density $p_\theta(x)$ or a conditional $p_\theta(y \mid x)$ to large amounts of data, and so reproduce what people produced. Reinforcement learning is different. It descends from classical reinforcement learning (an agent receives a state S_t and reward R_t and emits an action A_t), evolutionary algorithms, control, and optimization; deep reinforcement learning combines these with deep networks. Because it optimizes rather than imitates, it can discover new solutions, such as the unexpected "move 37" of AlphaGo against Lee Sedol.

Definition 1.1 [Reinforcement learning]. Reinforcement learning is both a mathematical formalism for learning-based decision making and an approach for learning decision making and control from experience.

Standard supervised learning is given $\mathcal{D} = \{(x_i, y_i)\}$ and learns to predict y from x, $f(x) \approx y$. It usually assumes i.i.d. data and known ground-truth outputs during training. In reinforcement learning the data is not i.i.d., since previous outputs influence future inputs, and the ground-truth answer is not known: we only know whether we succeeded or failed, or more generally the reward. Figure 1 contrasts the two settings: in reinforcement learning the agent picks its own actions, and nobody hands it the data.

Decisions (actions) have consequences (observations, or states, and rewards). For a dog, actions are muscle contractions, observations are sight and smell, and the reward is food. For a robot, actions are motor currents or torques, observations are camera images, and the reward is a task-success measure such as running speed. In inventory management, actions are what to purchase, observations are inventory levels, and the reward is profit. Applications range from dexterous manipulation, agile locomotion, and navigation to Atari games, waste sorting with a fleet of mobile manipulators, traffic control, fine-tuning language models from human feedback, training diffusion models with reinforcement learning (DDPO), and chip design.

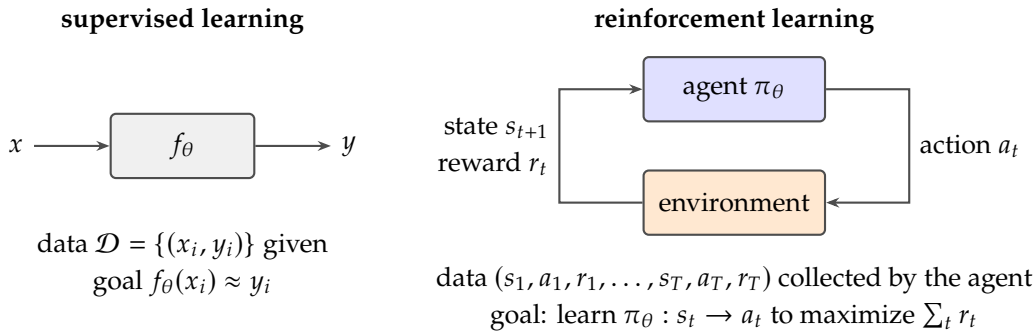


Figure 1: Supervised learning maps inputs to given labels; a reinforcement learning agent acts on an environment, observes the next state and a reward, and learns a policy that maximizes total reward.

1.2 Why study deep reinforcement learning?

Data-driven AI is all about using data: it learns about the real world from data but does not try to do better than the data. Reinforcement learning is all about optimization: it optimizes a goal with emergent behavior, but we still need to figure out how to use it at scale. Data without optimization does not let us solve new problems in new ways; optimization without data is hard to apply to the real world outside of simulators.

Sutton's "bitter lesson" says that building in how we think we think does not work in the long run, and that the two methods that seem to scale arbitrarily are learning and search. This is often misread as "use more data". The intended reading pairs two ingredients:

- *learning* uses data to extract patterns and lets us understand the world;
- *search* uses computation to extract inferences: some (typically iterative) optimization process that makes rational decisions, leveraging that understanding for emergence.

A postulate: we need machine learning for one reason only, to produce adaptable and complex decisions (how to move the joints, how to steer the car). Even an image label matters only through what is done with it afterwards. Wolpert makes the parallel claim about brains: we have a brain to produce adaptable and complex movements, the only way we have of affecting the world.

Now is the time because big end-to-end trained models work quite well, and we have reinforcement learning algorithms that can feasibly be combined with deep networks. Yet learning-based control in truly real-world settings remains a major open problem.

1.3 Beyond learning from reward

Basic reinforcement learning maximizes rewards, but that is not the only problem that matters for sequential decision making. Rewards are themselves problematic: in a game the score is the

reward, but for a cheetah chasing a gazelle, or a person, rewards are extremely sparse and it is unclear where they come from. Other forms of supervision include:

- *learning from demonstrations*: directly copying observed behavior (IMITATION LEARNING, for example end-to-end driving), or inferring rewards and intentions from observed behavior (INVERSE REINFORCEMENT LEARNING); even young children infer the intention behind an action rather than copying it;
- *learning from observing the world*: learning to predict, and unsupervised learning; predictive models of video can be used for real-world control and tool use, and have improved rapidly;
- *learning from other tasks*: TRANSFER LEARNING, and META-LEARNING (learning to learn), including leveraging pretrained vision-language models to produce robot actions.

1.4 How do we build intelligent machines?

Some things we can all do, such as walking; others we can only learn, such as driving a car. We can learn a huge variety of difficult things, so our learning mechanism(s) are likely powerful enough to do everything we associate with intelligence, though it may still be convenient to hard-code a few important bits. Rather than an algorithm for each module of the brain, there may be a single flexible algorithm: people can learn to "see" through electrodes on the tongue, and rewiring visual input to the auditory cortex lets it learn to process vision. That algorithm must interpret rich sensory inputs and choose complex actions. Hence deep reinforcement learning: *deep* for scalable learning from large, complex datasets, *reinforcement learning* for optimization.

There is some evidence for both. Unsupervised deep models reproduce receptive fields of primary sensory cortex. In the brain, percepts that anticipate reward come to produce firing patterns similar to the reward itself, the basal ganglia appear related to the reward system, and model-free reinforcement-learning-like adaptation often (but not always) fits experimental data on animal adaptation.

Challenges remain. We have great methods that learn from huge amounts of data and great optimization methods for reinforcement learning, but not yet methods that do both well. Humans learn incredibly quickly while deep reinforcement learning is usually slow; humans reuse past knowledge, while transfer in reinforcement learning is an open problem; and it is not clear what the reward function should be, nor what role prediction should play. Turing's suggestion fits the picture: instead of simulating the adult mind, simulate the child's and educate it, that is, couple a general learning algorithm with an environment through observations and actions.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 1 (Fall 2023).

Lecture 2: Supervised Learning of Behaviors

2.1 Terminology and notation

A policy maps what the agent perceives to an action. We write s_t for the state, o_t for the observation, and a_t for the action at time t. A policy is a distribution $\pi_\theta(a_t \mid o_t)$ with parameters θ (for example a deep network mapping an image to a distribution over actions); when the state is observed directly, $\pi_\theta(a_t \mid s_t)$ is a fully observed policy. Actions may be discrete (on seeing a tiger: run away, ignore, or pet it) or continuous. The state is a complete physical description of the system (positions and velocities); the observation, such as an image, is produced from the state and may not determine it, for instance when the tiger is occluded.

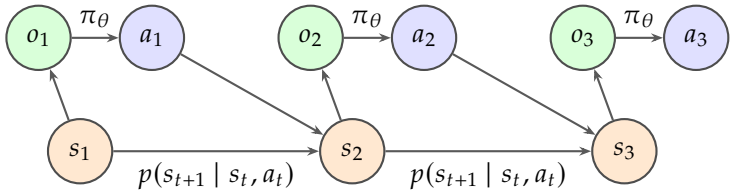


Figure 2: Graphical model of a controlled system. Observations are emitted by states, the policy picks actions from observations, and the dynamics $p(s_{t+1} \mid s_t, a_t)$ produce the next state.

The states satisfy the Markov property: given s_t and a_t, the next state s_{t+1} is independent of s_{t-1} (Figure 2). Observations in general do not: past observations can carry information about the current state that the current observation lacks. In the control literature the state is written x_t and the action u_t (from the Russian *upravlenie*, control); in dynamic programming, s_t and a_t.

2.2 Behavioral cloning and distributional shift

Definition 2.1 [Behavioral cloning]. Imitation learning by behavioral cloning collects observation-action pairs (o_t, a_t) from a demonstrator, such as a human driver, and trains $\pi_\theta(a_t \mid o_t)$ on them by supervised learning:

$$\max_\theta \ \mathbb{E}_{o_t \sim p_{\text{data}}(o_t)} \big[\log \pi_\theta(a_t \mid o_t) \big].$$

The original deep imitation learning system, ALVINN (Autonomous Land Vehicle In a Neural Network, 1989), drove a vehicle this way with a small network mapping a 30×32 video retina to 30 steering outputs. Does it work? In general, no: a small error takes the policy to a state slightly unlike the training data, where it errs more, and the errors compound (Figure 3). Yet NVIDIA's end-to-end driving system (2016) works. Its trick is in the data: besides the center camera, left and right cameras record the same drive, and their images are labeled with the recorded steering angle

adjusted for the shift and rotation, so the data shows how to steer back toward the center. Random shifts and rotations augment the data further.

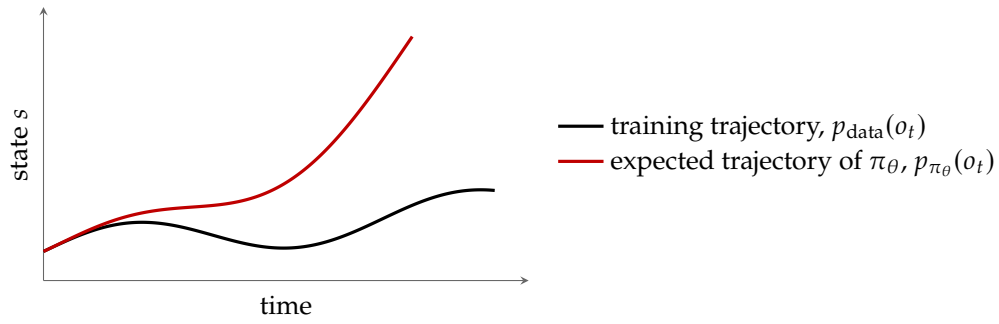


Figure 3: Distributional shift: the policy is trained on $p_{\text{data}}(o_t)$ but tested on $p_{\pi_\theta}(o_t)$, and small mistakes compound. Schematic.

Imitation learning via behavioral cloning is therefore not guaranteed to work, unlike supervised learning, because the i.i.d. assumption does not hold: we train under $p_{\text{data}}(o_t)$ but test under $p_{\pi_\theta}(o_t) \neq p_{\text{data}}(o_t)$. This is the DISTRIBUTIONAL SHIFT problem. It can be addressed by being smart about how we collect (and augment) the data, by using very powerful models that make very few mistakes, by multi-task learning, or by changing the algorithm (DAgger).

2.3 A bit of theory

The likelihood objective does not say what makes a learned policy good. Measure instead the mistakes it makes when run: with an expert policy $\pi^\star$, let

$$c(s_t, a_t) = \begin{cases} 0 & \text{if } a_t = \pi^\star(s_t), \\ 1 & \text{otherwise,} \end{cases} \qquad \text{and minimize} \qquad \mathbb{E}_{s_t \sim p_{\pi_\theta}(s_t)}\big[c(s_t, a_t)\big]$$

summed over t (here states and observations are used interchangeably).

Proposition 2.2 [COMPOUNDING ERRORS]. *Suppose that on the training distribution the policy errs with probability at most ϵ: $\mathbb{E}_{p_{\text{train}}(s)}\big[\pi_\theta(a \neq \pi^\star(s) \mid s)\big] \leq \epsilon$. Then over a horizon T, $\mathbb{E}\big[\sum_t c(s_t, a_t)\big] = O(\epsilon T^2)$, and this order is attained in the worst case.*

Proof. For the worst case, take a tightrope walk: once the policy makes a mistake it is off the training data, where it may err at every remaining step. Assuming errors at most ϵ on all training states,

$$\mathbb{E}\Big[\sum_t c(s_t, a_t)\Big] \leq \epsilon T + (1-\epsilon)\big(\epsilon(T-1) + (1-\epsilon)(\ldots)\big),$$

which has T terms, each $O(\epsilon T)$, so the total is $O(\epsilon T^2)$.

In general, with probability $(1-\epsilon)^t$ no mistake has been made by time t, so

$$p_\theta(s_t) = (1-\epsilon)^t p_{\text{train}}(s_t) + \big(1-(1-\epsilon)^t\big) p_{\text{mistake}}(s_t)$$

for some other distribution p_{mistake}. Hence, summing over states and using $(1-\epsilon)^t \geq 1-\epsilon t$ for $\epsilon \in [0,1]$,

$$\sum_{s_t} \big|p_\theta(s_t) - p_{\text{train}}(s_t)\big| = \big(1-(1-\epsilon)^t\big) \sum_{s_t} \big|p_{\text{mistake}}(s_t) - p_{\text{train}}(s_t)\big| \leq 2\big(1-(1-\epsilon)^t\big) \leq 2\epsilon t.$$

With $c_{\max} = 1$,

$$\begin{aligned}\sum_t \mathbb{E}_{p_\theta(s_t)}[c_t] &= \sum_t \sum_{s_t} p_\theta(s_t) c_t(s_t) \leq \sum_t \sum_{s_t} \Big(p_{\text{train}}(s_t) c_t(s_t) + \big|p_\theta(s_t) - p_{\text{train}}(s_t)\big| c_{\max}\Big) \\ &\leq \sum_t (\epsilon + 2\epsilon t) = O(\epsilon T^2). \qquad \square\end{aligned}$$

The bound is pessimistic: in reality we can often recover from mistakes. But that does not mean imitation learning will learn how to recover. This gives a paradox: imitation learning can work better if the data has more mistakes (and recoveries).

2.4 Addressing the problem in practice

Collect and augment data. Behavioral cloning is easy when the data covers the states the policy will visit. Intentionally adding mistakes and corrections helps: the mistakes hurt, but the corrections often help more than the mistakes hurt. Data augmentation adds "fake" data that illustrates corrections, such as the side-facing cameras above. Trail following for a quadrotor can be cast as classification of the front camera image into turn left, go straight, or turn right; imitation with inexpensive robot arms is another example.

Fit the expert better. Two reasons we might fail to fit the expert are non-Markovian behavior and multimodal behavior.

A policy $\pi_\theta(a_t \mid o_t)$ depends only on the current observation: if it sees the same thing twice, it does the same thing twice regardless of what happened before. Human demonstrators are often unnatural under this constraint, since their behavior depends on all past observations, $\pi_\theta(a_t \mid o_1, \ldots, o_t)$. Stacking frames into one network input gives a variable number of frames and too many weights. Instead, an encoder with shared weights processes each frame, and a sequence model (Transformer, LSTM) combines the encodings. History can also hurt through CAUSAL CONFUSION: if the brake indicator is visible, a policy can learn to brake whenever the indicator is on, attending to the consequence of braking rather than to its cause (the pedestrian). Whether including history, or DAgger, mitigates causal confusion is a natural question.

Multimodal behavior arises when, for example, the demonstrator passes a tree on the left in some

demonstrations and on the right in others. A unimodal Gaussian averages the modes and drives into the tree. Two remedies are more expressive continuous distributions and discretization.

- *Mixture of Gaussians*: the policy is $\pi(a \mid o) = \sum_i w_i \mathcal{N}(\mu_i, \Sigma_i)$, and the network outputs the weights, means, and covariances w_1, μ_1, Σ_1 through w_N, μ_N, Σ_N.
- *Latent variable models*: the network also takes noise $\xi \sim \mathcal{N}(0, I)$, and different noise values produce different modes. The most widely used model of this sort is the (conditional) variational autoencoder, covered later in the course.
- *Diffusion models*: with x_0 the true sample and $x_{i+1} = x_i + \text{noise}$, a network learns $f(x_i) = x_{i-1}$ (in practice it predicts the noise) and generates by $x_{i-1} = x_i - f(x_i)$ from pure noise $x_T \sim \mathcal{N}(0, I)$. For actions, $a_{t,0}$ is the true action, $a_{t,i+1} = a_{t,i} + \text{noise}$, and the network $f(s_t, a_{t,i})$ is conditioned on the state; starting from noise and iterating $a_{t,i-1} = a_{t,i} - f(s_t, a_{t,i})$ yields a_t.
- *Discretization*: a softmax over bins is great for one-dimensional actions, but discretizing a high-dimensional action space is impractical. Autoregressive discretization discretizes one dimension at a time. For $a_t = (a_{t,0}, a_{t,1}, a_{t,2})$, a sequence model (LSTM or Transformer) on top of an image encoder outputs $p(a_{t,0} \mid s_t)$, then $p(a_{t,1} \mid s_t, a_{t,0})$, then $p(a_{t,2} \mid s_t, a_{t,0}, a_{t,1})$, sampling each dimension and feeding it back. This works by the chain rule:

$$p(a_{t,2} \mid s_t, a_{t,0}, a_{t,1})\, p(a_{t,1} \mid s_t, a_{t,0})\, p(a_{t,0} \mid s_t) = p(a_{t,0}, a_{t,1}, a_{t,2} \mid s_t) = p(a_t \mid s_t).$$

Case studies include Diffusion Policy (imitation with diffusion models), imitation with latent variables and Transformers for fine-grained bimanual manipulation on low-cost hardware, and RT-1 (imitation with Transformers and discretized actions).

Multi-task learning. Learning many tasks can be easier than one. A policy $\pi_\theta(a \mid s)$ for reaching one goal p_1 sees only states near the demonstrations of that goal; a policy $\pi_\theta(a \mid s, p)$ for reaching any p is trained on data covering many more states.

Definition 2.3 [Goal-conditioned behavioral cloning]. A demonstration $\{s_1, a_1, \ldots, s_{T-1}, a_{T-1}, s_T\}$ is a successful demonstration for reaching its own final state s_T. Train a goal-conditioned policy $\pi_\theta(a \mid s, g)$ by maximizing, for each demonstration i, $\log \pi_\theta(a_t^i \mid s_t^i, g = s_T^i)$.

Distributional shift now appears in two places: in the states, and in the goals the policy is asked to reach. Examples include learning latent plans from unstructured "play" data (collect data, train a goal-conditioned latent-variable policy, reach goals) and goal-conditioned behavioral cloning at large scale across many navigation datasets and robots. Going beyond imitation, goal-conditioned behavioral cloning can improve itself: start with a random policy, collect data with random goals, treat the data as demonstrations for the goals that were actually reached, use it to improve the policy, and repeat. Hindsight experience replay applies the same relabeling principle in off-policy value-based reinforcement learning.

2.5 DAgger

Can we make $p_{\text{data}}(o_t) = p_{\pi_\theta}(o_t)$? Instead of being clever about $p_{\pi_\theta}(o_t)$, be clever about $p_{\text{data}}(o_t)$: collect training data from $p_{\pi_\theta}(o_t)$ by running the policy. The only obstacle is that this data needs action labels.

Algorithm 2.1: DAgger (Dataset Aggregation)

Input: human data $\mathcal{D} = \{o_1, a_1, \ldots, o_N, a_N\}$

repeat
- train $\pi_\theta(a_t \mid o_t)$ from $\mathcal{D}$
- run $\pi_\theta(a_t \mid o_t)$ to get a dataset $\mathcal{D}_\pi = \{o_1, \ldots, o_M\}$
- ask a human to label $\mathcal{D}_\pi$ with actions a_t
- aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$

until done

Eventually the aggregated data covers the observations the policy itself visits, so training and test distributions agree; DAgger has been used, for example, to navigate through a forest from onboard camera images. The practical problem is the labeling step: the human must say what action to take for observations they did not generate, which is often unnatural.

2.6 Cost and reward functions

Imitation learning has deeper problems: humans must provide data, which is typically finite, while deep learning works best with plentiful data; humans are not good at providing some kinds of actions (the rotor speeds of a quadrotor, the joint torques of a humanoid); and humans learn autonomously, from unlimited experience and with continuous self-improvement, so our machines should too.

The alternative is to specify what we want. A COST FUNCTION $c(s_t, a_t)$, or a REWARD FUNCTION $r(s_t, a_t)$, scores each step, and we solve

$$\min_\theta \; \mathbb{E}_{s_{1:T}, a_{1:T}} \left[\sum_t c(s_t, a_t) \right],$$

for example with $c(s_t, a_t) = \delta(s_t = \text{eaten by tiger})$. The two conventions are related by $r(s, a) = -c(x, u)$: dynamic programming (Bellman) uses states, actions, and rewards; optimal control (Pontryagin) uses x_t, u_t, and costs. Imitation is a special case: with

$$r(s, a) = \log p\big(a = \pi^\star(s) \mid s\big) \qquad \text{or} \qquad c(s, a) = \begin{cases} 0 & \text{if } a = \pi^\star(s), \\ 1 & \text{otherwise,} \end{cases}$$

imitation learning, when it works well, maximizes $\mathbb{E}_{s_t \sim p_{\pi_\theta}(s_t)}[r(s_t, a_t)]$: it does maximize reward, for a very particular choice of reward.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 2 (Fall 2023).

Lecture 3: PyTorch Tutorial

3.1 Why PyTorch

Reinforcement learning alternates between collecting data with an agent and training a model, such as a policy $\pi_\theta(a \mid s)$, a Q-function $Q_\theta(s, a)$, or a dynamics model $\hat{\Delta}_{t+1} = f_\theta(s_t, a_t)$. Training a model means solving

$$\theta^\star = \arg\min_\theta \sum_{(x,y)\in\mathcal{D}} \mathcal{L}\big(f_\theta(x), y\big)$$

by gradient descent over a dataset $\mathcal{D}$, with a loss $\mathcal{L}$ and a neural network f_θ. PyTorch is a Python library for defining neural networks, automating the computation of gradients, and more (datasets and data loading, common network operations, built-in optimizers such as Adam and SGD, GPUs). We define the forward computation, for example $h_1 = \sigma(W_1 x)$, $h_2 = \sigma(W_2 h_1)$, $y = \sigma(W_3 h_2)$, and PyTorch computes the derivatives by the chain rule, such as $\frac{\partial y}{\partial W_1} = \frac{\partial y}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W_1}$. Like NumPy it has fast imperative CPU implementations; unlike NumPy it also runs on GPUs and supports automatic differentiation.

3.2 Tensors

Element-wise operations and indexing look the same as in NumPy and run about 100× faster than the equivalent Python list comprehensions:

```python
import torch

tensor_a = torch.tensor([1, 3, 4, 5, 9])
tensor_b = torch.tensor([9, 5, 7, 2, 5])

# Element-wise operations
tensor_sum = tensor_a + tensor_b
tensor_prod = tensor_a * tensor_b
tensor_doubled = 2 * tensor_a

# Indexing
value = tensor_a[3]
tensor_slice = tensor_a[2:3]
tensor_idx = torch.tensor([3, 2, 1])
tensor_indexed = tensor_a[tensor_idx]
```

A scalar, vector, and matrix are 0-, 1-, and 2-way tensors; in general an N-way tensor has N axes. For

$$A = \begin{pmatrix} 32 & 27 & 5 & 54 & 1 \\ 99 & 4 & 23 & 3 & 57 \\ 76 & 42 & 34 & 82 & 5 \end{pmatrix}, \qquad \texttt{A.shape == (3, 5)},$$

axis 0 indexes rows and axis 1 columns: `A[0, 3]` is 54, `A[:, 3]` is the column $(54, 3, 82)$, `A[0, :]` is the first row, and `A[0, 2:4]` is $(5, 54)$. For a tensor of shape `(3, 5, 4)`, `A[0, ...]` selects index 0 along the first axis and `A[..., 1]` index 1 along the last, where the ellipsis stands for all remaining axes.

```
A = torch.randn((10, 15))
A[None]                 # adds an axis of size 1 -> shape (1, 10, 15)
A[None].squeeze(0)      # removes an axis of size 1 -> shape (10, 15)
A.permute((1, 0))       # switches axes -> shape (15, 10)
# !!! BE CAREFUL WITH VIEW !!!
A.view(15, 10)          # -> shape (15, 10)
A.view(3, 25, -1)       # -> shape (3, 25, 2)
```

In NumPy the corresponding operations are `A[np.newaxis]`, `squeeze`, `transpose`, and `reshape`.

3.3 Devices

NumPy arrays all live in the CPU's RAM; PyTorch tensors live in CPU or GPU memory. Move a tensor to the GPU with `.to("cuda")` or `.cuda()`, and back with `.to("cpu")` or `.cpu()`. Operations between tensors on different devices are not allowed:

```
device = torch.device("cuda")
x = torch.zeros((2, 3))
y = torch.ones((2, 3), device=device)
z = x + y   # RuntimeError: Expected all tensors to be on the same device,
            # but found at least two devices, cuda:0 and cpu!
```

3.4 Gradients

Tensors created with the flag `requires_grad=True` are leaves of a computation graph, and calling `backward()` on a scalar fills in their `.grad` fields. `.detach()` returns a copy cut off from the graph, so no gradient flows through it:

```
P = torch.randn((1024, 1024))
print(P.requires_grad)  # -> False
P = torch.randn((1024, 1024), requires_grad=True)
b = torch.randn((1024,), requires_grad=True)
print(P.grad)           # -> None

x = torch.randn((32, 1024))
y = torch.relu(x @ P + b)
target = 3
loss = torch.mean((y - target) ** 2)
loss.backward()         # fills P.grad and b.grad
# torch.mean((y - target) ** 2).detach() would block gradients to P and b
```

3.5 The training loop and networks

The training loop always looks the same: load a batch, compute the loss, then `zero_grad()`, `backward()`, `step()`.

```
net = (...).to("cuda")
dataset = ...
dataloader = ...
optimizer = ...
loss_fn = ...
for epoch in range(num_epochs):
    # Training
    net.train()
    for data, target in dataloader:
        data = torch.from_numpy(data).float().cuda()
        target = torch.from_numpy(target).float().cuda()

        prediction = net(data)
        loss = loss_fn(prediction, target)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    net.eval()
    # Do evaluation
```

To convert NumPy to PyTorch, use `torch.from_numpy(numpy_array).float()` (NumPy defaults to 64-bit floats, PyTorch to 32-bit). To convert back, detach the tensor if it requires gradients, move

it to the CPU if it is on the GPU, and call `.numpy()`: `torch_tensor.detach().cpu().numpy()`.

Networks subclass `nn.Module`. Call `net(x)` rather than `net.forward(x)`, and keep the network and its inputs on the same device.

```python
import torch.nn as nn

class SingleLayerNetwork(nn.Module):
    def __init__(self, in_dim: int, out_dim: int, hidden_dim: int):
        super().__init__()  # <- Don't forget this!
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)

batch_size = 256
my_net = SingleLayerNetwork(2, 1, 32).to("cuda")
output = my_net(torch.randn(size=(batch_size, 2)).cuda())
```

3.6 Best practices

- When in doubt, `assert` is your friend, for example `assert x.shape == (B, N)` with a message reporting the actual shape.
- Be extra careful with `.reshape` and `.view`: assert shapes before and after, use them only to collapse or expand a single dimension, and prefer `.flatten()`, `.permute()`, and `.unflatten()`.
- Test complicated operations against a pure Python implementation.
- Do not mix NumPy and PyTorch code: understand the boundary between them, cast 64-bit NumPy arrays to 32 bits, and use only `torch.Tensor` inside an `nn.Module`.

Source. Kyle Stachowicz, *CS 285: Deep Reinforcement Learning*, Lecture 3 (Fall 2023).

Lecture 4: Introduction to Reinforcement Learning

4.1 Markov decision processes

Lecture 2 introduced states s_t, observations o_t, actions a_t, and policies $\pi_\theta(a_t \mid o_t)$ or $\pi_\theta(a_t \mid s_t)$. Instead of imitating, we now specify which actions are better or worse with a reward function $r(s, a)$, which tells us which states and actions are better (driving smoothly has high reward, a crash low reward). The quantities s, a, $r(s, a)$, and $p(s' \mid s, a)$ define a Markov decision process.

Definition 4.1 [Markov chain]. A Markov chain $\mathcal{M} = \{\mathcal{S}, \mathcal{T}\}$ has a state space $\mathcal{S}$ (states $s \in \mathcal{S}$, discrete or continuous) and a transition operator $\mathcal{T}$ given by $p(s_{t+1} \mid s_t)$. With $\mu_{t,i} = p(s_t = i)$ and $\mathcal{T}_{i,j} = p(s_{t+1} = i \mid s_t = j)$, the vector of probabilities evolves linearly, $\vec{\mu}_{t+1} = \mathcal{T}\,\vec{\mu}_t$, which is why $\mathcal{T}$ is called an operator. The next state is independent of s_{t-1} given s_t (the Markov property).

Definition 4.2 [Markov decision process]. A Markov decision process (MDP) $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$ adds an action space $\mathcal{A}$ (actions $a \in \mathcal{A}$, discrete or continuous) and a reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. The transition operator is now a tensor: with $\mu_{t,j} = p(s_t = j)$, $\xi_{t,k} = p(a_t = k)$, and $\mathcal{T}_{i,j,k} = p(s_{t+1} = i \mid s_t = j, a_t = k)$,

$$\mu_{t+1,i} = \sum_{j,k} \mathcal{T}_{i,j,k}\, \mu_{t,j}\, \xi_{t,k}.$$

A partially observed Markov decision process (POMDP) $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, r\}$ also has an observation space $\mathcal{O}$ and an emission probability $\mathcal{E}$, $p(o_t \mid s_t)$.

4.2 The goal of reinforcement learning

A policy $\pi_\theta(a \mid s)$ (partial observability is revisited later) acting in an environment with dynamics $p(s' \mid s, a)$ induces a distribution over trajectories $\tau = (s_1, a_1, \ldots, s_T, a_T)$:

$$p_\theta(\tau) = p_\theta(s_1, a_1, \ldots, s_T, a_T) = p(s_1) \prod_{t=1}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t).$$

The goal is

$$\theta^\star = \arg\max_\theta \; \mathbb{E}_{\tau \sim p_\theta(\tau)} \Big[\sum_t r(s_t, a_t) \Big].$$

The pairs (s_t, a_t) form a Markov chain on the augmented state, with transitions $p\big((s_{t+1}, a_{t+1}) \mid (s_t, a_t)\big) = p(s_{t+1} \mid s_t, a_t)\, \pi_\theta(a_{t+1} \mid s_{t+1})$.

Finite horizon. By linearity of expectation,

$$\theta^\star = \arg\max_\theta \sum_{t=1}^{T} \mathbb{E}_{(s_t,a_t)\sim p_\theta(s_t,a_t)}\big[r(s_t,a_t)\big],$$

where $p_\theta(s_t, a_t)$ is the STATE-ACTION MARGINAL at time t.

Infinite horizon. If $T = \infty$, does $p(s_t, a_t)$ converge to a STATIONARY DISTRIBUTION, one that is the same before and after a transition? Writing the state-action transition operator as $(s_{t+1}, a_{t+1})^T = \mathcal{T}(s_t, a_t)^T$, so that $(s_{t+k}, a_{t+k})^T = \mathcal{T}^k(s_t, a_t)^T$, a stationary μ satisfies $\mu = \mathcal{T}\mu$, that is, $(\mathcal{T} - I)\mu = 0$: $\mu = p_\theta(s, a)$ is an eigenvector of $\mathcal{T}$ with eigenvalue 1, which always exists under some regularity conditions. The undiscounted average-reward objective then becomes an expectation under the stationary distribution:

$$\theta^\star = \arg\max_\theta \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_{(s_t,a_t)\sim p_\theta(s_t,a_t)}\big[r(s_t,a_t)\big] \longrightarrow \mathbb{E}_{(s,a)\sim p_\theta(s,a)}\big[r(s,a)\big] \quad \text{as } T \to \infty.$$

In reinforcement learning we almost always care about *expectations*, which makes the objective smooth even when the reward is not. A car on a mountain road gets $r(x) = +1$ on the road and -1 after falling off the cliff, which is not smooth; but if $\pi_\theta(a = \text{fall}) = \theta$, then $\mathbb{E}_{\pi_\theta}[r(x)] = (1-\theta) - \theta = 1 - 2\theta$ is smooth in θ. This is what lets gradient-based methods optimize non-differentiable rewards.

4.3 The anatomy of a reinforcement learning algorithm

Every algorithm in the course has the three parts of Figure 4.

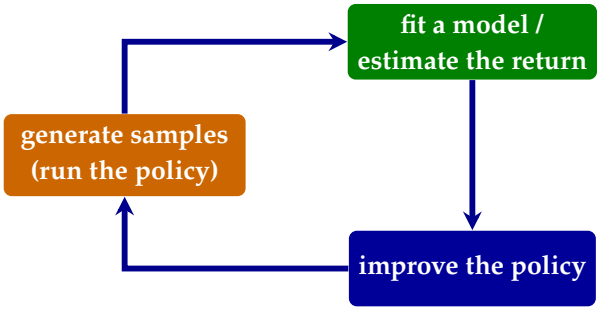


Figure 4: The anatomy of a reinforcement learning algorithm.

A simple example: generate samples, estimate the return $J(\theta) = \mathbb{E}_\pi\big[\sum_t r_t\big] \approx \frac{1}{N}\sum_{i=1}^{N}\sum_t r_t^i$, and improve the policy by $\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$. Another example, reinforcement learning by backpropagation: learn a model f_ϕ with $s_{t+1} \approx f_\phi(s_t, a_t)$, then backpropagate through f_ϕ and r to train a deterministic policy $\pi_\theta(s_t) = a_t$.

Which parts are expensive? Generating samples runs at 1× real time on a real robot, car, or power

grid (until we invent time travel), but up to 10000× real time in a simulator such as MuJoCo. Estimating the return by averaging is trivial and fast, while learning a model $s_{t+1} \approx f_\phi(s_t, a_t)$ is expensive; likewise a single gradient step is cheap, while backpropagating through f_ϕ and r is not.

4.4 Value functions

The objective is a nested expectation:

$$\mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\sum_{t=1}^{T} r(s_t, a_t)\Big] = \mathbb{E}_{s_1\sim p(s_1)}\Big[\mathbb{E}_{a_1\sim\pi(a_1|s_1)}\Big[r(s_1, a_1) + \mathbb{E}_{s_2\sim p(s_2|s_1,a_1)}\big[\mathbb{E}_{a_2\sim\pi(a_2|s_2)}[r(s_2, a_2) + \ldots \mid s_2] \,\big|\, s_1, a_1\big] \,\Big|\, s_1\Big]\Big].$$

If we knew the inner part, $Q(s_1, a_1) = r(s_1, a_1) + \mathbb{E}_{s_2\sim p(s_2|s_1,a_1)}\big[\mathbb{E}_{a_2\sim\pi(a_2|s_2)}[r(s_2, a_2) + \ldots \mid s_2] \mid s_1, a_1\big]$, the objective would be $\mathbb{E}_{s_1\sim p(s_1)}\big[\mathbb{E}_{a_1\sim\pi(a_1|s_1)}[Q(s_1, a_1) \mid s_1]\big]$, and it would be easy to modify $\pi_\theta(a_1 \mid s_1)$, for example by $\pi(a_1 \mid s_1) = 1$ if $a_1 = \arg\max_{a_1} Q(s_1, a_1)$.

Definition 4.3 [Q-function and value function]. The Q-function of a policy π is the total expected reward from taking a_t in s_t and following π afterwards,

$$Q^\pi(s_t, a_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\big[r(s_{t'}, a_{t'}) \mid s_t, a_t\big],$$

and the value function is the total expected reward from s_t,

$$V^\pi(s_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\big[r(s_{t'}, a_{t'}) \mid s_t\big] = \mathbb{E}_{a_t\sim\pi(a_t|s_t)}\big[Q^\pi(s_t, a_t)\big].$$

The reinforcement learning objective is $\mathbb{E}_{s_1\sim p(s_1)}[V^\pi(s_1)]$.

Two ideas recur throughout the course.

1. If we have a policy π and know $Q^\pi(s, a)$, we can improve π: set $\pi'(a \mid s) = 1$ if $a = \arg\max_a Q^\pi(s, a)$. This policy is at least as good as π (and probably better), whatever π is.
2. Compute a gradient to increase the probability of good actions. If $Q^\pi(s, a) > V^\pi(s)$ then a is better than average, since $V^\pi(s) = \mathbb{E}[Q^\pi(s, a)]$ under $\pi(a \mid s)$; modify $\pi(a \mid s)$ to increase the probability of such a.

The "fit a model / estimate the return" step often uses Q-functions or value functions.

4.5 Types of algorithms

All aim at $\theta^\star = \arg\max_\theta \mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\sum_t r(s_t, a_t)\right]$.

- Policy gradients directly differentiate this objective: evaluate returns $R_\tau = \sum_t r(s_t, a_t)$, then $\theta \leftarrow \theta + \alpha\nabla_\theta\mathbb{E}\left[\sum_t r(s_t, a_t)\right]$.
- Value-based methods estimate the value function or Q-function of the optimal policy, with no explicit policy: fit $V(s)$ or $Q(s, a)$ and set $\pi(s) = \arg\max_a Q(s, a)$.
- Actor-critic methods estimate the value function or Q-function of the current policy and use it to improve the policy: fit $V(s)$ or $Q(s, a)$, then $\theta \leftarrow \theta + \alpha\nabla_\theta\mathbb{E}\left[Q(s_t, a_t)\right]$.
- Model-based methods estimate the transition model $p(s_{t+1} \mid s_t, a_t)$ and then improve the policy in one of a few ways:
  1. just use the model to plan, with no policy: trajectory optimization or optimal control (primarily in continuous spaces, essentially backpropagation to optimize over actions), or discrete planning in discrete action spaces, such as Monte Carlo tree search;
  2. backpropagate gradients into the policy, which requires some tricks to make it work;
  3. use the model to learn a value function, by dynamic programming or by generating simulated experience for a model-free learner.

4.6 Tradeoffs between algorithms

There are so many algorithms because they make different tradeoffs (sample efficiency, stability and ease of use) and different assumptions (stochastic or deterministic, continuous or discrete, episodic or infinite horizon), and because different things are easy or hard in different settings: sometimes the policy is easier to represent, sometimes the model.

Sample efficiency is how many samples we need to get a good policy. The most important question is whether the algorithm is off-policy, able to improve the policy without generating new samples from that policy, or on-policy, needing new samples each time the policy changes, even a little bit (as after just one gradient step). Figure 5 orders the families. We might still use a less efficient algorithm because wall-clock time is not the same as efficiency: if simulation is cheap, the less efficient method may be faster.

Stability and ease of use. Does the algorithm converge, to what, and does it converge every time? These are questions because supervised learning is almost always gradient descent, while reinforcement learning often is not: Q-learning is fixed-point iteration, model-based RL fits a model that is not optimized for expected reward, and policy gradient is gradient descent but often the least efficient.

- Value function fitting at best minimizes the error of fit (the "Bellman error"), which is not the same as expected reward; at worst it does not optimize anything, since many popular deep RL

value fitting algorithms are not guaranteed to converge to anything in the nonlinear case.

- Model-based RL: the model minimizes error of fit and will converge, but there is no guarantee that a better model gives a better policy.
- Policy gradient is the only one that actually performs gradient descent (ascent) on the true objective.

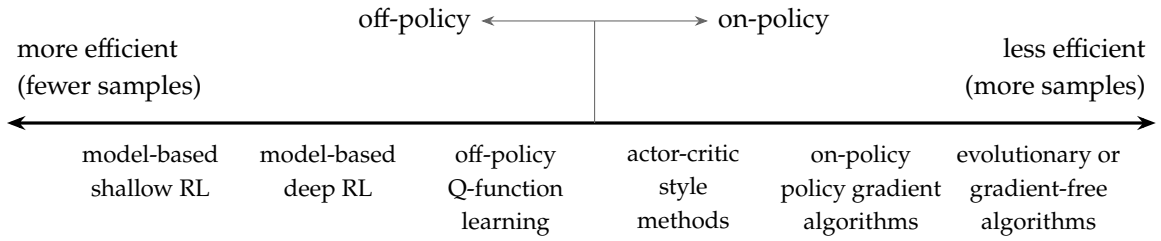


Figure 5: Sample efficiency of the algorithm families, from model-based (most efficient) to gradient-free (least efficient).

Assumptions. Full observability is generally assumed by value function fitting methods and can be mitigated by adding recurrence. Episodic learning is often assumed by pure policy gradient methods and by some model-based methods. Continuity or smoothness is assumed by some continuous value function learning methods and often by model-based methods.

Examples. Value function fitting: Q-learning and DQN, temporal difference learning, fitted value iteration. Policy gradients: REINFORCE, natural policy gradient, trust region policy optimization. Actor-critic: asynchronous advantage actor-critic (A3C), soft actor-critic (SAC). Model-based: Dyna, guided policy search. Applications include Atari games with Q-learning on convolutional networks, image-based robotic manipulation with guided policy search, simulated walking with trust region policy optimization and generalized advantage estimation, and real-world robotic grasping from images with Q-learning (QT-Opt).

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 4 (Fall 2023).

Lecture 5: Policy Gradients

5.1 Direct policy differentiation

Recall the objective: with $p_\theta(\tau) = p(s_1)\prod_{t=1}^{T}\pi_\theta(a_t \mid s_t)p(s_{t+1} \mid s_t, a_t)$,

$$\theta^\star = \arg\max_\theta J(\theta), \qquad J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\sum_t r(s_t, a_t)\Big] \approx \frac{1}{N}\sum_i\sum_t r(s_{i,t}, a_{i,t}),$$

where the estimate averages over N trajectories sampled from π_θ. (The infinite-horizon version maximizes $\mathbb{E}_{(s,a)\sim p_\theta(s,a)}[r(s,a)]$ and the finite-horizon version $\sum_{t=1}^{T}\mathbb{E}_{(s_t,a_t)\sim p_\theta(s_t,a_t)}[r(s_t,a_t)]$.) Write $r(\tau) = \sum_{t=1}^{T} r(s_t, a_t)$, so $J(\theta) = \int p_\theta(\tau)r(\tau)\,d\tau$.

Proposition 5.1 [Policy gradient]**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\Big(\sum_{t=1}^{T}\nabla_\theta \log\pi_\theta(a_t \mid s_t)\Big)\Big(\sum_{t=1}^{T} r(s_t, a_t)\Big)\Big].$$

Proof. A convenient identity is $p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) = p_\theta(\tau)\frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau)$. Hence

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau)r(\tau)\,d\tau = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)r(\tau)\,d\tau = \mathbb{E}_{\tau\sim p_\theta(\tau)}\big[\nabla_\theta \log p_\theta(\tau)r(\tau)\big].$$

Taking logs of both sides of the trajectory distribution, $\log p_\theta(\tau) = \log p(s_1) + \sum_{t=1}^{T}\big(\log\pi_\theta(a_t \mid s_t) + \log p(s_{t+1} \mid s_t, a_t)\big)$. The initial-state and dynamics terms do not depend on θ, so $\nabla_\theta \log p_\theta(\tau) = \sum_{t=1}^{T}\nabla_\theta \log\pi_\theta(a_t \mid s_t)$. □

The gradient needs no knowledge of the dynamics or the initial state distribution, only samples. Estimating it from N sampled trajectories and taking a gradient step gives the basic algorithm.

Algorithm 5.1: REINFORCE

```
repeat
    sample {τ^i} from π_θ(a_t | s_t) (run the policy)
    ∇_θ J(θ) ≈ Σ_i (Σ_t ∇_θ log π_θ(a_t^i | s_t^i)) (Σ_t r(s_t^i, a_t^i))
    θ ← θ + α ∇_θ J(θ)
until done
```

5.2 Understanding policy gradients

Compare the estimate

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\Big(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})\Big)\Big(\sum_{t=1}^{T} r(s_{i,t}, a_{i,t})\Big)$$

with the gradient of the maximum-likelihood (behavioral cloning) objective,

$$\nabla_\theta J_{\mathrm{ML}}(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\Big(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})\Big).$$

Writing $\nabla_\theta \log \pi_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})$, the policy gradient is $\frac{1}{N}\sum_i \nabla_\theta \log \pi_\theta(\tau_i) r(\tau_i)$: a likelihood gradient weighted by reward. Good trajectories are made more likely and bad ones less likely; the policy gradient simply formalizes the notion of trial and error.

Example: *Gaussian policies.* For continuous actions take $\pi_\theta(a_t \mid s_t) = \mathcal{N}\big(f_{\text{neural network}}(s_t); \Sigma\big)$. Then $\log \pi_\theta(a_t \mid s_t) = -\frac{1}{2}\|f(s_t) - a_t\|^2_\Sigma + \text{const}$ and

$$\nabla_\theta \log \pi_\theta(a_t \mid s_t) = -\Big(\frac{df}{d\theta}\Big)^T \Sigma^{-1}\big(f(s_t) - a_t\big),$$

which plugs into REINFORCE directly, for example to learn a walking gait on a simulated robot.

Partial observability. The derivation never used the Markov property, so the same estimator with $\pi_\theta(a_{i,t} \mid o_{i,t})$ in place of $\pi_\theta(a_{i,t} \mid s_{i,t})$ works in partially observed MDPs without modification.

What is wrong with the policy gradient? High variance. The estimate $\frac{1}{N}\sum_i \nabla_\theta \log \pi_\theta(\tau_i) r(\tau_i)$ from a few samples changes drastically if a constant is added to all rewards: with negative rewards on every sample, all sampled trajectories are pushed down and the distribution moves away from them; with large positive rewards, all are pushed up. Even worse, if the two "good" samples have $r(\tau) = 0$, they contribute nothing at all. With finite samples the update depends heavily on such arbitrary choices.

5.3 Reducing variance

Causality. The policy at time t' cannot affect the reward at time t when $t < t'$. Dropping the past rewards from each term gives

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})\,\hat{Q}_{i,t}, \qquad \hat{Q}_{i,t} = \sum_{t'=t}^{T} r(s_{i,t'}, a_{i,t'}),$$

where $\hat{Q}_{i,t}$ is the REWARD TO GO. The dropped terms have zero expectation (by the same argument as for baselines below, applied to each time step), so the estimator stays unbiased, and it sums fewer

random terms, so its variance is lower.

Baselines. Subtract a constant b from the return,

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \log p_\theta(\tau_i)\big[r(\tau_i) - b\big], \qquad b = \frac{1}{N}\sum_{i=1}^{N} r(\tau_i).$$

This is allowed: by the convenient identity,

$$\mathbb{E}\big[\nabla_\theta \log p_\theta(\tau) b\big] = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) b \, d\tau = \int \nabla_\theta p_\theta(\tau) b \, d\tau = b\nabla_\theta \int p_\theta(\tau)\, d\tau = b\nabla_\theta 1 = 0,$$

so subtracting a baseline is unbiased in expectation. The average reward is not the best baseline, but it is pretty good.

Proposition 5.2 [Optimal baseline]. *Write $g(\tau) = \nabla_\theta \log p_\theta(\tau)$ and treat one component of the gradient at a time. The baseline minimizing the variance of $g(\tau)(r(\tau) - b)$ is*

$$b = \frac{\mathbb{E}\big[g(\tau)^2 r(\tau)\big]}{\mathbb{E}\big[g(\tau)^2\big]},$$

the expected reward weighted by squared gradient magnitudes.

Proof. Since $\mathrm{Var}[x] = \mathbb{E}[x^2] - \mathbb{E}[x]^2$,

$$\mathrm{Var} = \mathbb{E}_{\tau\sim p_\theta(\tau)}\big[\big(g(\tau)(r(\tau) - b)\big)^2\big] - \mathbb{E}_{\tau\sim p_\theta(\tau)}\big[g(\tau)(r(\tau) - b)\big]^2.$$

The second term equals $\mathbb{E}[g(\tau)r(\tau)]^2$ because baselines are unbiased, so it does not depend on b. Then

$$\frac{d\,\mathrm{Var}}{db} = \frac{d}{db}\Big(\mathbb{E}[g(\tau)^2 r(\tau)^2] - 2\mathbb{E}[g(\tau)^2 r(\tau)]b + b^2\mathbb{E}[g(\tau)^2]\Big) = -2\mathbb{E}[g(\tau)^2 r(\tau)] + 2b\mathbb{E}[g(\tau)^2] = 0,$$

and the variance is convex in b, so this stationary point is the minimum. □

5.4 Off-policy policy gradients

The policy gradient is on-policy: $\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)]$ needs samples from the current policy, so the sampling step of REINFORCE cannot be skipped. Neural networks change only a little with each gradient step, so on-policy learning can be extremely inefficient.

If we have samples from some $\bar{p}(\tau)$ instead, use importance sampling:

$$\mathbb{E}_{x\sim p(x)}[f(x)] = \int p(x) f(x)\, dx = \int \frac{q(x)}{q(x)} p(x) f(x)\, dx = \mathbb{E}_{x\sim q(x)}\Big[\frac{p(x)}{q(x)} f(x)\Big].$$

Then $J(\theta) = \mathbb{E}_{\tau\sim\bar{p}(\tau)}\big[\frac{p_\theta(\tau)}{\bar{p}(\tau)} r(\tau)\big]$, and the initial-state and dynamics factors cancel in the ratio:

$$\frac{p_\theta(\tau)}{\bar{p}(\tau)} = \frac{p(s_1)\prod_{t=1}^{T}\pi_\theta(a_t \mid s_t)p(s_{t+1} \mid s_t, a_t)}{p(s_1)\prod_{t=1}^{T}\bar{\pi}(a_t \mid s_t)p(s_{t+1} \mid s_t, a_t)} = \frac{\prod_{t=1}^{T}\pi_\theta(a_t \mid s_t)}{\prod_{t=1}^{T}\bar{\pi}(a_t \mid s_t)}.$$

To estimate the value of new parameters θ' from samples of π_θ, write $J(\theta') = \mathbb{E}_{\tau\sim p_\theta(\tau)}\big[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)} r(\tau)\big]$, in which only $p_{\theta'}(\tau)$ depends on θ'. By the convenient identity,

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\frac{\nabla_{\theta'} p_{\theta'}(\tau)}{p_\theta(\tau)} r(\tau)\Big] = \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)} \nabla_{\theta'} \log p_{\theta'}(\tau) r(\tau)\Big].$$

Estimated locally at $\theta = \theta'$ this recovers the on-policy gradient $\mathbb{E}_{\tau\sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)]$. When $\theta \neq \theta'$,

$$\begin{aligned}
\nabla_{\theta'} J(\theta') &= \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\Big(\prod_{t=1}^{T}\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}\Big)\Big(\sum_{t=1}^{T}\nabla_{\theta'}\log\pi_{\theta'}(a_t \mid s_t)\Big)\Big(\sum_{t=1}^{T} r(s_t, a_t)\Big)\Big] \\
&= \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\sum_{t=1}^{T}\nabla_{\theta'}\log\pi_{\theta'}(a_t \mid s_t)\Big(\prod_{t'=1}^{t}\frac{\pi_{\theta'}(a_{t'} \mid s_{t'})}{\pi_\theta(a_{t'} \mid s_{t'})}\Big)\Big(\sum_{t'=t}^{T} r(s_{t'}, a_{t'})\Big(\prod_{t''=t}^{t'}\frac{\pi_{\theta'}(a_{t''} \mid s_{t''})}{\pi_\theta(a_{t''} \mid s_{t''})}\Big)\Big)\Big],
\end{aligned}$$

using causality twice: future actions do not affect the current weight, and the reward at t' depends only on the ratios up to t'. Ignoring the last product gives a policy iteration algorithm (revisited in a later lecture). The remaining product of ratios is exponential in T: each factor below one shrinks it, so the weights vanish or explode.

A first-order approximation. Write the objective a bit differently, as a sum over time steps with $(s_{i,t}, a_{i,t}) \sim \pi_\theta(s_t, a_t)$ (the state-action marginal). On-policy, $\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_i \sum_t \nabla_\theta \log\pi_\theta(a_{i,t} \mid s_{i,t})\hat{Q}_{i,t}$; off-policy,

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\frac{\pi_{\theta'}(s_{i,t})}{\pi_\theta(s_{i,t})}\frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})}\nabla_{\theta'}\log\pi_{\theta'}(a_{i,t} \mid s_{i,t})\hat{Q}_{i,t}.$$

Ignoring the state-marginal ratio $\pi_{\theta'}(s_{i,t})/\pi_\theta(s_{i,t})$ leaves only a per-step action ratio; why this is reasonable is shown later in the course (Lecture 9).

5.5 Implementing policy gradients

Computing every $\nabla_\theta \log\pi_\theta(a_{i,t} \mid s_{i,t})$ explicitly is inefficient. Instead, build a graph whose gradient is the policy gradient. Maximum likelihood uses $J_{\mathrm{ML}}(\theta) \approx \frac{1}{N}\sum_i \sum_t \log\pi_\theta(a_{i,t} \mid s_{i,t})$; so implement a "pseudo-loss" as a weighted maximum likelihood,

$$\tilde{J}(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\log\pi_\theta(a_{i,t} \mid s_{i,t})\hat{Q}_{i,t},$$

with cross entropy for discrete actions or squared error for Gaussian ones. With discrete actions:

```
# Given:
# actions - (N*T) x Da tensor of actions
# states - (N*T) x Ds tensor of states
# q_values - (N*T) x 1 tensor of estimated state-action values
# Build the graph:
logits = policy.predictions(states)  # (N*T) x Da tensor of action logits
negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(
    labels=actions, logits=logits)
weighted_negative_likelihoods = tf.multiply(negative_likelihoods, q_values)
loss = tf.reduce_mean(weighted_negative_likelihoods)
gradients = tf.gradients(loss, variables)
```

Without the multiplication by `q_values` this is exactly the maximum-likelihood graph. In practice, remember that the gradient has high variance: unlike supervised learning, gradients are really noisy, so use much larger batches. Tweaking learning rates is very hard; adaptive step size rules like Adam can be OK-ish, and policy-gradient-specific step size methods come later.

5.6 Natural policy gradient

What else is wrong with the policy gradient? Consider a one-dimensional problem with $r(s_t, a_t) = -s_t^2 - a_t^2$ and the Gaussian policy $\log \pi_\theta(a_t \mid s_t) = -\frac{1}{2\sigma^2}(ks_t - a_t)^2 + \text{const}$ with $\theta = (k, \sigma)$. As σ shrinks, the gradient with respect to σ dominates the gradient with respect to k, so the vanilla gradient mostly reduces exploration and converges slowly to the optimal gain. This is the same problem as gradient descent on a poorly conditioned quadratic: some parameters change probabilities a lot more than others.

A gradient step $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ solves

$$\theta' \leftarrow \arg\max_{\theta'} (\theta' - \theta)^T \nabla_\theta J(\theta) \quad \text{s.t.} \quad \|\theta' - \theta\|^2 \le \epsilon,$$

where ϵ controls how far we go, in parameter space. Rescale the gradient by constraining a parameterization-independent divergence instead, $D(\pi_{\theta'}, \pi_\theta) \le \epsilon$, usually the KL divergence $D_{\mathrm{KL}}(\pi_{\theta'} \| \pi_\theta) = \mathbb{E}_{\pi_{\theta'}}[\log \pi_{\theta'} - \log \pi_\theta]$. To second order,

$$D_{\mathrm{KL}}(\pi_{\theta'} \| \pi_\theta) \approx \tfrac{1}{2}(\theta' - \theta)^T F (\theta' - \theta), \qquad F = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)^T\right],$$

where F is the Fisher information matrix, which can be estimated with samples. Solving the constrained problem with this quadratic constraint gives the natural gradient

$$\theta \leftarrow \theta + \alpha F^{-1} \nabla_\theta J(\theta).$$

The natural gradient method picks α; TRUST REGION POLICY OPTIMIZATION (TRPO) picks ϵ and solves for the optimal α while computing $F^{-1}\nabla_\theta J(\theta)$, for which conjugate gradient works well. In the example above, the natural gradient points toward the optimum rather than mostly reducing σ.

Examples include importance-sampled policy gradients that incorporate example demonstrations to train neural network locomotion policies, and TRPO (natural gradient with automatic step adjustment) for discrete and continuous actions.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 5 (Fall 2023).

Lecture 6: Actor-Critic Algorithms

6.1 Improving the policy gradient

REINFORCE with reward to go estimates

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})\hat{Q}^\pi_{i,t}, \qquad \hat{Q}^\pi_{i,t} = \sum_{t'=t}^{T} r(s_{i,t'}, a_{i,t'}).$$

Here $\hat{Q}_{i,t}$ estimates the expected reward if we take $a_{i,t}$ in $s_{i,t}$, but it is a single-sample estimate: many different futures could branch from $s_{i,t}$. A better estimate is the true expected reward to go, $Q(s_t, a_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t, a_t]$, giving $\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) Q(s_{i,t}, a_{i,t})$.

For the baseline, average the Q-values, $b_t = \frac{1}{N}\sum_i Q(s_{i,t}, a_{i,t})$: averaging over actions in the state gives the value function $V(s_t) = \mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)}[Q(s_t, a_t)]$, so

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})\big(Q(s_{i,t}, a_{i,t}) - V(s_{i,t})\big).$$

Definition 6.1 [Advantage]. With $Q^\pi(s_t, a_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t, a_t]$ (total reward from taking a_t in s_t) and $V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)}[Q^\pi(s_t, a_t)]$ (total reward from s_t), the advantage

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

measures how much better a_t is than the policy's average action.

The policy gradient becomes $\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) A^\pi(s_{i,t}, a_{i,t})$, and the better the estimate of A^π, the lower the variance. The Monte Carlo version, $\sum_{t'} r(s_{i,t'}, a_{i,t'}) - b$, is unbiased but a high-variance single-sample estimate.

6.2 Value function fitting and policy evaluation

Which of Q^π, V^π, A^π should we fit? Since the first reward does not depend on the future,

$$\begin{aligned} Q^\pi(s_t, a_t) &= r(s_t, a_t) + \sum_{t'=t+1}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t, a_t] \\ &= r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t)}\big[V^\pi(s_{t+1})\big] \approx r(s_t, a_t) + V^\pi(s_{t+1}), \end{aligned}$$

using the single observed next state, and therefore

$$A^\pi(s_t, a_t) \approx r(s_t, a_t) + V^\pi(s_{t+1}) - V^\pi(s_t).$$

So it suffices to fit $V^\pi(s)$, a function of the state alone, with a neural network $\hat{V}^\pi_\phi(s)$ with parameters ϕ.

Fitting V^π is POLICY EVALUATION: $V^\pi(s_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t]$ and $J(\theta) = \mathbb{E}_{s_1 \sim p(s_1)}[V^\pi(s_1)]$. MONTE CARLO POLICY EVALUATION (what the policy gradient does) uses $V^\pi(s_t) \approx \sum_{t'=t}^{T} r(s_{t'}, a_{t'})$, or averages N rollouts from s_t, which would require resetting the simulator to s_t. With function approximation, a single-sample target is not as good as the average but still pretty good, because the same function must fit multiple samples from nearby states and so generalizes across them. Train on $\left\{\left(s_{i,t},\ y_{i,t} = \sum_{t'=t}^{T} r(s_{i,t'}, a_{i,t'})\right)\right\}$ by supervised regression,

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}^\pi_\phi(s_i) - y_i \right\|^2.$$

We can do better. The ideal target is

$$y_{i,t} = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_{i,t}] \approx r(s_{i,t}, a_{i,t}) + V^\pi(s_{i,t+1}) \approx r(s_{i,t}, a_{i,t}) + \hat{V}^\pi_\phi(s_{i,t+1}),$$

which directly uses the previous fitted value function. Training on $\left\{\left(s_{i,t},\ r(s_{i,t}, a_{i,t}) + \hat{V}^\pi_\phi(s_{i,t+1})\right)\right\}$ with the same regression loss is a BOOTSTRAPPED estimate: lower variance than the Monte Carlo target, but biased when $\hat{V}^\pi_\phi$ is wrong. TD-Gammon (backgammon) and AlphaGo are examples of policy evaluation: the reward is the game outcome and $\hat{V}^\pi_\phi(s_t)$ is the expected outcome given the board state.

6.3 From evaluation to actor-critic

Algorithm 6.1: Batch actor-critic

repeat
  sample $\{s_i, a_i\}$ from $\pi_\theta(a \mid s)$ (run it on the robot)
  fit $\hat{V}^\pi_\phi(s)$ to sampled reward sums
  evaluate $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \hat{V}^\pi_\phi(s'_i) - \hat{V}^\pi_\phi(s_i)$
  $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i \mid s_i) \hat{A}^\pi(s_i, a_i)$
  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
until done

The policy is the ACTOR and the value function the CRITIC.

Discount factors. With bootstrapped targets $y_{i,t} \approx r(s_{i,t}, a_{i,t}) + \hat{V}^\pi_\phi(s_{i,t+1})$, what if the episode length

T is ∞? Then $\hat{V}_\phi^\pi$ can get infinitely large in many cases, for continuous or cyclical tasks unlike episodic ones. The simple trick is that it is better to get rewards sooner than later:

$$y_{i,t} \approx r(s_{i,t}, a_{i,t}) + \gamma \hat{V}_\phi^\pi(s_{i,t+1}),$$

with a DISCOUNT FACTOR $\gamma \in [0, 1]$ (0.99 works well). Discounting changes the MDP: it is equivalent to adding a "death" state reached with probability $1 - \gamma$ at every step, so that $\tilde{p}(s' \mid s, a) = \gamma p(s' \mid s, a)$ (Figure 6).

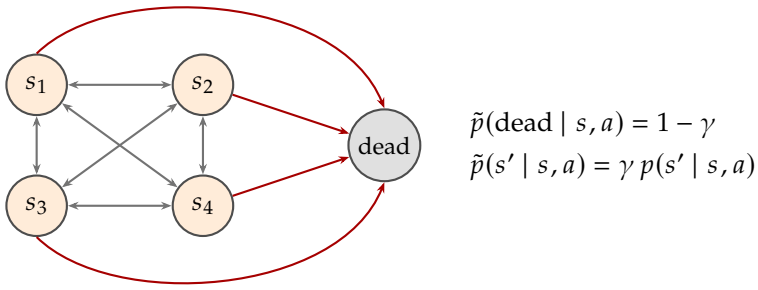


Figure 6: A discount factor γ as a death state entered with probability $1 - \gamma$ from every state.

With a critic, $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(r(s_{i,t}, a_{i,t}) + \gamma \hat{V}_\phi^\pi(s_{i,t+1}) - \hat{V}_\phi^\pi(s_{i,t})\big)$. For Monte Carlo policy gradients there are two options, and they are not the same:

$$\text{option 1:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \Big(\sum_{t'=t}^{T} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'}) \Big),$$

$$\text{option 2:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \Big(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \Big) \Big(\sum_{t=1}^{T} \gamma^{t-1} r(s_{i,t}, a_{i,t}) \Big)$$

$$\approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \gamma^{t-1} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \Big(\sum_{t'=t}^{T} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'}) \Big).$$

Option 2, the true gradient of the discounted objective (after applying causality, $\gamma^{t'-1} = \gamma^{t-1}\gamma^{t'-t}$), down-weights later steps: later steps do not matter if you are dead. Option 1 is what is actually used, because we usually want a policy that does well at every time step of a long or cyclical task; there the discount serves as a variance reduction device rather than as part of the objective.

With discount, batch actor-critic is Algorithm 6.1 with the advantage $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \gamma \hat{V}_\phi^\pi(s_i') - \hat{V}_\phi^\pi(s_i)$. Bootstrapped targets also allow a fully online version that updates after every transition.

Algorithm 6.2: Online actor-critic

repeat
  take action $a \sim \pi_\theta(a \mid s)$, get (s, a, s', r)
  update $\hat{V}^\pi_\phi$ using target $r + \gamma \hat{V}^\pi_\phi(s')$
  evaluate $\hat{A}^\pi(s, a) = r(s, a) + \gamma \hat{V}^\pi_\phi(s') - \hat{V}^\pi_\phi(s)$
  $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(a \mid s) \hat{A}^\pi(s, a)$
  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
until done

6.4 Actor-critic design decisions

Architecture. A two-network design, one network for $\hat{V}^\pi_\phi(s)$ and one for $\pi_\theta(a \mid s)$, is simple and stable but shares no features between actor and critic. A shared network with two heads shares features but is harder to train.

Parallelism. The online algorithm's updates use one sample and work best with a batch, for example from parallel workers. In synchronized parallel actor-critic, all workers collect (s, a, s', r) and then update θ together; in asynchronous parallel actor-critic, each worker updates shared parameters as soon as it is ready.

Removing the on-policy assumption. Instead, form a batch from old, previously seen transitions stored in a REPLAY BUFFER $\mathcal{R}$: off-policy actor-critic. Naively:

1. take action $a \sim \pi_\theta(a \mid s)$, get (s, a, s', r), store in $\mathcal{R}$;
2. sample a batch $\{s_i, a_i, r_i, s'_i\}$ from $\mathcal{R}$;
3. update $\hat{V}^\pi_\phi$ using targets $y_i = r_i + \gamma \hat{V}^\pi_\phi(s'_i)$ for each s_i, with $\mathcal{L}(\phi) = \frac{1}{N} \sum_i \| \hat{V}^\pi_\phi(s_i) - y_i \|^2$;
4. evaluate $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \gamma \hat{V}^\pi_\phi(s'_i) - \hat{V}^\pi_\phi(s_i)$;
5. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i \mid s_i) \hat{A}^\pi(s_i, a_i)$;
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

This algorithm is broken. The action a_i in the buffer came from an old policy, so $r_i + \gamma \hat{V}^\pi_\phi(s'_i)$ is not the right target value for the current policy, and $\nabla_\theta \log \pi_\theta(a_i \mid s_i)$ is evaluated at an action π_θ would not have taken.

Fixing the value function. Learn $Q^\pi(s_t, a_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t, a_t]$, the total reward if we take a_t in s_t and then follow π, which is valid for any action. Update $\hat{Q}^\pi_\phi$ using targets

$$y_i = r_i + \gamma \hat{V}^\pi_\phi(s'_i) = r_i + \gamma \hat{Q}^\pi_\phi(s'_i, a'_i), \qquad a'_i \sim \pi_\theta(a'_i \mid s'_i),$$

with $\mathcal{L}(\phi) = \frac{1}{N} \sum_i \| \hat{Q}^\pi_\phi(s_i, a_i) - y_i \|^2$. The next action a'_i is sampled from the current policy, not

taken from the replay buffer, since $V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi(a_t|s_t)}[Q(s_t, a_t)]$.

Fixing the policy update. Use the same trick, this time for a_i rather than a'_i: sample $a_i^\pi \sim \pi_\theta(a \mid s_i)$ (not from the buffer) and use $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i^\pi \mid s_i) \hat{A}^\pi(s_i, a_i^\pi)$. In practice, the higher-variance but convenient

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i^\pi \mid s_i) \hat{Q}^\pi(s_i, a_i^\pi)$$

is used. Higher variance is acceptable here because no more environment interaction is needed to reduce it: we can sample as many actions a_i^π as we like.

Algorithm 6.3: Off-policy online actor-critic

repeat
  take action $a \sim \pi_\theta(a \mid s)$, get (s, a, s', r), store in $\mathcal{R}$
  sample a batch $\{s_i, a_i, r_i, s'_i\}$ from buffer $\mathcal{R}$
  update $\hat{Q}^\pi_\phi$ using targets $y_i = r_i + \gamma \hat{Q}^\pi_\phi(s'_i, a'_i)$, $a'_i \sim \pi_\theta(a \mid s'_i)$, for each s_i, a_i
  $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i^\pi \mid s_i) \hat{Q}^\pi(s_i, a_i^\pi)$ where $a_i^\pi \sim \pi_\theta(a \mid s_i)$
  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
until done

One problem remains: s_i did not come from $p_\theta(s)$. Nothing can be done about this, so accept it. Intuitively, we want the optimal policy on $p_\theta(s)$, but we get the optimal policy on a broader distribution. There are fancier ways to fit Q-functions (the next two lectures), and the policy update could use the reparameterization trick to estimate the integral better. Soft actor-critic (SAC) is a practical algorithm of this form; algorithms that do this with deterministic policies come later.

6.5 Critics as baselines

Compare the actor-critic and Monte Carlo estimators:

$$\text{actor-critic:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(r(s_{i,t}, a_{i,t}) + \gamma \hat{V}^\pi_\phi(s_{i,t+1}) - \hat{V}^\pi_\phi(s_{i,t})\big),$$

$$\text{policy gradient:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \Big(\Big(\sum_{t'=t}^{T} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'})\Big) - b\Big).$$

The first has lower variance (due to the critic) but is not unbiased if the critic is not perfect; the second has no bias but higher variance (single-sample estimate). A STATE-DEPENDENT BASELINE keeps the estimator unbiased while using the critic:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \Big(\Big(\sum_{t'=t}^{T} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'})\Big) - \hat{V}^\pi_\phi(s_{i,t})\Big),$$

which has no bias (a baseline depending only on the state integrates to zero against $\nabla_\theta \pi_\theta(a \mid s)$) and lower variance, since the baseline is closer to the rewards.

Control variates: action-dependent baselines. The advantage estimate $\hat{A}^\pi(s_t, a_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) - V_\phi^\pi(s_t)$ has no bias but higher variance; $\hat{A}^\pi(s_t, a_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) - Q_\phi^\pi(s_t, a_t)$ goes to zero in expectation if the critic is correct, but is not correct as a gradient estimator on its own. Adding back the expectation of the subtracted term fixes this:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(\hat{Q}_{i,t} - Q_\phi^\pi(s_{i,t}, a_{i,t})\big) + \frac{1}{N} \sum_{i,t} \nabla_\theta \mathbb{E}_{a \sim \pi_\theta(a_t \mid s_{i,t})} \big[Q_\phi^\pi(s_{i,t}, a_t)\big].$$

This uses a critic without the bias (still unbiased), provided the second term can be evaluated, for example in closed form for Gaussian policies and quadratic critics (Q-Prop).

6.6 Eligibility traces and n-step returns

The two extremes are

$$\hat{A}_{\mathrm{C}}^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \hat{V}_\phi^\pi(s_{t+1}) - \hat{V}_\phi^\pi(s_t), \qquad \hat{A}_{\mathrm{MC}}^\pi(s_t, a_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t).$$

The critic estimate has lower variance but higher bias if the value is wrong (it always is); the Monte Carlo estimate has no bias but higher variance (single-sample estimate). Variance grows with the distance into the future, while the near future is fairly predictable. Cut the sum before the variance gets too big and bootstrap from the critic there: the n-step return estimator

$$\hat{A}_n^\pi(s_t, a_t) = \sum_{t'=t}^{t+n-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t) + \gamma^n \hat{V}_\phi^\pi(s_{t+n})$$

controls the bias-variance tradeoff, and choosing $n > 1$ often works better.

Generalized advantage estimation. Rather than choosing one n, cut everywhere all at once with a weighted combination of n-step returns, $\hat{A}_{\mathrm{GAE}}^\pi(s_t, a_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(s_t, a_t)$. Mostly prefer cutting earlier (less variance), with exponential falloff $w_n \propto \lambda^{n-1}$, normalized as $w_n = (1-\lambda)\lambda^{n-1}$. Then

$$\begin{aligned}\hat{A}_{\mathrm{GAE}}^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \Big(&(1-\lambda) \hat{V}_\phi^\pi(s_{t+1}) \\ &+ \lambda \big(r(s_{t+1}, a_{t+1}) + \gamma \big((1-\lambda) \hat{V}_\phi^\pi(s_{t+2}) + \lambda r(s_{t+2}, a_{t+2}) + \ldots \big) \big) \Big) - \hat{V}_\phi^\pi(s_t),\end{aligned}$$

which telescopes into

$$\hat{A}_{\mathrm{GAE}}^\pi(s_t, a_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'}, \qquad \delta_{t'} = r(s_{t'}, a_{t'}) + \gamma \hat{V}_\phi^\pi(s_{t'+1}) - \hat{V}_\phi^\pi(s_{t'}).$$

The factor λ has a similar effect to the discount, which matches option 1 above: the discount is a variance reduction device.

Examples: batch-mode actor-critic with GAE for high-dimensional continuous control, blending Monte Carlo and function-approximator estimators; and asynchronous advantage actor-critic (A3C), an online actor-critic with a parallelized batch, n-step returns with $n = 4$, and a single network for actor and critic.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 6 (Fall 2023).

Lecture 7: Value Function Methods

7.1 Can we omit the policy gradient?

Batch actor-critic fits $\hat{V}^\pi_\phi$ and takes a policy gradient step using $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \hat{V}^\pi_\phi(s'_i) - \hat{V}^\pi_\phi(s_i)$. But $A^\pi(s_t, a_t)$ already says how much better a_t is than the average action according to π, so $\arg\max_{a_t} A^\pi(s_t, a_t)$ is the best action from s_t if we then follow π. It is at least as good as any $a_t \sim \pi(a_t \mid s_t)$, regardless of what $\pi(a_t \mid s_t)$ is. Forget policies and just do this:

$$\pi'(a_t \mid s_t) = \begin{cases} 1 & \text{if } a_t = \arg\max_{a_t} A^\pi(s_t, a_t), \\ 0 & \text{otherwise,} \end{cases}$$

which is at least as good as π (and probably better).

Algorithm 7.1: Policy iteration

```
repeat
    evaluate A^π(s, a)
    set π ← π'
until converged
```

As before, $A^\pi(s, a) = r(s, a) + \gamma \mathbb{E}[V^\pi(s')] - V^\pi(s)$, so it suffices to evaluate $V^\pi(s)$.

7.2 Dynamic programming

Assume we know $p(s' \mid s, a)$ and that s and a are both discrete and small, for example a 4×4 grid with 16 states and 4 actions per state. Then the full $V^\pi(s)$ can be stored in a table, and $\mathcal{T}$ is a $16 \times 16 \times 4$ tensor. The bootstrapped update

$$V^\pi(s) \leftarrow \mathbb{E}_{a \sim \pi(a|s)}\left[r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s,a)}[V^\pi(s')]\right]$$

just uses the current estimate on the right. Since π' is a deterministic policy $\pi(s) = a$, this simplifies to $V^\pi(s) \leftarrow r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim p(s'|s,\pi(s))}[V^\pi(s')]$. Policy iteration with dynamic programming alternates this policy evaluation (repeated to convergence) with setting $\pi \leftarrow \pi'$.

Even simpler. Since $\arg\max_{a_t} A^\pi(s_t, a_t) = \arg\max_{a_t} Q^\pi(s_t, a_t)$ with $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}[V^\pi(s')]$ (a bit simpler), the policy is just $\arg\max_a Q(s, a)$, and taking the max over the table of $Q(s, a)$ approximates the new value. Skip the policy and compute values directly.

Algorithm 7.2: Value iteration

repeat
  set $Q(s,a) \leftarrow r(s,a) + \gamma \mathbb{E}[V(s')]$
  set $V(s) \leftarrow \max_a Q(s,a)$
until converged

7.3 Fitted value iteration and Q-iteration

A big table with one entry for each discrete s suffers from the CURSE OF DIMENSIONALITY: for a 200×200 color image, $|\mathcal{S}| = (255^3)^{200 \times 200}$, more than the number of atoms in the universe. Instead use a neural network $V_\phi : \mathcal{S} \to \mathbb{R}$ with parameters ϕ, trained with $\mathcal{L}(\phi) = \frac{1}{2}\left\|V_\phi(s) - \max_a Q^\pi(s,a)\right\|^2$.

Algorithm 7.3: Fitted value iteration

repeat
  set $y_i \leftarrow \max_{a_i}\left(r(s_i,a_i) + \gamma \mathbb{E}[V_\phi(s_i')]\right)$
  set $\phi \leftarrow \arg\min_\phi \frac{1}{2}\sum_i \|V_\phi(s_i) - y_i\|^2$
until converged

Computing the target needs the outcomes of different actions from the same state, so it needs the transition dynamics. Back to policy iteration: its evaluation step can instead fit the Q-function,

$$Q^\pi(s,a) \leftarrow r(s,a) + \gamma \mathbb{E}_{s' \sim p(s'|s,a)}\left[Q^\pi(s', \pi(s'))\right],$$

which can be fit using samples. Doing the "max" trick again with Q-values, we approximate $\mathbb{E}[V(s_i')] \approx \max_{a'} Q_\phi(s_i', a_i')$, which does not require simulating actions.

Algorithm 7.4: Full fitted Q-iteration

Input: dataset size N, collection policy, iterations K, gradient steps S
repeat
  collect dataset $\{(s_i, a_i, s_i', r_i)\}$ using some policy
  for K iterations **do**
    set $y_i \leftarrow r(s_i,a_i) + \gamma \max_{a_i'} Q_\phi(s_i', a_i')$
    set $\phi \leftarrow \arg\min_\phi \frac{1}{2}\sum_i \|Q_\phi(s_i,a_i) - y_i\|^2$ (with S gradient steps)
until done

This works even for off-policy samples (unlike actor-critic), uses only one network, and avoids the high-variance policy gradient; but it has no convergence guarantees for nonlinear function approximation. Value-based methods do not learn a policy explicitly: if we have the Q-function, we have the policy $a = \arg\max_a Q_\phi(s,a)$.

Why is it off-policy? Given s and a, the transition is independent of π, and the max approximates the value of the greedy policy π' at s_i'. So fitted Q-iteration can use any dataset of transitions.

What is it optimizing? The regression step minimizes the Bellman error

$$\mathcal{E} = \frac{1}{2}\mathbb{E}_{(s,a)\sim\beta}\left[\left(Q_\phi(s,a) - \left[r(s,a) + \gamma \max_{a'} Q_\phi(s',a')\right]\right)^2\right],$$

where β is the data distribution, and the max in the target improves the policy (in the tabular case). If $\mathcal{E} = 0$ then $Q_\phi(s,a) = r(s,a) + \gamma \max_{a'} Q_\phi(s',a')$: this is an optimal Q-function, sometimes written $Q^\star$, corresponding to the optimal policy $\pi^\star(a_t \mid s_t) = 1$ if $a_t = \arg\max_{a_t} Q_\phi(s_t,a_t)$, which maximizes reward. Most guarantees are lost when we leave the tabular case, for example with neural networks.

7.4 From Q-iteration to Q-learning

The online analogue of fitted Q-iteration takes one step on one transition.

Algorithm 7.5: Online Q-learning

repeat
  take some action a_i and observe (s_i, a_i, s_i', r_i)
  $y_i = r(s_i,a_i) + \gamma \max_{a'} Q_\phi(s_i',a_i')$
  $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(s_i,a_i)\big(Q_\phi(s_i,a_i) - y_i\big)$
until done

Being off-policy, the action-selection step has many choices. The final policy is the greedy $\pi(a_t \mid s_t) = 1$ if $a_t = \arg\max_{a_t} Q_\phi(s_t,a_t)$, but this is a bad idea for selecting actions during learning: a deterministic greedy policy never tries actions whose values are underestimated, so it explores poorly. Two standard EXPLORATION rules are EPSILON-GREEDY,

$$\pi(a_t \mid s_t) = \begin{cases} 1-\epsilon & \text{if } a_t = \arg\max_{a_t} Q_\phi(s_t,a_t), \\ \epsilon/(|\mathcal{A}|-1) & \text{otherwise,} \end{cases}$$

and BOLTZMANN EXPLORATION, $\pi(a_t \mid s_t) \propto \exp\big(Q_\phi(s_t,a_t)\big)$. Exploration is treated in detail in Lectures 13 and 14.

7.5 Value functions in theory

Does value iteration converge, and if so, to what? Define the BELLMAN BACKUP operator $\mathcal{B}$ by

$$\mathcal{B}V = \max_a\big(r_a + \gamma \mathcal{T}_a V\big),$$

where r_a is the stacked vector of rewards at all states for action a, $\mathcal{T}_a$ is the matrix of transitions for action a with $\mathcal{T}_{a,i,j} = p(s' = i \mid s = j, a)$, and the max is elementwise. The optimal value $V^\star(s) = \max_a r(s,a) + \gamma\mathbb{E}[V^\star(s')]$ is a FIXED POINT of $\mathcal{B}$, $V^\star = \mathcal{B}V^\star$. It always exists, is always unique,

and always corresponds to the optimal policy. But will we reach it?

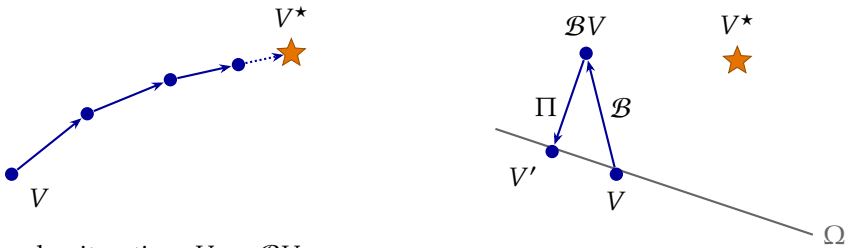
Proposition 7.1 [Value iteration converges]. *$\mathcal{B}$ is a contraction in the ∞-norm: for any V and $\bar{V}$, $\|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty \le \gamma\|V - \bar{V}\|_\infty$. Hence value iteration $V \leftarrow \mathcal{B}V$ converges to $V^\star$.*

Proof. For each state, the difference of two maxima is at most the largest difference, $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$. Here $f(a) - g(a) = \gamma\big(\mathcal{T}_a(V - \bar{V})\big)(s)$, an average of entries of $V - \bar{V}$ under a probability distribution, so its magnitude is at most $\gamma\|V - \bar{V}\|_\infty$. Choosing $\bar{V} = V^\star$ and using $\mathcal{B}V^\star = V^\star$ gives $\|\mathcal{B}V - V^\star\|_\infty \le \gamma\|V - V^\star\|_\infty$: the gap always gets smaller by a factor γ. □

Non-tabular value function learning. Define a second operator, the projection onto a set Ω of representable value functions (for example neural networks) in the ℓ_2 norm,

$$\Pi V = \arg\min_{V' \in \Omega} \frac{1}{2} \sum \|V'(s) - V(s)\|^2.$$

The regression step of fitted value iteration computes $V' \leftarrow \arg\min_{V' \in \Omega} \frac{1}{2} \sum \|V'(s) - (\mathcal{B}V)(s)\|^2$, so fitted value iteration is $V \leftarrow \Pi\mathcal{B}V$, while value iteration is $V \leftarrow \mathcal{B}V$. Now $\mathcal{B}$ is a contraction in the ∞-norm (the max norm), and Π is a contraction in the ℓ_2-norm (Euclidean distance), $\|\Pi V - \Pi\bar{V}\|^2 \le \|V - \bar{V}\|^2$; but $\Pi\mathcal{B}$ is not a contraction of any kind (Figure 7). Value iteration converges in the tabular case; fitted value iteration does not converge in general, and often not in practice.



value iteration: $V \leftarrow \mathcal{B}V$
converges (tabular)

fitted value iteration: $V \leftarrow \Pi\mathcal{B}V$
need not converge

Figure 7: Left: tabular value iteration contracts toward $V^\star$. Right: the backup $\mathcal{B}V$ can leave the representable set Ω, and projecting back gives V', which may be farther from $V^\star$. Schematic.

Fitted Q-iteration. With $\mathcal{B}Q = r + \gamma\mathcal{T}\max_a Q$ (the max now after the transition operator) and $\Pi Q = \arg\min_{Q' \in \Omega} \frac{1}{2} \sum \|Q'(s,a) - Q(s,a)\|^2$, fitted Q-iteration is $Q \leftarrow \Pi\mathcal{B}Q$, with the same conclusion. This applies also to online Q-learning. Its update looks like gradient descent, which converges, but Q-learning is not gradient descent: in

$$\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(s_i, a_i)\Big(Q_\phi(s_i, a_i) - \big[r(s_i, a_i) + \gamma \max_{a'} Q_\phi(s_i', a_i')\big]\Big)$$

no gradient flows through the target value.

A sad corollary. Batch actor-critic fits $\hat{V}^\pi_\phi$ to $y_{i,t} \approx r(s_{i,t}, a_{i,t}) + \gamma \hat{V}^\pi_\phi(s_{i,t+1})$ with $\mathcal{L}(\phi) = \frac{1}{2}\sum_i \|\hat{V}^\pi_\phi(s_i) - y_i\|^2$: an ℓ_∞ contraction $\mathcal{B}$ (without the max) followed by an ℓ_2 contraction Π. So fitted bootstrapped policy evaluation does not converge either. On terminology: V^π is the value function for policy π, which is what the critic computes; $V^\star$ is the value function of the optimal policy $\pi^\star$, which is what value iteration computes. Q-learning and fitted Q-iteration do not converge with function approximation in general, but they can be made to work in practice, as the next lecture shows.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 7 (Fall 2023).

Lecture 8: Deep RL with Q-Functions

8.1 Replay buffers

Online Q-learning (Lecture 7) takes some action a_i, observes (s_i, a_i, s'_i, r_i), sets $y_i = r(s_i, a_i) + \gamma \max_{a'} Q_\phi(s'_i, a'_i)$, and updates

$$\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(s_i, a_i)\big(Q_\phi(s_i, a_i) - [r(s_i, a_i) + \gamma \max_{a'} Q_\phi(s'_i, a'_i)]\big).$$

Two things are wrong. Q-learning is not gradient descent, since there is no gradient through the target value. And sequential states are strongly correlated while the target value is always changing, so the network overfits locally to whatever it saw most recently. Parallel workers (synchronized or asynchronous, as in parallel actor-critic) partly decorrelate the samples.

Another solution is a replay buffer $\mathcal{B}$. Online Q-learning is the special case of fitted Q-iteration with $K = 1$ and one gradient step. In fitted Q-iteration any policy with broad support works for data collection, so instead of collecting data we can just load it from a buffer, still taking one gradient step:

1. sample a batch (s_i, a_i, s'_i, r_i) from $\mathcal{B}$;
2. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(s_i, a_i)\big(Q_\phi(s_i, a_i) - [r(s_i, a_i) + \gamma \max_{a'} Q_\phi(s'_i, a'_i)]\big)$.

Samples are no longer correlated, and multiple samples in the batch give a low-variance gradient. The data must come from somewhere, so periodically feed the buffer with transitions collected by a policy such as ϵ-greedy.

Algorithm 8.1: Full Q-learning with replay buffer

```
repeat
    collect dataset {(s_i, a_i, s'_i, r_i)} using some policy, add it to B
    for K times (K = 1 is common, larger K more efficient) do
        sample a batch (s_i, a_i, s'_i, r_i) from B
        φ ← φ − α Σ_i (dQ_φ/dφ)(s_i, a_i)(Q_φ(s_i, a_i) − [r(s_i, a_i) + γ max_a' Q_φ(s'_i, a'_i)])
until done
```

8.2 Target networks

The replay buffer fixes correlation, but the moving target remains a problem. Full Q-learning with a replay buffer takes one gradient step toward a moving target, while fitted Q-iteration's

$\phi \leftarrow \arg\min_\phi \frac{1}{2}\sum_i \|Q_\phi(s_i,a_i) - y_i\|^2$ is a perfectly well-defined, stable regression. A TARGET NETWORK $Q_{\phi'}$ keeps the targets fixed in the inner loop, so each inner loop is supervised regression.

Algorithm 8.2: Q-learning with replay buffer and target network

repeat
  save target network parameters: $\phi' \leftarrow \phi$
  for N times **do**
    collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$
    for K times **do**
      sample a batch (s_i, a_i, s'_i, r_i) from $\mathcal{B}$
      $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(s_i,a_i)\big(Q_\phi(s_i,a_i) - [r(s_i,a_i) + \gamma \max_{a'} Q_{\phi'}(s'_i,a'_i)]\big)$
until done

With $K = 1$ this is the "classic" deep Q-learning algorithm.

Algorithm 8.3: Deep Q-learning (DQN)

repeat
  take some action a_i and observe (s_i, a_i, s'_i, r_i), add it to $\mathcal{B}$
  sample mini-batch $\{s_j, a_j, s'_j, r_j\}$ from $\mathcal{B}$ uniformly
  compute $y_j = r_j + \gamma \max_{a'_j} Q_{\phi'}(s'_j, a'_j)$ using the target network $Q_{\phi'}$
  $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j,a_j)\big(Q_\phi(s_j,a_j) - y_j\big)$
  update ϕ': copy ϕ every N steps
until done

Alternative target network. Copying every N steps gives an uneven lag: right after a copy the target has no lag, just before the next copy it has maximal lag. A popular alternative, similar to Polyak averaging, always has the same lag:

$$\phi' \leftarrow \tau\phi' + (1-\tau)\phi, \qquad \tau = 0.999 \text{ works well.}$$

8.3 A general view of Q-learning

Fitted Q-iteration, written in the same form, collects M datapoints, saves $\phi' \leftarrow \phi$, and runs K steps of SGD on the batch: its inner loop is just SGD. Q-learning with a replay buffer and target network collects M datapoints per inner iteration and updates the target every N iterations; DQN has $N = 1$, $K = 1$ in this form. All of these run three processes (Figure 8):

- process 1, data collection: the policy acts in the environment and adds (s, a, s', r) to the buffer, which evicts old data;
- process 2, target update: $\phi' \leftarrow \phi$;
- process 3, Q-function regression on batches from the buffer, using targets from ϕ' and updating ϕ.

Online Q-learning evicts immediately, and all three processes run at the same speed. In DQN, processes 1 and 3 run at the same speed and process 2 is slow. In fitted Q-iteration, process 3 is in the inner loop of process 2, which is in the inner loop of process 1.

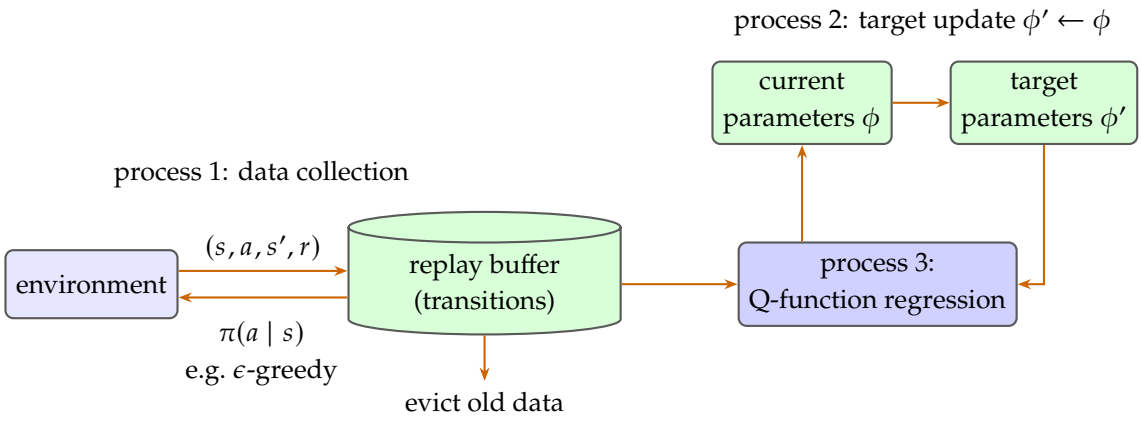


Figure 8: Q-learning as three processes around a replay buffer.

8.4 Improving Q-learning

Are the Q-values accurate? On Atari games, as the predicted Q increases during training, so does the return, and the Q-values track meaningful events in an episode. But the absolute values are badly overestimated: DQN's value estimates are far above the true discounted return of its policy, while double DQN's estimates are much closer.

Overestimation. The target $y_j = r_j + \gamma \max_{a'_j} Q_{\phi'}(s'_j, a'_j)$ is the problem. For two random variables, $\mathbb{E}[\max(X_1, X_2)] \geq \max(\mathbb{E}[X_1], \mathbb{E}[X_2])$. Since $Q_{\phi'}(s', a')$ is not perfect and looks noisy, $\max_{a'} Q_{\phi'}(s', a')$ overestimates the next value. Note that

$$\max_{a'} Q_{\phi'}(s', a') = Q_{\phi'}\big(s', \arg\max_{a'} Q_{\phi'}(s', a')\big) :$$

the value also comes from $Q_{\phi'}$, with the action selected according to $Q_{\phi'}$. If the noise in these is decorrelated, the problem goes away.

Definition 8.1 [Double Q-learning]**.** Do not use the same network to choose the action and evaluate the value. Double Q-learning uses two networks:

$$Q_{\phi_A}(s, a) \leftarrow r + \gamma Q_{\phi_B}\big(s', \arg\max_{a'} Q_{\phi_A}(s', a')\big), \qquad Q_{\phi_B}(s, a) \leftarrow r + \gamma Q_{\phi_A}\big(s', \arg\max_{a'} Q_{\phi_B}(s', a')\big).$$

If the two Q-functions are noisy in different ways, there is no problem.

In practice, the two Q-functions are the current and target networks. Standard Q-learning uses

$y = r + \gamma Q_{\phi'}(s', \arg\max_{a'} Q_{\phi'}(s', a'))$; double Q-learning uses

$$y = r + \gamma Q_{\phi'}\big(s', \arg\max_{a'} Q_{\phi}(s', a')\big),$$

selecting the action with the current network and still evaluating its value with the target network.

Multi-step returns. In the target $y_{j,t} = r_{j,t} + \gamma \max_{a_{j,t+1}} Q_{\phi'}(s_{j,t+1}, a_{j,t+1})$ the rewards are the only values that matter if $Q_{\phi'}$ is bad, and the Q-values are important if $Q_{\phi'}$ is good. Like the one-step critic in actor-critic, Q-learning has maximal bias and minimal variance. As in actor-critic, construct multi-step targets, the N-step return estimator

$$y_{j,t} = \sum_{t'=t}^{t+N-1} \gamma^{t'-t} r_{j,t'} + \gamma^N \max_{a_{j,t+N}} Q_{\phi'}(s_{j,t+N}, a_{j,t+N}).$$

It gives less biased target values when the Q-values are inaccurate and typically faster learning, especially early on. But it is only correct when learning on-policy: it is supposed to estimate $Q^\pi(s_{j,t}, a_{j,t})$ for the greedy policy π, so the transitions $s_{j,t'}, a_{j,t'}, s_{j,t'+1}$ for $t' - t < N - 1$ must come from π (not an issue when $N = 1$). Fixes: ignore the problem (often works very well); cut the trace, dynamically choosing N to get only on-policy data (works well when data is mostly on-policy and the action space is small); or importance sampling.

8.5 Q-learning with continuous actions

With continuous actions, both the policy $\pi(a_t \mid s_t) = \mathbf{1}\big[a_t = \arg\max_{a_t} Q_\phi(s_t, a_t)\big]$ and the target $y_j = r_j + \gamma \max_{a'_j} Q_{\phi'}(s'_j, a'_j)$ need a max, the second particularly problematic since it sits in the inner loop of training. How do we perform the max?

Option 1: optimization. Gradient-based optimization such as SGD is a bit slow in the inner loop, but the action space is typically low-dimensional, so stochastic optimization is an option. The simple solution is

$$\max_a Q(s, a) \approx \max\{Q(s, a_1), \ldots, Q(s, a_N)\},$$

with $(a_1, \ldots, a_N)$ sampled from some distribution such as uniform. It is dead simple and efficiently parallelizable, but not very accurate. But how good does the target need to be anyway? More accurate solutions are the cross-entropy method (CEM), a simple iterative stochastic optimization, and CMA-ES, a substantially less simple one; these work OK for up to about 40 dimensions.

Option 2: a function class that is easy to optimize. Normalized advantage functions (NAF) use a network that outputs $\mu_\phi(s)$, $P_\phi(s)$, and $V_\phi(s)$ and set

$$Q_\phi(s, a) = -\frac{1}{2}\big(a - \mu_\phi(s)\big)^T P_\phi(s)\big(a - \mu_\phi(s)\big) + V_\phi(s),$$

so that $\arg\max_a Q_\phi(s, a) = \mu_\phi(s)$ and $\max_a Q_\phi(s, a) = V_\phi(s)$ (with $P_\phi(s)$ positive definite). This

needs no change to the algorithm and is just as efficient as Q-learning, but loses representational power.

Option 3: learn an approximate maximizer. DDPG is a "deterministic" actor-critic, really approximate Q-learning. Since $\max_a Q_\phi(s,a) = Q_\phi(s, \arg\max_a Q_\phi(s,a))$, train another network $\mu_\theta(s) \approx \arg\max_a Q_\phi(s,a)$ by solving $\theta \leftarrow \arg\max_\theta Q_\phi(s, \mu_\theta(s))$, with gradient $\frac{dQ_\phi}{d\theta} = \frac{da}{d\theta}\frac{dQ_\phi}{da}$. The new target is $y_j = r_j + \gamma Q_{\phi'}(s'_j, \mu_\theta(s'_j)) \approx r_j + \gamma Q_{\phi'}(s'_j, \arg\max_{a'} Q_{\phi'}(s'_j, a'_j))$.

Algorithm 8.4: DDPG

repeat
  take some action a_i and observe (s_i, a_i, s'_i, r_i), add it to $\mathcal{B}$
  sample mini-batch $\{s_j, a_j, s'_j, r_j\}$ from $\mathcal{B}$ uniformly
  compute $y_j = r_j + \gamma Q_{\phi'}(s'_j, \mu_{\theta'}(s'_j))$ using target nets $Q_{\phi'}$ and $\mu_{\theta'}$
  $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)\big(Q_\phi(s_j, a_j) - y_j\big)$
  $\theta \leftarrow \theta + \beta \sum_j \frac{d\mu}{d\theta}(s_j)\frac{dQ_\phi}{da}(s_j, \mu(s_j))$
  update ϕ' and θ' (e.g. Polyak averaging)
until done

8.6 Implementation tips and examples

Q-learning takes some care to stabilize. Test on easy, reliable tasks first to make sure the implementation is correct (Pong is reliable; Video Pinball and Venture are much noisier across runs). Large replay buffers help improve stability, making the method look more like fitted Q-iteration. It takes time, so be patient: it might be no better than random for a while. Start with high exploration (ϵ) and gradually reduce it.

Advanced tips: Bellman error gradients can be big, so clip gradients or use the Huber loss (Figure 9)

$$L(x) = \begin{cases} x^2/2 & \text{if } |x| \le \delta, \\ \delta|x| - \delta^2/2 & \text{otherwise.} \end{cases}$$

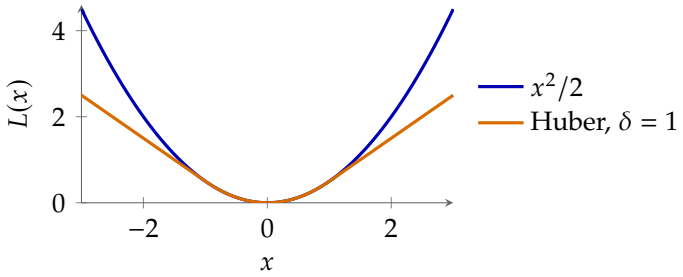


Figure 9: The Huber loss with $\delta = 1$ grows linearly beyond $|x| = \delta$, bounding the gradient of large Bellman errors.

Double Q-learning helps a lot in practice and is simple, with no downsides. N-step returns also help a lot but have some downsides. Schedule exploration (high to low) and learning rates (high to low); Adam can help too. Run multiple random seeds, since results are very inconsistent between runs.

Examples: fitted Q-iteration in a latent space learned with an autoencoder (with extra random trees for function approximation) for a slot car; human-level Atari control with convolutional Q-networks, a replay buffer, a target network, a one-step backup, and one gradient step, which double Q-learning (and other tricks) improve a lot; continuous control with a maximizer network (DDPG) using Polyak-averaged targets and one gradient step per simulator step; real robotic manipulation with NAF, four gradient steps per simulator step for efficiency, parallelized across multiple robots; and QT-Opt, large-scale Q-learning with continuous actions in which stored off-policy data and live on-policy data feed training buffers, Bellman updater threads compute $Q_T(s, a) = r + \max_{a'} Q_\theta(s', a')$, and training threads minimize $\|Q_\theta(s, a) - Q_T(s, a)\|^2$.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 8 (Fall 2023).

Lecture 9: Advanced Policy Gradients

9.1 Policy gradient as policy iteration

The policy gradient $\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \hat{A}^\pi_{i,t}$ (with reward to go, or a learned advantage) alternates two steps: estimate $\hat{A}^\pi(s_t, a_t)$ for the current policy π, then use it to get an improved policy π'. This looks like policy iteration (evaluate $A^\pi(s,a)$, set $\pi \leftarrow \pi'$), except that the policy gradient changes the policy gently instead of jumping to the argmax. Write the discounted objective as $J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}\big[\sum_t \gamma^t r(s_t, a_t)\big]$.

Proposition 9.1 [Policy improvement as advantage]. *For any two policies,*

$$J(\theta') - J(\theta) = \mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\Big[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t)\Big].$$

Proof. Since $J(\theta) = \mathbb{E}_{s_0\sim p(s_0)}[V^{\pi_\theta}(s_0)]$ and the initial state distribution does not depend on the policy, we may take the expectation of $V^{\pi_\theta}(s_0)$ under trajectories of $\pi_{\theta'}$:

$$\begin{aligned}
J(\theta') - J(\theta) &= J(\theta') - \mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\big[V^{\pi_\theta}(s_0)\big] \\
&= J(\theta') - \mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\Big[\sum_{t=0}^{\infty} \gamma^t V^{\pi_\theta}(s_t) - \sum_{t=1}^{\infty} \gamma^t V^{\pi_\theta}(s_t)\Big] \\
&= J(\theta') + \mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\Big[\sum_{t=0}^{\infty} \gamma^t \big(\gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)\big)\Big] \\
&= \mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\Big[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)\big)\Big] = \mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\Big[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(s_t, a_t)\Big],
\end{aligned}$$

where the last step uses $A^{\pi_\theta}(s_t, a_t) = \mathbb{E}[r(s_t, a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)]$ under the dynamics. □

So maximizing $J(\theta') - J(\theta)$ means maximizing the expected advantage under π_θ, with the expectation taken under $\pi_{\theta'}$. By importance sampling over actions,

$$\mathbb{E}_{\tau\sim p_{\theta'}(\tau)}\Big[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t)\Big] = \sum_t \mathbb{E}_{s_t\sim p_{\theta'}(s_t)}\Big[\mathbb{E}_{a_t\sim\pi_\theta(a_t|s_t)}\Big[\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \gamma^t A^{\pi_\theta}(s_t, a_t)\Big]\Big].$$

The states still come from $p_{\theta'}(s_t)$. Is it OK to use $p_\theta(s_t)$ instead, that is, is the left side above approximately

$$\bar{A}(\theta') = \sum_t \mathbb{E}_{s_t\sim p_\theta(s_t)}\Big[\mathbb{E}_{a_t\sim\pi_\theta(a_t|s_t)}\Big[\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \gamma^t A^{\pi_\theta}(s_t, a_t)\Big]\Big]?$$

We want this to be true because then $J(\theta') - J(\theta) \approx \bar{A}(\theta')$ and $\theta' \leftarrow \arg\max_{\theta'} \bar{A}(\theta')$ improves the policy using only samples from π_θ. The only part of $\bar{A}(\theta')$ that depends on θ' is the ratio. The claim is that $p_\theta(s_t)$ is close to $p_{\theta'}(s_t)$ when π_θ is close to $\pi_{\theta'}$.

9.2 Bounding the distribution change

Simple case. Let π_θ be deterministic, $a_t = \pi_\theta(s_t)$, and call $\pi_{\theta'}$ close to π_θ if $\pi_{\theta'}\big(a_t \neq \pi_\theta(s_t) \mid s_t\big) \leq \epsilon$. As in the behavioral cloning analysis of Lecture 2,

$$p_{\theta'}(s_t) = (1-\epsilon)^t p_\theta(s_t) + \big(1 - (1-\epsilon)^t\big) p_{\text{mistake}}(s_t),$$

where $(1-\epsilon)^t$ is the probability of making no mistakes and p_{mistake} is some other distribution. Then, using $(1-\epsilon)^t \geq 1 - \epsilon t$ for $\epsilon \in [0, 1]$,

$$|p_{\theta'}(s_t) - p_\theta(s_t)| = \big(1 - (1-\epsilon)^t\big)|p_{\text{mistake}}(s_t) - p_\theta(s_t)| \leq 2\big(1 - (1-\epsilon)^t\big) \leq 2\epsilon t,$$

where $|\cdot|$ summed over states is the total variation. Not a great bound, but a bound.

General case. Let π_θ be an arbitrary distribution, and call $\pi_{\theta'}$ close to π_θ if $|\pi_{\theta'}(a_t \mid s_t) - \pi_\theta(a_t \mid s_t)| \leq \epsilon$ for all s_t (in total variation). A useful lemma: if $|p_X(x) - p_Y(x)| = \epsilon$, there exists a joint $p(x, y)$ with marginals $p(x) = p_X(x)$ and $p(y) = p_Y(y)$ and $p(x = y) = 1 - \epsilon$. So p_X "agrees" with p_Y except with probability ϵ, and $\pi_{\theta'}(a_t \mid s_t)$ takes a different action than $\pi_\theta(a_t \mid s_t)$ with probability at most ϵ. The same argument then gives $|p_{\theta'}(s_t) - p_\theta(s_t)| \leq 2\epsilon t$.

Proposition 9.2 [Surrogate bound]. *If $|\pi_{\theta'}(a_t \mid s_t) - \pi_\theta(a_t \mid s_t)| \leq \epsilon$ for all s_t, then*

$$\sum_t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)}\left[\mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)}\left[\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \gamma^t A^{\pi_\theta}(s_t, a_t)\right]\right]$$
$$\geq \sum_t \mathbb{E}_{s_t \sim p_\theta(s_t)}\left[\mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)}\left[\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \gamma^t A^{\pi_\theta}(s_t, a_t)\right]\right] - \sum_t 2\epsilon t C,$$

where C bounds the inner expectation, $C = O(T r_{\max})$ or $O\big(r_{\max}/(1-\gamma)\big)$.

Proof. For any f,

$$\mathbb{E}_{p_{\theta'}(s_t)}[f(s_t)] = \sum_{s_t} p_{\theta'}(s_t) f(s_t) \geq \sum_{s_t} p_\theta(s_t) f(s_t) - |p_\theta(s_t) - p_{\theta'}(s_t)| \max_{s_t} f(s_t)$$
$$\geq \mathbb{E}_{p_\theta(s_t)}[f(s_t)] - 2\epsilon t \max_{s_t} f(s_t).$$

Apply this to the inner expectation at each t and sum. □

Maximizing the right-hand side maximizes a bound on the thing we want. So

$$\theta' \leftarrow \arg\max_{\theta'} \bar{A}(\theta') \quad \text{s.t.} \quad |\pi_{\theta'}(a_t \mid s_t) - \pi_\theta(a_t \mid s_t)| \le \epsilon$$

is, for small enough ϵ, guaranteed to improve $J(\theta') - J(\theta)$.

9.3 Policy gradients with constraints

A more convenient bound uses the KL divergence $D_{\mathrm{KL}}(p_1(x) \| p_2(x)) = \mathbb{E}_{x \sim p_1(x)}\left[\log \frac{p_1(x)}{p_2(x)}\right]$:

$$|\pi_{\theta'}(a_t \mid s_t) - \pi_\theta(a_t \mid s_t)| \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}\big(\pi_{\theta'}(a_t \mid s_t) \,\|\, \pi_\theta(a_t \mid s_t)\big)},$$

so the KL divergence bounds the state marginal difference. KL divergence has some very convenient properties that make it much easier to approximate. The problem becomes

$$\theta' \leftarrow \arg\max_{\theta'} \bar{A}(\theta') \quad \text{s.t.} \quad D_{\mathrm{KL}}\big(\pi_{\theta'}(a_t \mid s_t) \,\|\, \pi_\theta(a_t \mid s_t)\big) \le \epsilon.$$

Enforcing the constraint. Form the Lagrangian

$$\mathcal{L}(\theta', \lambda) = \bar{A}(\theta') - \lambda\big(D_{\mathrm{KL}}(\pi_{\theta'}(a_t \mid s_t) \| \pi_\theta(a_t \mid s_t)) - \epsilon\big)$$

and alternate: maximize $\mathcal{L}(\theta', \lambda)$ with respect to θ' (this can be done incompletely, for a few gradient steps), then $\lambda \leftarrow \lambda + \alpha\big(D_{\mathrm{KL}}(\pi_{\theta'}(a_t \mid s_t) \| \pi_\theta(a_t \mid s_t)) - \epsilon\big)$. Intuitively, raise λ if the constraint is violated too much and lower it otherwise. This is an instance of DUAL GRADIENT DESCENT (more later in the course).

9.4 Natural gradient

Another way to optimize the objective is a first-order Taylor approximation (linearization) of $\bar{A}$ around θ:

$$\theta' \leftarrow \arg\max_{\theta'} \nabla_\theta \bar{A}(\theta)^T(\theta' - \theta) \quad \text{s.t.} \quad D_{\mathrm{KL}}\big(\pi_{\theta'}(a_t \mid s_t) \| \pi_\theta(a_t \mid s_t)\big) \le \epsilon.$$

By the same derivation as the policy gradient,

$$\nabla_{\theta'} \bar{A}(\theta') = \sum_t \mathbb{E}_{s_t \sim p_\theta(s_t)}\left[\mathbb{E}_{a_t \sim \pi_\theta(a_t \mid s_t)}\left[\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \gamma^t \nabla_{\theta'} \log \pi_{\theta'}(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t)\right]\right],$$

and at $\theta' = \theta$ the ratio is 1:

$$\nabla_\theta \bar{A}(\theta) = \sum_t \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[\mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)} \left[\gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t) \right] \right] = \nabla_\theta J(\theta),$$

exactly the normal policy gradient. Can we just use the gradient then? No: gradient ascent $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ solves

$$\theta' \leftarrow \arg\max_{\theta'} \nabla_\theta J(\theta)^T (\theta' - \theta) \quad \text{s.t.} \quad \|\theta - \theta'\|^2 \le \epsilon, \qquad \text{giving} \qquad \theta' = \theta + \sqrt{\frac{\epsilon}{\|\nabla_\theta J(\theta)\|^2}} \nabla_\theta J(\theta),$$

a constraint in parameter space, which is not the same as the KL constraint: some parameters change probabilities a lot more than others. The second-order Taylor expansion of the KL divergence is

$$D_{\mathrm{KL}}(\pi_{\theta'} \| \pi_\theta) \approx \tfrac{1}{2} (\theta' - \theta)^T F (\theta' - \theta), \qquad F = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)^T \right],$$

with the Fisher-information matrix F, which can be estimated with samples. The constraint region becomes an ellipse instead of a circle, and the solution is the NATURAL GRADIENT

$$\theta' = \theta + \alpha F^{-1} \nabla_\theta J(\theta), \qquad \alpha = \sqrt{\frac{2\epsilon}{\nabla_\theta J(\theta)^T F \nabla_\theta J(\theta)}}.$$

This is a problem in practice. For $r(s_t, a_t) = -s_t^2 - a_t^2$ and $\log \pi_\theta(a_t \mid s_t) = -\frac{1}{2\sigma^2}(k s_t - a_t)^2 + \text{const}$ with $\theta = (k, \sigma)$, the vanilla gradient field is dominated by σ as $\sigma \to 0$, while the natural gradient points toward the optimal gain (Lecture 5); it is essentially the same problem as gradient descent on a poorly conditioned quadratic.

Practical methods.

- NATURAL POLICY GRADIENT, $\theta' = \theta + \alpha F^{-1} \nabla_\theta J(\theta)$, is generally a good choice to stabilize policy gradient training. A practical implementation requires efficient Fisher-vector products, which is a bit nontrivial without computing the full matrix.
- TRUST REGION POLICY OPTIMIZATION chooses ϵ and sets $\alpha = \sqrt{2\epsilon / (\nabla_\theta J(\theta)^T F \nabla_\theta J(\theta))}$.
- Or just use the importance-sampled objective $\bar{A}(\theta')$ directly, with regularization to stay close to the old policy, as in PROXIMAL POLICY OPTIMIZATION.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 9 (Fall 2023).

Lecture 10: Optimal Control and Planning

10.1 Model-based reinforcement learning

The reinforcement learning objective is

$$\theta^\star = \arg\max_\theta \mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\sum_t r(s_t, a_t)\Big], \qquad p_\theta(\tau) = p(s_1)\prod_{t=1}^{T}\pi_\theta(a_t \mid s_t)p(s_{t+1} \mid s_t, a_t).$$

Model-free methods assume $p(s_{t+1} \mid s_t, a_t)$ is unknown and do not even attempt to learn it. But often we do know the dynamics: games (Atari, chess, Go), easily modeled systems (navigating a car), and simulated environments (simulated robots, video games). And often we can learn them, by system identification (fitting unknown parameters of a known model) or by fitting a general-purpose model to observed transition data. Knowing the dynamics often makes things easier. Model-based reinforcement learning learns the transition dynamics and then figures out how to choose actions. This lecture asks how to make decisions when the dynamics are known: optimal control, trajectory optimization, and planning. The next lectures learn unknown dynamics and then policies, for example by imitating optimal control.

10.2 The objective

Planning optimizes actions directly. With a cost, for example the probability of being eaten by the tiger, and deterministic dynamics $s_{t+1} = f(s_t, a_t)$,

$$\min_{a_1,\dots,a_T}\sum_{t=1}^{T} c(s_t, a_t) \quad \text{s.t.} \quad s_t = f(s_{t-1}, a_{t-1}),$$

or, with rewards, $a_1, \dots, a_T = \arg\max_{a_1,\dots,a_T}\sum_t r(s_t, a_t)$ s.t. $s_{t+1} = f(s_t, a_t)$. In the deterministic case the environment reveals s_1, the agent plans $a_1, \dots, a_T$ and sends them.

The stochastic open-loop case. With $p_\theta(s_1, \dots, s_T \mid a_1, \dots, a_T) = p(s_1)\prod_{t=1}^{T} p(s_{t+1} \mid s_t, a_t)$,

$$a_1, \dots, a_T = \arg\max_{a_1,\dots,a_T} \mathbb{E}\Big[\sum_t r(s_t, a_t) \;\Big|\; a_1, \dots, a_T\Big].$$

This is suboptimal: an open-loop plan receives the state only at $t = 1$ and then communication is one-way, so it cannot react to the states actually reached. A closed-loop agent observes each s_t and acts by $a_t \sim \pi(a_t \mid s_t)$.

The stochastic closed-loop case. With $p(s_1, a_1, \dots, s_T, a_T) = p(s_1)\prod_{t=1}^{T}\pi(a_t \mid s_t)p(s_{t+1} \mid s_t, a_t)$, solve

$\pi = \arg\max_\pi \mathbb{E}_{\tau\sim p(\tau)}\left[\sum_t r(s_t, a_t)\right]$. The policy may be a neural net (global) or, for example, a time-varying linear controller $K_t s_t + k_t$ (local); more on this later. For now, open-loop planning.

10.3 Stochastic optimization

Abstract away the structure of optimal control: $a_1, \dots, a_T = \arg\max_{a_1,\dots,a_T} J(a_1, \dots, a_T)$, written $A = \arg\max_A J(A)$, without caring what J is. The simplest method is guess and check, the RANDOM SHOOTING METHOD: pick $A_1, \dots, A_N$ from some distribution (for example uniform) and choose A_i by $\arg\max_i J(A_i)$. We can do better by improving the sampling distribution.

Algorithm 10.1: Cross-entropy method (CEM) with continuous inputs

```
repeat
    sample A_1, ..., A_N from p(A)
    evaluate J(A_1), ..., J(A_N)
    pick the elites A_{i_1}, ..., A_{i_M} with the highest value, where M < N
    refit p(A) to the elites A_{i_1}, ..., A_{i_M}
until converged
```

Typically $p(A)$ is Gaussian. CMA-ES is sort of like CEM with momentum. These methods are very fast if parallelized and extremely simple, but they have a very harsh dimensionality limit and do only open-loop planning.

10.4 Monte Carlo tree search

In the discrete case (for example an Atari game with a handful of actions), planning is a search problem over a tree of actions. The full tree is exponential in the horizon, so approximate the value of a node without expanding the full tree: from a leaf, run a DEFAULT POLICY, for example a random policy, and use its return. We cannot search all paths, so where to search first? Choose nodes with the best reward, but also prefer rarely visited nodes (Figure 10).

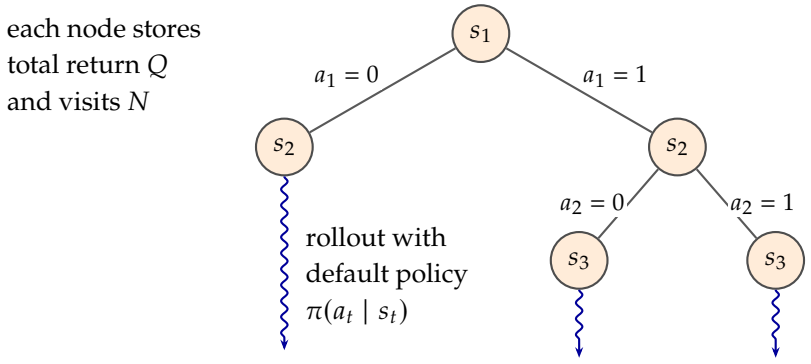


Figure 10: Monte Carlo tree search: the tree policy descends the partially expanded tree, and each new leaf is evaluated by a rollout of the default policy. Schematic.

Algorithm 10.2: Generic MCTS

```
repeat
    find a leaf s_l using TreePolicy(s_1)
    evaluate the leaf using DefaultPolicy(s_l)
    update all values in the tree between s_1 and s_l
until the computational budget is exhausted
take the best action from s_1
```

Each node stores the total return $Q(s_t)$ of the rollouts through it and its visit count $N(s_t)$. The UCT tree policy: if s_t is not fully expanded, choose a new a_t; else choose the child with the best

$$\text{Score}(s_t) = \frac{Q(s_t)}{N(s_t)} + 2C\sqrt{\frac{2\ln N(s_{t-1})}{N(s_t)}},$$

the average return plus a bonus for rarely visited children.

10.5 Trajectory optimization with derivatives

Return to continuous control in the optimal-control notation, with state x_t and action u_t:

$$\min_{u_1,\dots,u_T} \sum_{t=1}^{T} c(x_t, u_t) \quad \text{s.t.} \quad x_t = f(x_{t-1}, u_{t-1}),$$

$$\text{equivalently} \quad \min_{u_1,\dots,u_T} c(x_1, u_1) + c(f(x_1, u_1), u_2) + \cdots + c(f(f(\dots)\dots), u_T).$$

The usual story is to differentiate via backpropagation and optimize, which needs $\frac{df}{dx_t}, \frac{df}{du_t}, \frac{dc}{dx_t}, \frac{dc}{du_t}$; in practice it really helps to use a second-order method. Shooting methods optimize over actions only, as in the unconstrained form above, so a small change to an early action moves the whole rest of the trajectory. Collocation methods optimize over actions and states, with constraints: $\min_{u_1,\dots,u_T,x_1,\dots,x_T} \sum_t c(x_t, u_t)$ s.t. $x_t = f(x_{t-1}, u_{t-1})$.

10.6 Linear case: LQR

The linear quadratic regulator (LQR) is the shooting problem with linear dynamics and quadratic cost:

$$f(x_t, u_t) = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + f_t, \qquad c(x_t, u_t) = \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T c_t.$$

Base case. The last term $c(f(f(\dots)\dots), u_T)$ is the only one that depends on u_T, and x_T is unknown,

so solve for u_T only, as a function of x_T. With the partitions

$$C_T = \begin{bmatrix} C_{x_T,x_T} & C_{x_T,u_T} \\ C_{u_T,x_T} & C_{u_T,u_T} \end{bmatrix}, \qquad c_T = \begin{bmatrix} c_{x_T} \\ c_{u_T} \end{bmatrix}, \qquad Q(x_T, u_T) = \text{const} + \frac{1}{2} \begin{bmatrix} x_T \\ u_T \end{bmatrix}^T C_T \begin{bmatrix} x_T \\ u_T \end{bmatrix} + \begin{bmatrix} x_T \\ u_T \end{bmatrix}^T c_T,$$

$$\nabla_{u_T} Q(x_T, u_T) = C_{u_T,x_T} x_T + C_{u_T,u_T} u_T + c_{u_T} = 0 \quad \Longrightarrow \quad u_T = K_T x_T + k_T,$$
$$K_T = -C_{u_T,u_T}^{-1} C_{u_T,x_T}, \qquad k_T = -C_{u_T,u_T}^{-1} c_{u_T}.$$

Since u_T is fully determined by x_T, eliminate it by substitution: $V(x_T) = Q(x_T, K_T x_T + k_T)$, which expands to

$$V(x_T) = \text{const} + \tfrac{1}{2} x_T^T V_T x_T + x_T^T v_T, \qquad \begin{aligned} V_T &= C_{x_T,x_T} + C_{x_T,u_T} K_T + K_T^T C_{u_T,x_T} + K_T^T C_{u_T,u_T} K_T, \\ v_T &= c_{x_T} + C_{x_T,u_T} k_T + K_T^T c_{u_T} + K_T^T C_{u_T,u_T} k_T. \end{aligned}$$

One step back. Now u_{T-1} affects $x_T = f(x_{T-1}, u_{T-1})$, so

$$Q(x_{T-1}, u_{T-1}) = \text{const} + \frac{1}{2} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^T C_{T-1} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix} + \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^T c_{T-1} + V\big(f(x_{T-1}, u_{T-1})\big).$$

Substituting the linear dynamics into the quadratic $V(x_T)$ gives a quadratic term with matrix $F_{T-1}^T V_T F_{T-1}$ and linear terms $F_{T-1}^T V_T f_{T-1} + F_{T-1}^T v_T$, so

$$Q(x_{T-1}, u_{T-1}) = \text{const} + \frac{1}{2} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^T Q_{T-1} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix} + \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^T q_{T-1},$$

with $Q_{T-1} = C_{T-1} + F_{T-1}^T V_T F_{T-1}$ and $q_{T-1} = c_{T-1} + F_{T-1}^T V_T f_{T-1} + F_{T-1}^T v_T$. Setting

$$\nabla_{u_{T-1}} Q(x_{T-1}, u_{T-1}) = Q_{u_{T-1},x_{T-1}} x_{T-1} + Q_{u_{T-1},u_{T-1}} u_{T-1} + q_{u_{T-1}} = 0$$

gives $u_{T-1} = K_{T-1} x_{T-1} + k_{T-1}$ with $K_{T-1} = -Q_{u_{T-1},u_{T-1}}^{-1} Q_{u_{T-1},x_{T-1}}$ and $k_{T-1} = -Q_{u_{T-1},u_{T-1}}^{-1} q_{u_{T-1}}$. Repeating this gives the full algorithm, in which $Q(x_t, u_t)$ is the total cost from now until the end if we take u_t from state x_t, and $V(x_t) = \min_{u_t} Q(x_t, u_t)$ is the total cost from now until the end from state x_t.

Algorithm 10.3: LQR

// backward recursion; $V_{T+1} = 0$, $v_{T+1} = 0$

for $t = T$ **to** 1 **do**

$Q_t = C_t + F_t^T V_{t+1} F_t$

$q_t = c_t + F_t^T V_{t+1} f_t + F_t^T v_{t+1}$

$Q(x_t, u_t) = \text{const} + \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T Q_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T q_t$

$u_t \leftarrow \arg\min_{u_t} Q(x_t, u_t) = K_t x_t + k_t$ with $K_t = -Q_{u_t,u_t}^{-1} Q_{u_t,x_t}$, $k_t = -Q_{u_t,u_t}^{-1} q_{u_t}$

$V_t = Q_{x_t,x_t} + Q_{x_t,u_t} K_t + K_t^T Q_{u_t,x_t} + K_t^T Q_{u_t,u_t} K_t$

$v_t = q_{x_t} + Q_{x_t,u_t} k_t + K_t^T q_{u_t} + K_t^T Q_{u_t,u_t} k_t$

$V(x_t) = \text{const} + \frac{1}{2} x_t^T V_t x_t + x_t^T v_t$

// forward recursion, since x_1 is known

for $t = 1$ **to** T **do**

$u_t = K_t x_t + k_t$

$x_{t+1} = f(x_t, u_t)$

10.7 LQR for stochastic and nonlinear systems

Stochastic dynamics. If

$$p(x_{t+1} \mid x_t, u_t) = \mathcal{N}\left(F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + f_t,\ \Sigma_t\right),$$

the solution is still to choose actions according to $u_t = K_t x_t + k_t$. Now $x_t \sim p(x_t)$ is no longer deterministic, but $p(x_t)$ is Gaussian, and there is no change to the algorithm: Σ_t can be ignored by the symmetry of Gaussians, since the expectation of a quadratic under a Gaussian adds only a constant that does not depend on the action. This is the stochastic closed-loop case with a time-varying linear policy $K_t x_t + k_t$.

Nonlinear case: DDP and iterative LQR. Approximate a nonlinear system as a linear-quadratic one around a nominal trajectory $(\hat{x}_t, \hat{u}_t)$:

$$f(x_t, u_t) \approx f(\hat{x}_t, \hat{u}_t) + \nabla_{x_t,u_t} f(\hat{x}_t, \hat{u}_t) \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix},$$

$$c(x_t, u_t) \approx c(\hat{x}_t, \hat{u}_t) + \nabla_{x_t,u_t} c(\hat{x}_t, \hat{u}_t) \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix} + \frac{1}{2} \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix}^T \nabla^2_{x_t,u_t} c(\hat{x}_t, \hat{u}_t) \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix}.$$

With $\delta x_t = x_t - \hat{x}_t$ and $\delta u_t = u_t - \hat{u}_t$, this is

$$\bar{f}(\delta x_t, \delta u_t) = F_t \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}, \qquad \bar{c}(\delta x_t, \delta u_t) = \frac{1}{2} \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}^T C_t \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix} + \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}^T c_t,$$

with $F_t = \nabla_{x_t,u_t} f(\hat{x}_t, \hat{u}_t)$, $C_t = \nabla^2_{x_t,u_t} c(\hat{x}_t, \hat{u}_t)$, and $c_t = \nabla_{x_t,u_t} c(\hat{x}_t, \hat{u}_t)$. Now run LQR with dynamics $\bar{f}$, cost $\bar{c}$, state δx_t, and action δu_t.

Algorithm 10.4: Iterative LQR (simplified)

repeat
  $F_t = \nabla_{x_t,u_t} f(\hat{x}_t, \hat{u}_t)$, $c_t = \nabla_{x_t,u_t} c(\hat{x}_t, \hat{u}_t)$, $C_t = \nabla^2_{x_t,u_t} c(\hat{x}_t, \hat{u}_t)$
  run the LQR backward pass on state $\delta x_t = x_t - \hat{x}_t$ and action $\delta u_t = u_t - \hat{u}_t$
  run the forward pass with the real nonlinear dynamics and $u_t = K_t(x_t - \hat{x}_t) + \alpha k_t + \hat{u}_t$
  update $\hat{x}_t$ and $\hat{u}_t$ based on the states and actions in the forward pass
until convergence

Why does this work? Compare Newton's method for $\min_x g(x)$: until convergence, $g = \nabla_x g(\hat{x})$, $H = \nabla^2_x g(\hat{x})$, and $\hat{x} \leftarrow \arg\min_x \frac{1}{2}(x - \hat{x})^T H(x - \hat{x}) + g^T(x - \hat{x})$. Iterative LQR is the same idea: locally approximate a complex nonlinear function by a Taylor expansion. In fact iLQR approximates Newton's method for the shooting objective; to get Newton's method exactly, use a second-order dynamics approximation,

$$f(x_t, u_t) \approx f(\hat{x}_t, \hat{u}_t) + \nabla_{x_t,u_t} f(\hat{x}_t, \hat{u}_t) \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix} + \frac{1}{2}\left(\nabla^2_{x_t,u_t} f(\hat{x}_t, \hat{u}_t) \cdot \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}\right) \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix},$$

which gives DIFFERENTIAL DYNAMIC PROGRAMMING (DDP).

Jumping straight to the minimizer of the quadratic model is a bad idea: far from $\hat{x}$ the approximation can be poor and the true cost can increase. Hence the step size α in the forward pass ($\alpha = 0$ leaves the trajectory unchanged): search over α until improvement is achieved.

Case study: nonlinear model-predictive control. At every time step, observe the state x_t, use iLQR to plan $u_t, \ldots, u_{t+T}$ minimizing $\sum_{t'=t}^{t+T} c(x_{t'}, u_{t'})$, execute u_t, and discard $u_{t+1}, \ldots, u_{t+T}$. Replanning at every step produces complex, robust behaviors for simulated humanoids. The remaining problem with known dynamics is that for many real tasks (folding a towel, a robot arm in the world) we do not know them; the next lecture learns the dynamics model.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 10 (Fall 2023).

Lecture 11: Model-Based Reinforcement Learning

11.1 Learning a model and planning through it

If we knew $f(s_t, a_t) = s_{t+1}$ (or $p(s_{t+1} \mid s_t, a_t)$ in the stochastic case), we could use the planning tools of Lecture 10. So learn $f(s_t, a_t)$ from data, and then plan through it.

Algorithm 11.1: Model-based RL version 0.5

1. run base policy $\pi_0(a_t \mid s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
2. learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
3. plan through $f(s, a)$ to choose actions

Does it work? Yes: this is essentially how system identification works in classical robotics. Some care should be taken to design a good base policy, and it is particularly effective if we can hand-engineer a dynamics representation using our knowledge of physics and fit just a few parameters. But in general, no. A model trained on data from π_0 (walking near the base of a mountain, where going right goes up) may say "go right to get higher" everywhere, and the plan walks off the cliff: the planned policy visits $p_{\pi_f}(s_t) \neq p_{\pi_0}(s_t)$. This distribution mismatch problem becomes exacerbated as we use more expressive model classes.

Can we make $p_{\pi_0}(s_t) = p_{\pi_f}(s_t)$? As in DAgger, collect data from $p_{\pi_f}(s_t)$.

Algorithm 11.2: Model-based RL version 1.0

1. run base policy $\pi_0(a_t \mid s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
2. **repeat**
3. learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
4. plan through $f(s, a)$ to choose actions
5. execute those actions and add the resulting data $\{(s, a, s')_j\}$ to $\mathcal{D}$
6. **until** done

What if we make a mistake, for example a car whose model is slightly off drifts out of its lane while executing a whole open-loop plan? Replanning helps with model errors: execute only the first planned action, observe the resulting state, and plan again (model-predictive control, MPC).

Algorithm 11.3: Model-based RL version 1.5

run base policy $\pi_0(a_t \mid s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
repeat
  learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
  for N steps **do**
    plan through $f(s, a)$ to choose actions
    execute the first planned action, observe the resulting state s' (MPC)
    append (s, a, s') to dataset $\mathcal{D}$
until done

How to replan? The more you replan, the less perfect each individual plan needs to be. Shorter horizons can be used, and even random sampling can often work well here.

11.2 Uncertainty in model-based RL

A performance gap. On a simulated cheetah, pure model-based learning of this kind reaches a modest return after about 10 minutes of real time, while model-free training takes about 10 days to get much higher; a model-free learner initialized from the model-based one does best. Why the gap? The model must not overfit when there is little data, yet still have high capacity when there is a lot. With little data, a model can fit the training points but be wildly wrong between them, and the planner is very tempted to go exactly where the model erroneously predicts high reward. The planner exploits the model's errors.

How uncertainty helps. Suppose that instead of a point prediction we have a distribution over outcomes, with low variance near data and high variance far from it. The expected reward under a high-variance prediction is very low, even though the mean is the same: standing near a cliff edge is fine if we know exactly where the edge is, but not if we are uncertain. So in version 1.5, plan to take only actions for which we think we will get high reward in expectation with respect to the uncertain dynamics. This avoids exploiting the model, and the model then adapts and gets better. Caveats: we need to explore to get better; expected value is not the same as pessimistic value, nor the same as optimistic value; but expected value is often a good start.

11.3 Uncertainty-aware neural network models

Idea 1: use output entropy. A network that outputs a distribution $p(s_{t+1} \mid s_t, a_t)$ (a softmax or a Gaussian) has an entropy. Why is this not enough? There are two types of uncertainty:

- ALEATORIC (statistical) uncertainty, inherent noise in the data, which the output distribution does capture;
- EPISTEMIC (model) uncertainty: "the model is certain about the data, but we are not certain about the model."

A model that overfits a few points can have near-zero output variance everywhere, so its output entropy says nothing about where it is wrong.

Idea 2: estimate model uncertainty. Usually we estimate $\arg\max_\theta \log p(\theta \mid \mathcal{D}) = \arg\max_\theta \log p(\mathcal{D} \mid \theta)$ (with a uniform prior). Instead, estimate the posterior $p(\theta \mid \mathcal{D})$, whose entropy tells us the model uncertainty, and predict according to

$$\int p(s_{t+1} \mid s_t, a_t, \theta)\, p(\theta \mid \mathcal{D})\, d\theta.$$

Bayesian neural networks place a distribution on every weight. A common approximation is $p(\theta \mid \mathcal{D}) = \prod_i p(\theta_i \mid \mathcal{D})$ with $p(\theta_i \mid \mathcal{D}) = \mathcal{N}(\mu_i, \sigma_i)$, where μ_i is the expected weight and σ_i the uncertainty about the weight (examples are weight uncertainty in neural networks and concrete dropout; variational inference comes later in the course).

Bootstrap ensembles. Train multiple models and see whether they agree. Formally,

$$p(\theta \mid \mathcal{D}) \approx \frac{1}{N} \sum_i \delta(\theta_i), \qquad \int p(s_{t+1} \mid s_t, a_t, \theta) p(\theta \mid \mathcal{D})\, d\theta \approx \frac{1}{N} \sum_i p(s_{t+1} \mid s_t, a_t, \theta_i).$$

To get "independent" models we need "independent" datasets: train θ_i on $\mathcal{D}_i$, sampled with replacement from $\mathcal{D}$. In deep learning this basically works. It is a very crude approximation, because the number of models is usually small (< 10), and resampling with replacement is usually unnecessary, because SGD and random initialization usually make the models sufficiently independent.

11.4 Planning with uncertainty

Before, the planner maximized $J(a_1, \ldots, a_H) = \sum_{t=1}^H r(s_t, a_t)$ with $s_{t+1} = f(s_t, a_t)$. With an ensemble, a distribution over deterministic models, maximize

$$J(a_1, \ldots, a_H) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H r(s_{t,i}, a_t), \qquad s_{t+1,i} = f_i(s_{t,i}, a_t).$$

In general, for a candidate action sequence $a_1, \ldots, a_H$: sample $\theta \sim p(\theta \mid \mathcal{D})$; at each time step sample $s_{t+1} \sim p(s_{t+1} \mid s_t, a_t, \theta)$; calculate $R = \sum_t r(s_t, a_t)$; repeat and accumulate the average reward. Other options include moment matching and more complex posterior estimation with Bayesian neural networks.

With probabilistic ensembles and this planner, model-based RL on the half-cheetah exceeds the performance of model-free methods after about 40k steps (about 10 minutes of real time), and the same approach learns dexterous manipulation with a real robot hand (PDDM).

11.5 Model-based RL with images

What about complex observations such as images? Learning $f(s_t, a_t) = s_{t+1}$ directly on images is hard because of high dimensionality, redundancy, and partial observability. Images are high-dimensional but not dynamic, while the underlying state is low-dimensional but dynamic, which suggests learning $p(o_t \mid s_t)$ and $p(s_{t+1} \mid s_t, a_t)$ separately.

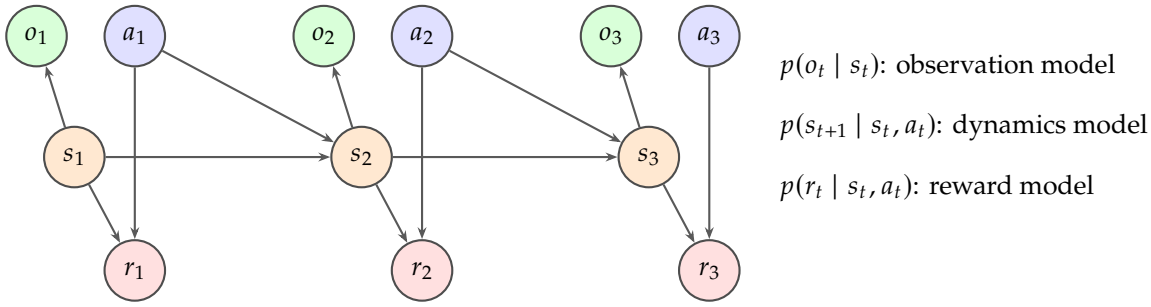


Figure 11: A latent state-space model with observation, dynamics, and reward models.

A STATE SPACE (latent space) model has an observation model $p(o_t \mid s_t)$, a dynamics model $p(s_{t+1} \mid s_t, a_t)$, and a reward model $p(r_t \mid s_t, a_t)$ (Figure 11). A standard fully observed model is trained by $\max_\phi \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \log p_\phi(s_{t+1,i} \mid s_{t,i}, a_{t,i})$. A latent space model is trained by

$$\max_\phi \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \mathbb{E}\big[\log p_\phi(s_{t+1,i} \mid s_{t,i}, a_{t,i}) + \log p_\phi(o_{t,i} \mid s_{t,i})\big],$$

with the expectation taken with respect to $(s_t, s_{t+1}) \sim p(s_t, s_{t+1} \mid o_{1:T}, a_{1:T})$. Learn an approximate posterior, an "encoder" $q_\psi(s_t \mid o_{1:t}, a_{1:t})$. There are many choices: the full smoothing posterior $q_\psi(s_t, s_{t+1} \mid o_{1:T}, a_{1:T})$ is the most accurate but most complicated, and the single-step encoder $q_\psi(s_t \mid o_t)$ is the simplest but least accurate. With the single-step encoder the expectation is over $s_t \sim q_\psi(s_t \mid o_t)$ and $s_{t+1} \sim q_\psi(s_{t+1} \mid o_{t+1})$. The stochastic case requires variational inference (Lecture 18). In the simple special case where $q(s_t \mid o_t)$ is deterministic, $q_\psi(s_t \mid o_t) = \delta(s_t = g_\psi(o_t))$, so $s_t = g_\psi(o_t)$, and with a reward model the objective is

$$\max_{\phi,\psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \underbrace{\log p_\phi\big(g_\psi(o_{t+1,i}) \mid g_\psi(o_{t,i}), a_{t,i}\big)}_{\text{latent space dynamics}} + \underbrace{\log p_\phi\big(o_{t,i} \mid g_\psi(o_{t,i})\big)}_{\text{image reconstruction}} + \underbrace{\log p_\phi\big(r_{t,i} \mid g_\psi(o_{t,i})\big)}_{\text{reward model}}.$$

Everything is differentiable, so it can be trained with backpropagation. Many practical methods use a stochastic encoder to model uncertainty.

Algorithm 11.4: Model-based RL with latent state

run base policy $\pi_0(a_t \mid o_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(o, a, o')_i\}$
repeat
  learn $p_\phi(s_{t+1} \mid s_t, a_t)$, $p_\phi(r_t \mid s_t)$, $p(o_t \mid s_t)$, $g_\psi(o_t)$
  for N steps **do**
    plan through the model to choose actions
    execute the first planned action, observe the resulting o' (MPC)
    append (o, a, o') to dataset $\mathcal{D}$
until done

Examples: Embed to Control (E2C) learns a locally linear latent dynamics model from raw images for swing-up tasks, and SOLAR learns structured latent representations with linear-Gaussian latent dynamics for real robot manipulation. Alternatively, learn directly in observation space, $p(o_{t+1} \mid o_t, a_t)$, with a video prediction model, and use its predictions to complete tasks: designate a pixel on an object and a goal pixel, and plan actions under which the predicted motion moves the designated pixel to the goal.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 11 (Fall 2023).

Lecture 12: Model-Based Policy Learning

12.1 Backpropagating into the policy

Model-based RL with MPC (version 1.5) replans through the model at every step, which is open-loop planning repeated. The stochastic open-loop problem $a_1, \dots, a_T = \arg\max \mathbb{E}\left[\sum_t r(s_t, a_t) \mid a_1, \dots, a_T\right]$ is suboptimal compared with the closed-loop problem $\pi = \arg\max_\pi \mathbb{E}_{\tau \sim p(\tau)}\left[\sum_t r(s_t, a_t)\right]$, where $p(s_1, a_1, \dots, s_T, a_T) = p(s_1) \prod_t \pi(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t)$ and π may be a global neural net or a local time-varying linear controller $K_t s_t + k_t$. Can we learn a policy with the model?

The simplest idea is to backpropagate directly into the policy through the model (Figure 12). This is easy for deterministic policies but also possible for stochastic policies.

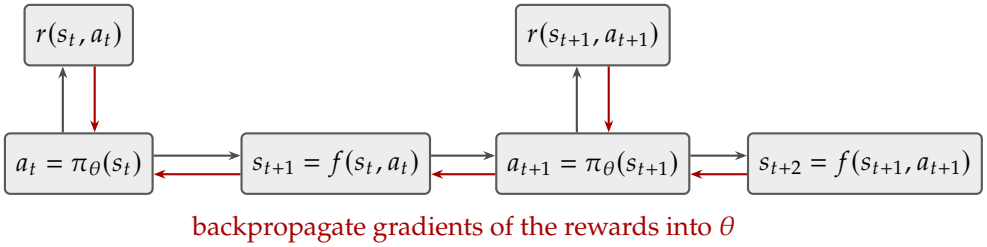


Figure 12: Backpropagating rewards through a learned model into the policy.

Algorithm 12.1: Model-based RL version 2.0

run base policy $\pi_0(a_t \mid s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
repeat
  learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
  backpropagate through $f(s, a)$ into the policy to optimize $\pi_\theta(a_t \mid s_t)$
  run $\pi_\theta(a_t \mid s_t)$, appending the visited tuples (s, a, s') to $\mathcal{D}$
until done

What is the problem? Gradients of late rewards pass through many Jacobians, so the gradients for early actions are big and those for late actions small: a small change to an early action moves the whole trajectory.

- There are similar parameter sensitivity problems as in shooting methods, but no convenient second-order LQR-like method, because the policy parameters couple all the time steps, so there is no dynamic programming.
- There are similar problems to training long RNNs with backpropagation through time: vanishing and exploding gradients. Unlike an LSTM, we cannot just choose a simple dynamics; the dynamics are chosen by nature.

The solution is to use derivative-free ("model-free") RL algorithms, with the model used to generate synthetic samples. This seems weirdly backwards, but actually works very well: it is essentially "model-based acceleration" for model-free RL.

12.2 Model-free learning with a model

Compare the policy gradient $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \hat{Q}^\pi_{i,t}$ with the backpropagation (pathwise) gradient

$$\nabla_\theta J(\theta) = \sum_{t=1}^{T} \frac{da_t}{d\theta} \frac{ds_{t+1}}{da_t} \left(\sum_{t'=t+1}^{T} \frac{dr_{t'}}{ds_{t'}} \left(\prod_{t''=t+2}^{t'} \left(\frac{ds_{t''}}{da_{t''-1}} \frac{da_{t''-1}}{ds_{t''-1}} + \frac{ds_{t''}}{ds_{t''-1}} \right) \right) \right).$$

The policy gradient might be more stable (if enough samples are used), because it does not require multiplying many Jacobians.

Algorithm 12.2: Model-based RL version 2.5

run base policy $\pi_0(a_t \mid s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
repeat
  learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
  repeat
    use $f(s, a)$ to generate trajectories $\{\tau_i\}$ with policy $\pi_\theta(a \mid s)$
    use $\{\tau_i\}$ to improve $\pi_\theta(a \mid s)$ via policy gradient
  until done improving
  run $\pi_\theta(a_t \mid s_t)$, appending the visited tuples (s, a, s') to $\mathcal{D}$
until done

The potential problem is the curse of long model-based rollouts. Running π_θ with the learned model drifts away from running it with the true dynamics, and, as in behavioral cloning (Lecture 2), the error accumulates as $O(\epsilon T^2)$. How can we get away with short rollouts (Figure 13)? Long rollouts have huge accumulating error. Short rollouts from the initial states have much lower error but never see later time steps. Short rollouts branched from states sampled along real trajectories have much lower error and see all time steps, at the cost of the wrong state distribution: the policy has changed since those states were collected.

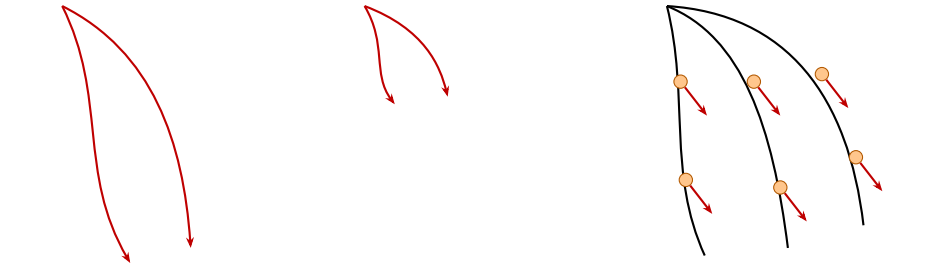


Figure 13: Long model rollouts (left), short rollouts from the start (middle), and short model rollouts (red) branched from states on real trajectories (black, right). Schematic.

Algorithm 12.3: Model-based RL version 3.0

run base policy $\pi_0(a_t \mid s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
repeat
  learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
  repeat
    pick states s_i from $\mathcal{D}$, use $f(s, a)$ to make *short* rollouts from them
    use *both* real and model data to improve $\pi_\theta(a \mid s)$ with *off-policy* RL
  until done improving
  run $\pi_\theta(a_t \mid s_t)$, appending the visited tuples (s, a, s') to $\mathcal{D}$
until done

12.3 Dyna-style algorithms

Dyna is an online Q-learning algorithm that performs model-free RL with a model.

Algorithm 12.4: Dyna

repeat
  given state s, pick action a using the exploration policy
  observe s' and r, to get transition (s, a, s', r)
  update model $\hat{p}(s' \mid s, a)$ and $\hat{r}(s, a)$ using (s, a, s')
  Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha \mathbb{E}_{s',r}\left[r + \max_{a'} Q(s', a') - Q(s, a)\right]$
  for K times **do**
    sample $(s, a) \sim \mathcal{B}$ from the buffer of past states and actions
    Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha \mathbb{E}_{s',r}\left[r + \max_{a'} Q(s', a') - Q(s, a)\right]$
until done

The general "Dyna-style" recipe: collect some data, consisting of transitions (s, a, s', r); learn a model $\hat{p}(s' \mid s, a)$ (and optionally $\hat{r}(s, a)$); then repeat K times: sample $s \sim \mathcal{B}$ from the buffer, choose an action a (from $\mathcal{B}$, from π, or at random), simulate $s' \sim \hat{p}(s' \mid s, a)$ (and $r = \hat{r}(s, a)$), train on (s, a, s', r) with model-free RL, and optionally take N more model-based steps. It only requires

short (as few as one step) rollouts from the model and still sees diverse states.

Model-accelerated off-policy RL. In the process view of Q-learning (Lecture 8), add process 4, model training on the replay buffer, and process 5, model data collection: rollouts that start from states $s \sim \mathcal{B}$ fill a second buffer of model-based transitions, which is evicted each time the model changes, and Q-function regression samples from both buffers. Model-based acceleration (MBA), model-based value expansion (MVE), and model-based policy optimization (MBPO) all follow this pattern:

1. take some action a_i and observe (s_i, a_i, s'_i, r_i), add it to $\mathcal{B}$;
2. sample a mini-batch $\{s_j, a_j, s'_j, r_j\}$ from $\mathcal{B}$ uniformly;
3. use $\{s_j, a_j, s'_j\}$ to update the model $\hat{p}(s' \mid s, a)$;
4. sample $\{s_j\}$ from $\mathcal{B}$;
5. for each s_j, perform a model-based rollout with $a = \pi(s)$;
6. use all transitions (s, a, s', r) along the rollout to update the Q-function.

This is a good idea because the model supplies many more transitions near the data, which increases sample efficiency. It is a bad idea when the model is wrong: the Q-function is then trained on erroneous transitions, which is why the rollouts are kept short.

12.4 Multi-step models and successor representations

What kind of model do we need to evaluate a policy? (If we can evaluate it, we can make it better.) The job of the model is to evaluate the policy: $J(\pi) = \mathbb{E}_{s\sim p(s_1)}[V^\pi(s_1)]$. Keep it simple with a state-dependent reward $r(s)$ (action-dependent rewards are easy to re-derive). Then

$$V^\pi(s_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} \mathbb{E}_{p(s_{t'}|s_t)}[r(s_{t'})] = \sum_{t'=t}^{\infty} \gamma^{t'-t} \sum_s p(s_{t'} = s \mid s_t) r(s) = \sum_s \Big(\sum_{t'=t}^{\infty} \gamma^{t'-t} p(s_{t'} = s \mid s_t) \Big) r(s).$$

Define the normalized discounted future-state distribution

$$p_\pi(s_{\text{future}} = s \mid s_t) = (1-\gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p(s_{t'} = s \mid s_t),$$

where the factor $1-\gamma$ just ensures it sums to 1. Then

$$V^\pi(s_t) = \frac{1}{1-\gamma} \sum_s p_\pi(s_{\text{future}} = s \mid s_t) r(s) = \frac{1}{1-\gamma} \mu^\pi(s_t)^T \vec{r},$$

with $\mu_i^\pi(s_t) = p_\pi(s_{\text{future}} = i \mid s_t)$.

Definition 12.1 [Successor representation]. The vector $\mu^\pi(s_t)$ with entries

$$\mu_i^\pi(s_t) = (1-\gamma)\sum_{t'=t}^{\infty} \gamma^{t'-t} p(s_{t'} = i \mid s_t)$$

is the successor representation of s_t. It satisfies a Bellman backup with "reward" $r(s_t) = (1-\gamma)\delta(s_t = i)$:

$$\mu_i^\pi(s_t) = (1-\gamma)\delta(s_t = i) + \gamma \mathbb{E}_{a_t \sim \pi(a_t|s_t),\, s_{t+1}\sim p(s_{t+1}|s_t,a_t)}\left[\mu_i^\pi(s_{t+1})\right].$$

In practice, vectorized backups compute all i at once. A few issues remain: it is not clear whether learning a successor representation is easier than model-free RL, nor how to scale it to large state spaces or extend it to continuous ones.

Successor features. Given features $\phi_j(s)$, define $\psi_j^\pi(s_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t}\mathbb{E}[\phi_j(s_{t'}) \mid s_t] = \frac{1}{1-\gamma}\sum_s \mu_s^\pi(s_t)\phi_j(s)$, that is, $\psi_j^\pi(s_t) = \frac{1}{1-\gamma}\mu^\pi(s_t)^T\vec{\phi}_j$. If $r(s) = \sum_j \phi_j(s) w_j = \phi(s)^T w$, then

$$V^\pi(s_t) = \frac{1}{1-\gamma}\mu^\pi(s_t)^T\vec{r} = \frac{1}{1-\gamma}\sum_j \mu^\pi(s_t)^T\vec{\phi}_j w_j = \sum_j \psi_j^\pi(s_t) w_j = \psi^\pi(s_t)^T w.$$

If the number of features is much less than the number of states, learning them is much easier. They satisfy

$$\psi_j^\pi(s_t) = \phi_j(s_t) + \gamma \mathbb{E}_{a_t \sim \pi(a_t|s_t),\, s_{t+1}\sim p(s_{t+1}|s_t,a_t)}\left[\psi_j^\pi(s_{t+1})\right],$$

and the successor representation is the special case $\phi_i(s_t) = (1-\gamma)\delta(s_t = i)$. A "Q-function-like" version is

$$\psi_j^\pi(s_t, a_t) = \phi_j(s_t) + \gamma \mathbb{E}_{s_{t+1}\sim p(s_{t+1}|s_t,a_t),\, a_{t+1}\sim\pi(a_{t+1}|s_{t+1})}\left[\psi_j^\pi(s_{t+1}, a_{t+1})\right],$$

with $Q^\pi(s_t, a_t) \approx \psi^\pi(s_t, a_t)^T w$ when $r(s_t) \approx \phi(s_t)^T w$.

Using successor features. Idea 1, recover a Q-function very quickly: train $\psi^\pi(s_t, a_t)$ via Bellman backups; get some reward samples $\{s_i, r_i\}$; get $w \leftarrow \arg\min_w \sum_i \|\phi(s_i)^T w - r_i\|^2$; recover $Q^\pi(s_t, a_t) \approx \psi^\pi(s_t, a_t)^T w$, and act by $\pi'(s) = \arg\max_a \psi^\pi(s, a)^T w$. This is not the optimal Q-function: it is equivalent to one step of policy iteration, better than nothing but not optimal. Idea 2, recover many Q-functions: train $\psi^{\pi_k}(s_t, a_t)$ for many policies π_k, fit w as before, recover $Q^{\pi_k}(s_t, a_t) \approx \psi^{\pi_k}(s_t, a_t)^T w$ for every π_k, and act by $\pi'(s) = \arg\max_a \max_k \psi^{\pi_k}(s, a)^T w$, which finds the highest reward policy in each state.

Continuous successor representations. With continuous states the term $(1-\gamma)\delta(s_t = i)$ is always zero for any sampled state. Frame the successor representation as *classification* instead: a binary classifier with $F = 1$ meaning that s_{future} is a future state from s_t, a_t under π, trained on positives

$\mathcal{D}_+ \sim p^\pi(s_{\text{future}} \mid s_t, a_t)$ and negatives $\mathcal{D}_- \sim p^\pi(s)$. The Bayes-optimal classifier is

$$p^\pi(F = 1 \mid s_t, a_t, s_{\text{future}}) = \frac{p^\pi(s_{\text{future}} \mid s_t, a_t)}{p^\pi(s_{\text{future}} \mid s_t, a_t) + p^\pi(s_{\text{future}})},$$
$$p^\pi(F = 0 \mid s_t, a_t, s_{\text{future}}) = \frac{p^\pi(s_{\text{future}})}{p^\pi(s_{\text{future}} \mid s_t, a_t) + p^\pi(s_{\text{future}})},$$

so

$$\frac{p^\pi(F = 1 \mid s_t, a_t, s_{\text{future}})}{p^\pi(F = 0 \mid s_t, a_t, s_{\text{future}})} p^\pi(s_{\text{future}}) = p^\pi(s_{\text{future}} \mid s_t, a_t),$$

where $p^\pi(s_{\text{future}})$ is a constant independent of a_t and s_t. The classifier recovers the successor representation up to this constant.

Algorithm 12.5: C-learning (on-policy)

repeat
  sample a negative $s \sim p^\pi(s)$ (e.g. run the policy, sample from trajectories)
  sample a positive $s \sim p^\pi(s_{\text{future}} \mid s_t, a_t)$ (e.g. pick $s_{t'}$ with $t' = t + \Delta$, $\Pr(\Delta = k) = (1-\gamma)\gamma^k$)
  update $p^\pi(F = 1 \mid s_t, a_t, s)$ using SGD with the cross-entropy loss
until done

This is an on-policy algorithm; an off-policy algorithm can also be derived.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 12 (Fall 2023).

Lecture 13: Exploration (Part 1)

13.1 The exploration problem

Some Atari games, such as Breakout, are easy (mostly); others, such as Montezuma's Revenge, are nearly impossible for standard methods. In Montezuma's Revenge, getting the key and opening a door give reward, but getting killed by a skull gives nothing (is it good? bad?), and finishing the game only weakly correlates with rewarding events. We know what to do because we understand what the sprites mean. Put yourself in the algorithm's shoes with the card game Mao: the only rule you may be told is that one; you incur a penalty when you break a rule, can discover rules only by trial and error, and the rules do not always make sense to you. Temporally extended tasks become increasingly difficult the more extended the task is and the less you know about the rules.

There are two potential definitions of the exploration problem:

- how can an agent discover high-reward strategies that require a temporally extended sequence of complex behaviors that, individually, are not rewarding?
- how can an agent decide whether to attempt new behaviors (to discover ones with higher reward) or continue to do the best thing it knows so far?

They are actually the same problem. Exploitation is doing what you know will yield the highest reward; exploration is doing things you have not done before, in the hope of getting even higher reward. Examples: going to your favorite restaurant or trying a new one; showing the most successful advertisement or a different random one; drilling at the best known location or at a new one.

Exploration is hard. Can we derive an optimal exploration strategy? What does optimal even mean: regret, or a Bayes-optimal strategy? Problems range from theoretically tractable to intractable: multi-armed bandits (one-step stateless RL problems), contextual bandits (one-step RL problems), small finite MDPs (tractable planning, the model-based setting), and large or infinite MDPs with continuous spaces. For bandits and contextual bandits we can formalize exploration as POMDP identification, and policy learning is trivial even with a POMDP. Small finite MDPs can be framed as Bayesian model identification, reasoning explicitly about the value of information. For large or infinite MDPs, optimal methods do not work, but we can take inspiration from optimal methods in smaller settings and use hacks.

13.2 Bandits

The bandit is the drosophila of exploration problems. A one-armed bandit has $\mathcal{A} = \{\text{pull arm}\}$ and an unknown $r(\text{pull arm})$; a multi-armed bandit has $\mathcal{A} = \{\text{pull}_1, \ldots, \text{pull}_n\}$, and we assume $r(a_n) \sim p(r \mid a_n)$, an unknown per-action reward distribution.

Definition 13.1 [Bandit as a POMDP and regret]. Assume $r(a_i) \sim p_{\theta_i}(r_i)$, for example $p(r_i = 1) = \theta_i$ and $p(r_i = 0) = 1 - \theta_i$, with $\theta_i \sim p(\theta)$ but otherwise unknown. This defines a POMDP with state $s = [\theta_1, \ldots, \theta_n]$ and belief state $\hat{p}(\theta_1, \ldots, \theta_n)$. The goodness of an exploration algorithm is measured by the regret, the difference from the optimal policy at time step T:

$$\text{Reg}(T) = T\mathbb{E}[r(a^\star)] - \sum_{t=1}^{T} r(a_t),$$

where $T\mathbb{E}[r(a^\star)]$ is the expected reward of the best action (the best we can hope for in expectation) and $r(a_t)$ is the actual reward of the action actually taken.

Solving the POMDP yields the optimal exploration strategy, but that is overkill, since the belief state is huge; we can do very well with much simpler strategies. There is a variety of relatively simple strategies, often with theoretical guarantees on regret (a variety of optimal algorithms, up to a constant factor), though empirical performance may vary. Exploration strategies for more complex MDPs are inspired by them.

13.3 Three classes of exploration methods

Optimistic exploration. Keep track of the average reward $\hat{\mu}_a$ for each action a. Exploitation picks $a = \arg\max \hat{\mu}_a$; an optimistic estimate picks $a = \arg\max \hat{\mu}_a + C\sigma_a$, with σ_a some sort of variance estimate. The intuition is to try each arm until you are *sure* it is not great. For example (UCB),

$$a = \arg\max_a \hat{\mu}_a + \sqrt{\frac{2 \ln T}{N(a)}},$$

where $N(a)$ is the number of times we picked action a; then $\text{Reg}(T)$ is $O(\log T)$, provably as good as any algorithm.

Probability matching and posterior sampling. With the belief state $\hat{p}(\theta_1, \ldots, \theta_n)$ as a *model* of the bandit: sample $\theta_1, \ldots, \theta_n \sim \hat{p}(\theta_1, \ldots, \theta_n)$, pretend the model $\theta_1, \ldots, \theta_n$ is correct, take the optimal action, update the model, and repeat. This is posterior sampling, or Thompson sampling. It is harder to analyze theoretically but can work very well empirically.

Information gain. Bayesian experimental design: to determine some latent variable z (for example the optimal action, or its value), which action should we take? Let $\mathcal{H}(\hat{p}(z))$ be the current entropy of our z estimate and $\mathcal{H}(\hat{p}(z) \mid y)$ the entropy after an observation y (for example $y = r(a)$); the

lower the entropy, the more precisely we know z. The INFORMATION GAIN

$$\mathrm{IG}(z, y) = \mathbb{E}_y\left[\mathcal{H}(\hat{p}(z)) - \mathcal{H}(\hat{p}(z) \mid y)\right]$$

typically depends on the action, giving $\mathrm{IG}(z, y \mid a)$: how much we learn about z from action a, given current beliefs. For example, information-directed sampling takes $y = r_a$, $z = \theta_a$ (the parameters of the model $p(r_a)$), $g(a) = \mathrm{IG}(\theta_a, r_a \mid a)$ the information gain of a, and $\Delta(a) = \mathbb{E}[r(a^\star) - r(a)]$ the expected suboptimality of a, and chooses

$$a = \arg\min_a \frac{\Delta(a)^2}{g(a)},$$

which avoids actions we are sure are suboptimal and does not bother with actions from which we will not learn anything.

General themes. UCB uses $a = \arg\max \hat{\mu}_a + \sqrt{2 \ln T / N(a)}$; Thompson sampling uses $\theta_1, \ldots, \theta_n \sim \hat{p}(\theta_1, \ldots, \theta_n)$ and $a = \arg\max_a \mathbb{E}_{\theta_a}[r(a)]$; information gain uses $\mathrm{IG}(z, y \mid a)$. Most exploration strategies require some kind of uncertainty estimation (even if naive), and usually assume some value to new information: unknown is good (optimism), the sample is the truth, or information gain is good. Bandits are easier to analyze and understand, give foundations for exploration methods, and these methods can then be applied to more complex MDPs. (Contextual bandits, optimal exploration in small MDPs, Bayesian model-based RL, and PAC exploration are not covered.)

13.4 Optimistic exploration in deep RL

The three classes carry over. Optimistic exploration treats a new state as a good state; it requires estimating state visitation frequencies or novelty and is typically realized by exploration bonuses. Thompson sampling style algorithms learn a distribution over Q-functions or policies, then sample and act according to the sample. Information gain style algorithms reason about the information gain from visiting new states.

In UCB the term $\sqrt{2 \ln T / N(a)}$ is an EXPLORATION BONUS; many functions work, so long as they decrease with $N(a)$. For MDPs, COUNT-BASED EXPLORATION uses $N(s, a)$ or $N(s)$ to add a bonus:

$$r^+(s, a) = r(s, a) + \mathcal{B}(N(s)),$$

with $\mathcal{B}$ decreasing in $N(s)$, and uses r^+ instead of r with any model-free algorithm. This is a simple addition to any RL algorithm, but the bonus weight needs tuning.

The trouble with counts. But what is a count? In Montezuma's Revenge the state includes the positions of the agent and of every sprite; in continuous robotic manipulation we never see the same thing twice. But some states are more similar than others.

Pseudo-counts. Fit a density model $p_\theta(s)$ (or $p_\theta(s, a)$); $p_\theta(s)$ can be high even for a new s if s is

similar to previously seen states. In a small MDP the true probability is $P(s) = N(s)/n$, with count $N(s)$ and n total states visited, and after we see s it becomes $P'(s) = (N(s) + 1)/(n + 1)$. Can we make $p_\theta(s)$ and $p_{\theta'}(s)$ obey these equations?

Algorithm 13.1: Exploring with pseudo-counts

```
repeat
    fit model p_θ(s) to all states 𝒟 seen so far
    take a step i and observe s_i
    fit new model p_θ'(s) to 𝒟 ∪ s_i
    use p_θ(s_i) and p_θ'(s_i) to estimate N̂(s)
    set r_i^+ = r_i + ℬ(N̂(s))
until done
```

To get the PSEUDO-COUNT $\hat{N}(s)$, solve the two equations $p_\theta(s_i) = \hat{N}(s_i)/\hat{n}$ and $p_{\theta'}(s_i) = (\hat{N}(s_i) + 1)/(\hat{n} + 1)$ for the two unknowns:

$$\hat{n} = \frac{1 - p_{\theta'}(s_i)}{p_{\theta'}(s_i) - p_\theta(s_i)}, \qquad \hat{N}(s_i) = \hat{n} p_\theta(s_i) = \frac{1 - p_{\theta'}(s_i)}{p_{\theta'}(s_i) - p_\theta(s_i)} p_\theta(s_i).$$

Bonuses in the literature are inspired by optimal methods for bandits or small MDPs: UCB uses $\mathcal{B}(N(s)) = \sqrt{2 \ln n / N(s)}$, MBIE-EB uses $\mathcal{B}(N(s)) = \sqrt{1/N(s)}$ (the one used with pseudo-counts on Atari), and BEB uses $\mathcal{B}(N(s)) = 1/N(s)$. With the bonus, DQN makes real progress on Montezuma's Revenge and explores many more of its rooms than without it. The density model needs to output densities but need not produce great samples, the opposite consideration from many popular generative models (such as GANs). The CTS model conditions each pixel on its top-left neighborhood; other options are stochastic neural networks, compression length, and EX2.

13.5 More novelty-seeking exploration

Counting with hashes. Still count states, but in a different space: compress s into a k-bit code via $\phi(s)$, then count $N(\phi(s))$. Shorter codes mean more hash collisions; similar states may get the same hash. Improve the odds by *learning* the compression, for example with an autoencoder.

Implicit density modeling with exemplar models. Can we explicitly compare the new state to past states? Intuition: a state is novel if it is easy to distinguish from all previously seen states by a classifier. For each observed state s, fit a classifier D_s to classify that state (positives $\{s\}$) against all past states $\mathcal{D}$ (negatives), and use the classifier's error to obtain a density:

$$p_\theta(s) = \frac{1 - D_s(s)}{D_s(s)},$$

where $D_s(s)$ is the probability that the classifier assigns to s being "positive". Are we just checking whether $s = s$? No: if $s \in \mathcal{D}$, the optimal $D_s(s) \neq 1$; in fact $D_s^\star(s) = \frac{1}{1+p(s)}$, which gives the formula

above. In reality each state is unique, so we regularize the classifier, and instead of one classifier per state we train one *amortized* model, a single network that takes the exemplar as input (EX2).

Heuristic estimation of counts via errors. The model need not even output great densities; it just needs to tell whether a state is novel. Given some target function $f^\star(s,a)$ and the buffer $\mathcal{D} = \{(s_i, a_i)\}$, fit $\hat{f}_\theta(s,a)$ and use $\mathcal{E}(s,a) = \|\hat{f}_\theta(s,a) - f^\star(s,a)\|^2$ as the bonus (Figure 14). One common choice of target is $f^\star(s,a) = s'$, next-state prediction, which is also related to information gain. Even simpler, $f^\star(s,a) = f_\phi(s,a)$ with a *random* parameter vector ϕ: RANDOM NETWORK DISTILLATION.

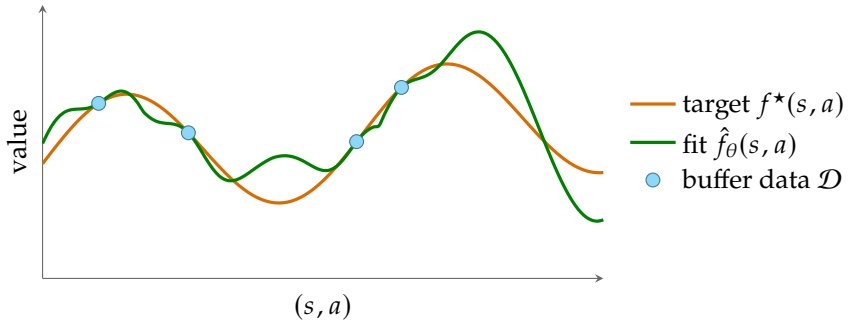


Figure 14: Error-based novelty: $\hat{f}_\theta$ matches the target near buffer data (low novelty) and differs from it far away (high novelty). Schematic.

13.6 Posterior sampling in deep RL

Thompson sampling samples $\theta_1, \ldots, \theta_n \sim \hat{p}(\theta_1, \ldots, \theta_n)$ and takes $a = \arg\max_a \mathbb{E}_{\theta_a}[r(a)]$. What do we sample, and how do we represent the distribution? In the bandit setting $\hat{p}$ is a distribution over rewards; the MDP analog is the Q-function. Sample a Q-function Q from $p(Q)$, act according to Q for one episode, update $p(Q)$, and repeat. Since Q-learning is off-policy, we do not care which Q-function was used to collect the data.

How can we represent a distribution over functions? Use the bootstrap: given a dataset $\mathcal{D}$, resample with replacement N times to get $\mathcal{D}_1, \ldots, \mathcal{D}_N$, train each model f_{θ_i} on $\mathcal{D}_i$, and to sample from $p(\theta)$, sample $i \in [1, \ldots, N]$ and use f_{θ_i}. Training N big neural nets is expensive, so use a shared network with K heads (bootstrapped DQN).

Why does this work? Exploring with random actions (for example ϵ-greedy) oscillates back and forth and might not go to a coherent or interesting place. Exploring with random Q-functions commits to a randomized but internally consistent strategy for an entire episode. There is no change to the original reward function, but very good bonuses often do better.

13.7 Information gain in deep RL

Information gain about what? About the reward $r(s,a)$: not very useful if the reward is sparse. About the state density $p(s)$: a bit strange, but it somewhat makes sense. About the dynamics

$p(s' \mid s, a)$: a good proxy for *learning* the MDP, though still heuristic. Information gain is generally intractable to use exactly, regardless of what is being estimated. A few approximations:

- *prediction gain* $\log p_{\theta'}(s) - \log p_\theta(s)$: if the density changed a lot, the state was novel;
- *variational inference* (VIME).

VIME. Information gain can be written equivalently as $D_{\mathrm{KL}}(p(z \mid y) \| p(z))$. Learn about transitions $p_\theta(s_{t+1} \mid s_t, a_t)$, with $z = \theta$ and $y = (s_t, a_t, s_{t+1})$:

$$D_{\mathrm{KL}}\big(p(\theta \mid h, s_t, a_t, s_{t+1}) \,\|\, p(\theta \mid h)\big),$$

where θ are the model parameters, h is the history of all prior transitions, and (s_t, a_t, s_{t+1}) is the newly observed transition. A transition is more informative if it causes the belief over θ to change. Use variational inference to estimate $q(\theta \mid \phi) \approx p(\theta \mid h)$, optimizing the variational lower bound $D_{\mathrm{KL}}(q(\theta \mid \phi) \| p(h \mid \theta) p(\theta))$, and represent $q(\theta \mid \phi)$ as a product of independent Gaussian parameter distributions with mean ϕ, $p(\theta \mid \mathcal{D}) = \prod_i p(\theta_i \mid \mathcal{D})$ with $p(\theta_i \mid \mathcal{D}) = \mathcal{N}(\mu_i, \sigma_i)$. Given a new transition (s, a, s'), update ϕ to get ϕ', that is, update the network weight means and variances, and use $D_{\mathrm{KL}}(q(\theta \mid \phi') \| q(\theta \mid \phi))$ as the approximate bonus. This approximate information gain has an appealing mathematical formalism, but the models are more complex and generally harder to use effectively.

Exploration with model errors. $D_{\mathrm{KL}}(q(\theta \mid \phi') \| q(\theta \mid \phi))$ can be seen as the change in the network (mean) parameters ϕ. If we forget about information gain, there are many other ways to measure this: encode image observations with an autoencoder, build a predictive model on its latent states, and use the model error as the exploration bonus; or give exploration bonuses for model error, for model gradient, and many other variations.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 13 (Fall 2023).

Lecture 14: Exploration (Part 2)

14.1 Unsupervised learning of diverse behaviors

What if we want to recover diverse behavior without any reward function at all? Reasons: to learn skills without supervision and then use them to accomplish goals, to learn sub-skills for hierarchical reinforcement learning, and to explore the space of possible behaviors. For example, a robot placed in a kitchen at training time, with no task specified, should prepare for an unknown future goal (such as doing the dishes) that will be given later.

14.2 Information theory

For a distribution $p(x)$, for example over observations x, the entropy

$$\mathcal{H}(p(x)) = -\mathbb{E}_{x\sim p(x)}[\log p(x)]$$

measures how "broad" $p(x)$ is. The mutual information

$$\mathcal{I}(x;y) = D_{\mathrm{KL}}\big(p(x,y)\|p(x)p(y)\big) = \mathbb{E}_{(x,y)\sim p(x,y)}\left[\log \frac{p(x,y)}{p(x)p(y)}\right] = \mathcal{H}(p(y)) - \mathcal{H}(p(y \mid x))$$

is high when x and y are dependent and low when they are independent.

In RL, $\pi(s)$ denotes the state *marginal* distribution of policy π, and $\mathcal{H}(\pi(s))$ its state marginal entropy, which quantifies coverage. An example of mutual information is empowerment,

$$\mathcal{I}(s_{t+1}; a_t) = \mathcal{H}(s_{t+1}) - \mathcal{H}(s_{t+1} \mid a_t),$$

which can be viewed as quantifying "control authority" in an information-theoretic way: the next state should be diverse, yet predictable from the action.

14.3 Learning without a reward function by reaching goals

Train a latent variable model of images, such as a VAE with prior $p(z)$, decoder $p_\theta(x \mid z)$, and encoder $q_\phi(z \mid x)$ (but there are many other choices). A goal image x_g given at test time is encoded to z_g, and the policy must reach it. Without any rewards, the robot can imagine its own goals $z_{g_1}, z_{g_2}, \ldots$:

Algorithm 14.1: Reinforcement learning with imagined goals

repeat
  propose goal: $z_g \sim p(z)$, $x_g \sim p_\theta(x_g \mid z_g)$
  attempt to reach the goal using $\pi(a \mid x, x_g)$, reaching $\bar{x}$
  use the data to update π
  use the data to update $p_\theta(x_g \mid z_g)$, $q_\phi(z_g \mid x_g)$
until done

How do we get diverse goals? The generative model is fit to the states the agent reached, so it proposes goals like those it already reaches. Standard maximum likelihood fits $\theta, \phi \leftarrow \arg\max_{\theta,\phi} \mathbb{E}[\log p(\bar{x})]$. Skew-Fit instead uses a weighted maximum likelihood,

$$\theta, \phi \leftarrow \arg\max_{\theta,\phi} \mathbb{E}\big[w(\bar{x}) \log p(\bar{x})\big], \qquad w(\bar{x}) = p_\theta(\bar{x})^\alpha.$$

The key result is that for any $\alpha \in [-1, 0)$ the entropy $\mathcal{H}(p_\theta(x))$ increases: rare states are up-weighted, the goal distribution spreads out, and the policy is then asked to reach more diverse goals.

What is the objective? Skew-Fit makes the goals get higher entropy, increasing $\mathcal{H}(p(G))$. RL trains $\pi(a \mid S, G)$ to reach goal G; as π gets better, the final state S gets close to G, so $p(G \mid S)$ becomes more deterministic, decreasing $\mathcal{H}(p(G \mid S))$. Together,

$$\max \ \mathcal{H}(p(G)) - \mathcal{H}(p(G \mid S)) = \max \ \mathcal{I}(S; G) :$$

maximizing mutual information between S and G leads to good exploration (state coverage, $\mathcal{H}(p(G))$) and effective goal reaching ($\mathcal{H}(p(G \mid S))$).

14.4 State distribution matching

Exploration with intrinsic motivation. A common method for exploration incentivizes the policy $\pi(a \mid s)$ to explore diverse states before seeing any reward, by rewarding novel states: if a state is visited often, it is not novel. So add an exploration bonus to the reward,

$$\tilde{r}(s) = r(s) - \log p_\pi(s),$$

with $p_\pi(s)$ the state density under $\pi(a \mid s)$, and alternate: update $\pi(a \mid s)$ to maximize $\mathbb{E}_\pi[\tilde{r}(s)]$, then update $p_\pi(s)$ to fit the state marginal. The policy keeps chasing whatever is currently novel.

Definition 14.1 [State marginal matching]. The state marginal matching problem is to learn $\pi(a \mid s)$ so as to minimize $D_{\mathrm{KL}}(p_\pi(s) \| p^\star(s))$ for a target state density $p^\star(s)$.

Can intrinsic motivation solve it? With $\tilde{r}(s) = \log p^\star(s) - \log p_\pi(s)$, the alternating procedure above does not perform marginal matching: the final policy only chases the latest novelty. The fix is:

Algorithm 14.2: State marginal matching

repeat
  learn $\pi^k(a \mid s)$ to maximize $\mathbb{E}_\pi[\tilde{r}^k(s)]$
  update $p_{\pi^k}(s)$ to fit *all* states seen so far
until done
return $\pi^\star(a \mid s) = \sum_k \pi^k(a \mid s)$ (the mixture of all iterates)

This does perform marginal matching: $p_\pi(s) = p^\star(s)$ is the Nash equilibrium of the two-player game between π^k and p_{π^k}. In the special case $\log p^\star(s) = C$, a uniform target, $D_{\mathrm{KL}}(p_\pi(s) \| U(s)) = -\mathcal{H}(p_\pi(s)) + \text{const}$, so matching the uniform target maximizes state entropy. On a three-armed maze the resulting policies cover the state space much better than maximum entropy on actions alone (as in SAC), and reach many more goals.

14.5 Is state coverage a good exploration objective?

Skew-Fit maximizes $\mathcal{H}(p(G)) - \mathcal{H}(p(G \mid S)) = \mathcal{I}(S; G)$, and state marginal matching with $p^\star(s) = C$ maximizes $\mathcal{H}(p_\pi(S))$; these are more or less the same thing. When is this a good idea?

Proposition 14.2 [Maximum entropy goals are minimax optimal]. *Suppose that at test time an adversary will choose the worst goal G. Then the goal distribution to use for training is $p(G) = \arg\max_p \mathcal{H}(p(G))$.*

This follows easily from classic maximum entropy modeling: without knowing which goal will be asked, covering all goals as uniformly as possible is the best defense against the worst case.

14.6 Covering the space of skills

Reaching diverse goals is not the same as performing diverse tasks: not all behaviors can be captured by goal reaching (for example, reaching a goal while avoiding a region). Learn a skill-conditioned policy $\pi(a \mid s, z)$, where z is a task index. Intuition: different skills should visit different state-space regions. Use a diversity-promoting reward that rewards states that are unlikely for other $z' \neq z$:

$$\pi(a \mid s, z) = \arg\max_\pi \sum_z \mathbb{E}_{s \sim \pi(s|z)}[r(s, z)], \qquad r(s, z) = \log p(z \mid s),$$

where $p(z \mid s)$ is a learned discriminator that predicts the skill from the state (Figure 15). Policy and discriminator are trained together; on the cheetah, ant, and mountain car tasks this yields visibly distinct skills, and in a 2D navigation task the skills head in different directions.

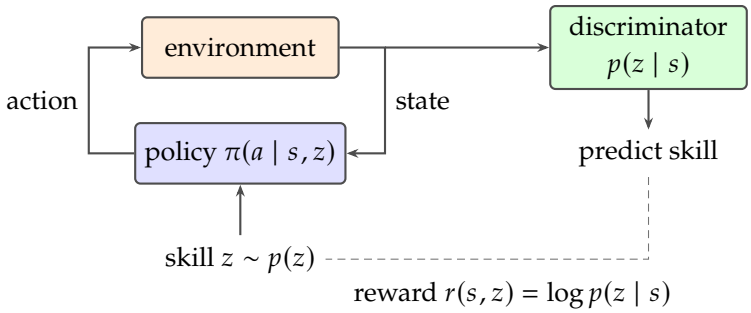


Figure 15: Learning diverse skills: the policy is rewarded when a discriminator can recover the skill z from the visited states.

A connection to mutual information. The same objective maximizes

$$\mathcal{I}(z, s) = \mathcal{H}(z) - \mathcal{H}(z \mid s),$$

where $\mathcal{H}(z)$ is maximized by using a uniform prior $p(z)$, and $\mathcal{H}(z \mid s)$ is minimized by maximizing $\log p(z \mid s)$.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 14 (Fall 2023).

Lecture 15: Offline Reinforcement Learning (Part 1)

15.1 Why offline reinforcement learning?

There is an enormous gulf between the settings where RL succeeds (Atari, simulated locomotion, robotic manipulation, Go), in which the collect-then-train loop is run many times, and the settings where modern machine learning works: large, diverse datasets combined with large models. Can we develop data-driven RL methods? Take big datasets from past interaction, train for many epochs, and only occasionally get more data.

Definition 15.1 [Offline reinforcement learning]. In offline reinforcement learning (batch RL, sometimes fully off-policy RL) we are given a dataset $\mathcal{D} = \{(s_i, a_i, s'_i, r_i)\}$ collected once by some behavior policy π_β, with

$$s \sim d^{\pi_\beta}(s), \qquad a \sim \pi_\beta(a \mid s), \qquad s' \sim p(s' \mid s, a), \qquad r \leftarrow r(s, a),$$

where π_β is generally *not* known. The goal is to learn the best possible policy π_θ for the RL objective $\max_\pi \sum_{t=0}^{T} \mathbb{E}_{s_t \sim d^\pi(s), a_t \sim \pi(a|s)}[\gamma^t r(s_t, a_t)]$ without further interaction, and then deploy it (Figure 16). Off-policy evaluation (OPE) instead asks, given $\mathcal{D}$, to estimate $J(\pi) = \mathbb{E}_\pi\left[\sum_{t=1}^{T} r(s_t, a_t)\right]$.

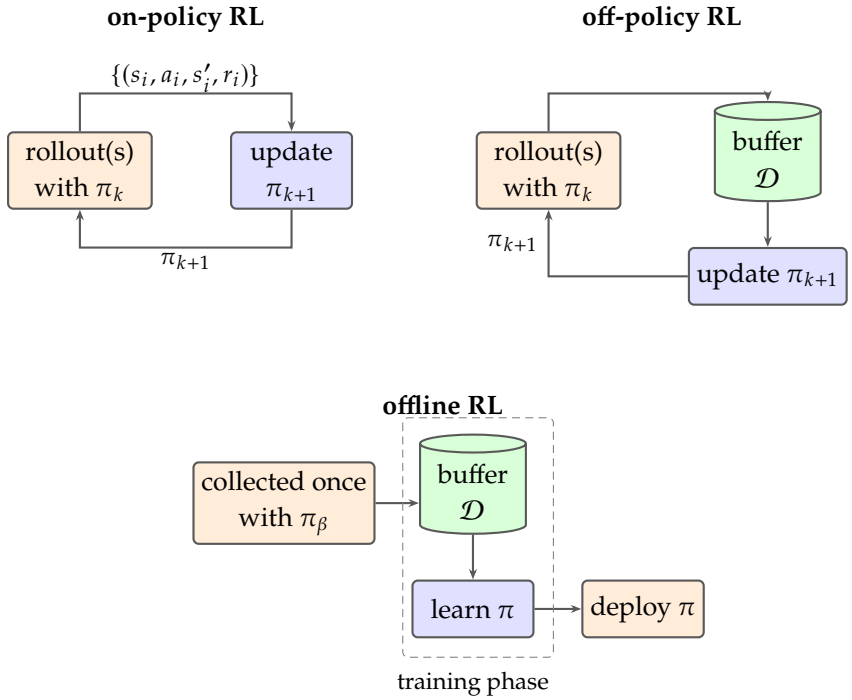


Figure 16: On-policy RL collects new data with each policy; off-policy RL adds it to a buffer; offline RL learns from a fixed buffer collected once by π_β and then deploys the policy.

"Best possible" is not necessarily obvious. How is this even possible? By finding the "good stuff" in a dataset full of good and bad behaviors; by generalization, since good behavior in one place may suggest good behavior in another; and by STITCHING, recombining parts of good behaviors (if the data contains $A \to B$ and $B \to C$, learn $A \to C$; in a maze, combine segments of many trajectories into a path never seen in full).

A bad intuition is that offline RL is like imitation learning (though it can be shown to be provably better than imitation learning even with optimal data, under some structural assumptions). A better intuition is getting order from chaos. Macro-scale stitching is just the clearest example; with algorithms that properly perform dynamic programming, micro-scale stitching at every step can yield near-optimal policies from highly suboptimal data. For instance, a robot trained to grasp from an open drawer can, using prior unlabeled data of opening drawers and removing obstacles, grasp when the drawer is closed or blocked. At scale, QT-Opt trained grasping from 580k offline grasp attempts to an 87% success rate, and to 96% after fine-tuning with 28k additional online attempts.

15.2 Why is offline RL hard?

Running off-policy Q-learning on fixed datasets of increasing size, the actual return stays poor while the learned Q-values (on a log scale) grow without bound: massive overestimation. The fundamental problem is COUNTERFACTUAL QUERIES. The training data shows a car driving straight; the policy wants to swerve. Is this good or bad? How do we know, if we did not see it in the data? Online RL algorithms do not have to handle this, since they can simply try the action and see what happens. Offline RL methods must somehow account for these unseen ("out-of-distribution") actions, ideally in a safe way, while still using generalization to find behaviors better than the best thing seen in the data.

Distribution shift in a nutshell. Consider the empirical risk minimization problem

$$\theta \leftarrow \arg\min_{\theta} \mathbb{E}_{x\sim p(x),\, y\sim p(y|x)}\left[(f_\theta(x) - y)^2\right].$$

Given some $x^\star$, is $f_\theta(x^\star)$ correct? The expected error under $x \sim p(x)$ is low, but under $x \sim \bar{p}(x)$ it is not, for general $\bar{p}(x) \neq p(x)$. If $x^\star \sim p(x)$ we are usually not worried, since neural nets generalize well. But what if we pick $x^\star \leftarrow \arg\max_x f_\theta(x)$? Then we find exactly the point where f_θ errs upward.

Where do we suffer from distribution shift? Q-learning (with an actor) uses the target $Q(s,a) \leftarrow r(s,a) + \mathbb{E}_{a'\sim\pi_{\text{new}}}[Q(s',a')] = y(s,a)$ and minimizes $\min_Q \mathbb{E}_{(s,a)\sim\pi_\beta(s,a)}\left[(Q(s,a) - y(s,a))^2\right]$ over the behavior distribution. We expect good accuracy when $\pi_\beta(a \mid s) = \pi_{\text{new}}(a \mid s)$, which rarely happens. Even worse, $\pi_{\text{new}} = \arg\max_\pi \mathbb{E}_{a\sim\pi(a|s)}[Q(s,a)]$, which is exactly picking $x^\star \leftarrow \arg\max_x f_\theta(x)$.

Issues with generalization are not corrected. Online, if the Q-function overestimates some action, the policy takes it, observes the outcome, and the error is corrected; offline, it never is. Existing challenges with sampling error and function approximation error in standard RL become much

more severe in offline RL.

15.3 Batch RL via importance sampling

The policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\Big[\sum_{t=0}^{T}\nabla_\theta\gamma^t\log\pi_\theta(a_t\mid s_t)\hat{Q}(s_t,a_t)\Big] \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta\gamma^t\log\pi_\theta(a_{t,i}\mid s_{t,i})\hat{Q}(s_{t,i},a_{t,i})$$

requires sampling from π_θ. With samples only from π_β, use importance sampling:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\frac{\pi_\theta(\tau_i)}{\pi_\beta(\tau_i)}\sum_{t=0}^{T}\nabla_\theta\gamma^t\log\pi_\theta(a_{t,i}\mid s_{t,i})\hat{Q}(s_{t,i},a_{t,i}), \qquad \frac{\pi_\theta(\tau)}{\pi_\beta(\tau)} = \prod_t\frac{\pi_\theta(a_t\mid s_t)}{\pi_\beta(a_t\mid s_t)},$$

where the initial-state and dynamics factors cancel. The weight is exponential in T, and the weights are likely to be degenerate as T becomes large. Using causality,

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\Big(\prod_{t'=0}^{t-1}\frac{\pi_\theta(a_{t',i}\mid s_{t',i})}{\pi_\beta(a_{t',i}\mid s_{t',i})}\Big)\nabla_\theta\gamma^t\log\pi_\theta(a_{t,i}\mid s_{t,i})\Big(\prod_{t'=t}^{T}\frac{\pi_\theta(a_{t',i}\mid s_{t',i})}{\pi_\beta(a_{t',i}\mid s_{t',i})}\Big)\hat{Q}(s_{t,i},a_{t,i}).$$

The first product accounts for the difference in the probability of landing in $s_{t,i}$ (we have $s_t\sim d^{\pi_\beta}(s_t)$ but want $s_t\sim d^{\pi_\theta}(s_t)$); dropping it is the approximation justified in Lecture 9. The second accounts for having an incorrect $\hat{Q}(s_{t,i},a_{t,i})$. Estimating the return itself by importance sampling,

$$\Big(\prod_{t'=t}^{T}\frac{\pi_\theta(a_{t',i}\mid s_{t',i})}{\pi_\beta(a_{t',i}\mid s_{t',i})}\Big)\hat{Q}(s_{t,i},a_{t,i}) \approx \sum_{t'=t}^{T}\Big(\prod_{t''=t}^{t'}\frac{\pi_\theta(a_{t'',i}\mid s_{t'',i})}{\pi_\beta(a_{t'',i}\mid s_{t'',i})}\Big)\gamma^{t'-t}r_{t',i},$$

is still exponential. To avoid exponentially exploding importance weights, we must use value function estimation: imagine we knew $Q^{\pi_\theta}(s,a)$.

The doubly robust estimator. With $\rho_t = \pi_\theta(a_t\mid s_t)/\pi_\beta(a_t\mid s_t)$, the importance-sampled value is

$$\begin{aligned}V^{\pi_\theta}(s_0) &\approx \sum_{t=0}^{T}\Big(\prod_{t'=0}^{t}\rho_{t'}\Big)\gamma^t r_t = \rho_0 r_0 + \rho_0\gamma\rho_1 r_1 + \rho_0\gamma\rho_1\gamma\rho_2 r_2 + \cdots\\ &= \rho_0\big(r_0 + \gamma(\rho_1(r_1 + \gamma(\rho_2(r_2 + \cdots))))\big) = \bar{V}^T,\end{aligned}$$

where $\bar{V}^{T+1-t} = \rho_t(r_t + \gamma\bar{V}^{T-t})$, which is exponential. In the bandit case, DOUBLY ROBUST estimation uses a model or function approximator $\hat{V}$, $\hat{Q}$ as a control variate: $V_{\mathrm{DR}}(s) = \hat{V}(s) + \rho(s,a)(r_{s,a} - \hat{Q}(s,a))$. Applying it at every step of the recursion,

$$\bar{V}_{\mathrm{DR}}^{T+1-t} = \hat{V}(s_t) + \rho_t\big(r_t + \gamma\bar{V}_{\mathrm{DR}}^{T-t} - \hat{Q}(s_t,a_t)\big).$$

Marginalized importance sampling. Instead of using $\prod_t \pi_\theta(a_t \mid s_t)/\pi_\beta(a_t \mid s_t)$, estimate the ratio of state-action marginals

$$w(s,a) = \frac{d^{\pi_\theta}(s,a)}{d^{\pi_\beta}(s,a)};$$

then $J(\theta) \approx \frac{1}{N}\sum_i w(s_i,a_i)r_i$. This is typically done for off-policy evaluation rather than policy learning. To determine $w(s,a)$, solve some kind of consistency condition, typically a fixed point problem; for example (GenDICE),

$$d^{\pi_\beta}(s',a')w(s',a') = (1-\gamma)p_0(s')\pi_\theta(a' \mid s') + \gamma \sum_{s,a} \pi_\theta(a' \mid s')p(s' \mid s,a)d^{\pi_\beta}(s,a)w(s,a),$$

where the first term is the probability of starting in (s',a') and the second the probability of transitioning into (s',a').

15.4 Batch RL via linear fitted value functions

Older work extended approximate dynamic programming and Q-learning to the offline setting and derived tractable solutions with simple (for example linear) function approximators. Current work derives approximate solutions with highly expressive function approximators such as deep nets, where the primary challenge turns out to be distributional shift, which older work generally did not focus on (maybe because it was less of a problem with linear models). For completeness:

Warmup: linear models. Let Φ be the $|S| \times K$ feature matrix (equivalently a vector-valued function $\Phi(s)$), and do offline model-based RL in feature space, for a fixed policy π with transition matrix P^π on states:

1. reward model $\Phi w_r \approx r$, by least squares $w_r = (\Phi^T\Phi)^{-1}\Phi^T\vec{r}$;
2. transition model $\Phi P_\Phi \approx P^\pi\Phi$, with the $K \times K$ estimated feature-space transition matrix $P_\Phi = (\Phi^T\Phi)^{-1}\Phi^T P^\pi\Phi$;
3. recover the value function $V^\pi \approx V^\pi_\Phi = \Phi w_V$;
4. improve the policy.

Since $V^\pi = r + \gamma P^\pi V^\pi$ gives $V^\pi = (I - \gamma P^\pi)^{-1}r$, the same equation in feature space gives $w_V = (I - \gamma P_\Phi)^{-1}w_r$. Do we even need the model? Substituting,

$$w_V = \left(I - \gamma(\Phi^T\Phi)^{-1}\Phi^T P^\pi\Phi\right)^{-1}(\Phi^T\Phi)^{-1}\Phi^T\vec{r} = \left(\Phi^T\Phi - \gamma\Phi^T P^\pi\Phi\right)^{-1}\Phi^T\vec{r},$$

which is LEAST-SQUARES TEMPORAL DIFFERENCE (LSTD).

Doing it all with samples. With $\mathcal{D} = \{(s_i,a_i,r_i,s'_i)\}$, let $\Phi_i = \phi(s_i)$ (now one row per sample), $\vec{r}_i = r(s_i,a_i)$, and replace $P^\pi\Phi$ by Φ' with $\Phi'_i = \phi(s'_i)$. Everything else works exactly the same way, only now with some sampling error.

Improving the policy. The typical step $\pi'(s) \leftarrow \text{Greedy}(\Phi w_V)$, then estimate $V^{\pi'}$, is not going to work offline: replacing $P^\pi \Phi$ by Φ' requires samples from π. Least-squares policy iteration (LSPI) replaces LSTD with LSTDQ, LSTD for Q-functions, using features $\phi(s, a)$ over the $|S||A|$ state-action tuples (typically replicated for each action):

$$w_Q = \left(\Phi^T \Phi - \gamma \Phi^T \Phi'\right)^{-1} \Phi^T \vec{r}, \qquad \Phi'_i = \phi(s'_i, \pi(s'_i)),$$

which encodes the action π would take, not the action in the data.

Algorithm 15.1: Least-squares policy iteration (LSPI)

```
repeat
    compute w_Q for π_k
    π_{k+1}(s) = arg max_a φ(s, a) w_Q
    set Φ'_i = φ(s'_i, π_{k+1}(s'_i))
until converged
```

What is the issue? The same as before: Φ' queries the Q-function at actions chosen by maximizing it, the counterfactual queries above. In general, all approximate dynamic programming methods (such as fitted value or Q-iteration) suffer from action distributional shift, and we must fix it.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 15 (Fall 2023).

Lecture 16: Offline Reinforcement Learning (Part 2)

16.1 Policy constraint methods

Recall the source of trouble (Lecture 15): the target $Q(s,a) \leftarrow r(s,a) + \mathbb{E}_{a' \sim \pi_{\text{new}}}[Q(s',a')]$ queries actions from $\pi_{\text{new}} = \arg\max_\pi \mathbb{E}_{a \sim \pi(a|s)}[Q(s,a)]$, while Q is trained only on actions from π_β. Prior methods alternate

$$Q(s,a) \leftarrow r(s,a) + \mathbb{E}_{a' \sim \pi_{\text{new}}}[Q(s',a')],$$
$$\pi_{\text{new}}(a \mid s) = \arg\max_\pi \mathbb{E}_{a \sim \pi(a|s)}[Q(s,a)] \quad \text{s.t.} \quad D_{\text{KL}}(\pi \| \pi_\beta) \le \epsilon,$$

a policy constraint method: a very old idea (KL-divergence control, passive dynamics in linearly solvable MDPs, trust regions, covariant and natural policy gradients) used in some form in many recent papers. This solves distribution shift, right? No more erroneous values? Issue 1: we usually do not know the behavior policy, since the data may be human-provided, come from a hand-designed controller, from many past RL runs, or all of the above. Issue 2: the constraint is both too pessimistic and not pessimistic enough.

What kinds of constraints? The KL divergence $D_{\text{KL}}(\pi \| \pi_\beta)$ is easy to implement but not necessarily what we want: the best policy under a KL constraint stays close to the shape of π_β even where the Q-values are reliable. A support constraint, $\pi(a \mid s) \ge 0$ only if $\pi_\beta(a \mid s) \ge \epsilon$ (approximated, for example, with MMD), is significantly more complex to implement but much closer to what we really want: the best in-support policy, avoiding only actions with unreliable out-of-distribution values.

Implementing explicit constraints. Modify the actor objective: since $D_{\text{KL}}(\pi \| \pi_\beta) = \mathbb{E}_\pi[\log \pi(a \mid s) - \log \pi_\beta(a \mid s)] = -\mathbb{E}_\pi[\log \pi_\beta(a \mid s)] - \mathcal{H}(\pi)$, use

$$\theta \leftarrow \arg\max_\theta \mathbb{E}_{s \sim \mathcal{D}}\Big[\mathbb{E}_{a \sim \pi_\theta(a|s)}\big[Q(s,a) + \lambda \log \pi_\beta(a \mid s)\big] + \lambda \mathcal{H}(\pi(a \mid s))\Big],$$

with Lagrange multiplier λ; the entropy is easy to compute and differentiate for Gaussian or categorical policies. Or modify the reward function, $\bar{r}(s,a) = r(s,a) - D(\pi, \pi_\beta)$, a simple modification that directly penalizes divergence and also accounts for *future* divergence. Generally, the best modern offline RL methods do not do either of these things.

Implicit policy constraints. The constrained problem $\pi_{\text{new}}(a \mid s) = \arg\max_\pi \mathbb{E}_{a \sim \pi(a|s)}[Q(s,a)]$ s.t. $D_{\text{KL}}(\pi \| \pi_\beta) \le \epsilon$ has the solution

$$\pi^\star(a \mid s) = \frac{1}{Z(s)} \pi_\beta(a \mid s) \exp\Big(\frac{1}{\lambda} A^\pi(s,a)\Big),$$

which is straightforward to show via duality. Approximate it by *weighted* maximum likelihood on samples from the dataset ($a \sim \pi_\beta(a \mid s)$), with the critic giving the weights:

$$\pi_{\text{new}}(a \mid s) = \arg\max_\pi \mathbb{E}_{(s,a)\sim\pi_\beta}\Big[\log \pi(a \mid s) \underbrace{\frac{1}{Z(s)} \exp\Big(\frac{1}{\lambda} A^{\pi_{\text{old}}}(s,a)\Big)}_{w(s,a)}\Big].$$

This gives advantage-weighted regression and advantage-weighted actor-critic (AWAC), with critic and actor losses

$$\mathcal{L}_C(\phi) = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\Big[\big(Q_\phi(s,a) - (r(s,a) + \gamma \mathbb{E}_{a'\sim\pi_\theta(a'|s')}[Q_\phi(s',a')])\big)^2\Big],$$
$$\mathcal{L}_A(\theta) = -\mathbb{E}_{(s,a)\sim\pi_\beta}\Big[\log \pi_\theta(a \mid s) \frac{1}{Z(s)} \exp\Big(\frac{1}{\lambda} A^{\pi_{\text{old}}}(s,a)\Big)\Big],$$

alternating $\phi \leftarrow \phi - \alpha\nabla_\phi \mathcal{L}_C(\phi)$ and $\theta \leftarrow \theta - \alpha\nabla_\theta \mathcal{L}_A(\theta)$. The actor update only uses actions from the data. The critic target still queries $a' \sim \pi_\theta$, though.

16.2 Implicit Q-learning

Can we also avoid all out-of-distribution actions in the Q update? In $Q(s,a) \leftarrow r(s,a) + \mathbb{E}_{a'\sim\pi_{\text{new}}}[Q(s',a')]$, the expectation $V(s')$ is just another neural network, which could be fit by

$$V \leftarrow \arg\min_V \frac{1}{N} \sum_{i=1}^N \ell\big(V(s_i), Q(s_i,a_i)\big),$$

where the action a_i comes from π_β, not from π_{new}. With the MSE loss $(V(s_i) - Q(s_i,a_i))^2$ we get $\mathbb{E}_{a\sim\pi_\beta}[Q(s,a)]$, the value of the behavior policy. But the distribution of targets is induced by actions only (states are shared by nearby samples), and its upper tail is the value of the best policy supported by the data. Another loss can give us that. The EXPECTILE loss

$$\ell_2^\tau(x) = \begin{cases} (1-\tau)x^2 & \text{if } x > 0, \\ \tau x^2 & \text{otherwise,} \end{cases}$$

applied to $x = V(s) - Q(s,a)$ with large τ, penalizes V falling below Q much more than rising above it (Figure 17). With ℓ_2^τ for big τ,

$$V(s) \leftarrow \max_{a\in\Omega(s)} Q(s,a), \qquad \Omega(s) = \{a : \pi_\beta(a \mid s) \geq \epsilon\}.$$

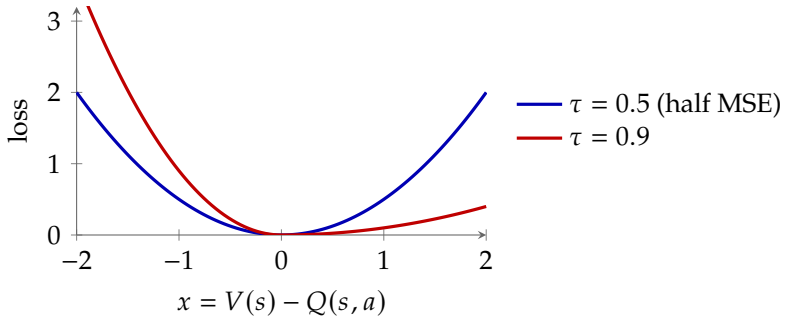


Figure 17: The expectile loss ℓ_2^τ for $\tau = 0.5$ (half the squared error) and $\tau = 0.9$, which penalizes V below Q nine times more than above.

Definition 16.1 [Implicit Q-learning]. Implicit Q-learning (IQL) performs Q-learning with *implicit* policy improvement:

$$Q(s,a) \leftarrow r(s,a) + V(s'), \qquad V \leftarrow \arg\min_V \frac{1}{N} \sum_{i=1}^{N} \ell_2^\tau\big(V(s_i), Q(s_i,a_i)\big).$$

For big τ this amounts to $Q(s,a) \leftarrow r(s,a) + \max_{a' \in \Omega(s')} Q(s',a')$, the backup of the "implicit" policy $\pi_{\text{new}}(a \mid s) = \delta\big(a = \arg\max_{a \in \Omega(s)} Q(s,a)\big)$.

Now value function updates never risk out-of-distribution actions: both Q and V are only evaluated at dataset actions. (A policy for deployment can then be extracted, for example by advantage-weighted regression.)

16.3 Conservative Q-learning

The second option is to fix the Q-values directly. Since the problem is overestimated Q-values for actions the policy picks, add a term that pushes down big Q-values:

$$\hat{Q}^\pi = \arg\min_Q \max_\mu \; \alpha \mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(a|s)}[Q(s,a)] + \mathbb{E}_{(s,a,s') \sim \mathcal{D}}\Big[\big(Q(s,a) - (r(s,a) + \mathbb{E}_\pi[Q(s',a')])\big)^2\Big].$$

One can show that $\hat{Q}^\pi \leq Q^\pi$, the true Q-function, for large enough α. This always pushes Q-values down. A better bound also pushes up on (s,a) samples in the data:

$$\mathcal{L}_{\text{CQL}}(\hat{Q}^\pi) = \max_\mu \; \alpha \mathbb{E}_{s \sim \mathcal{D}, a \sim \mu(a|s)}[Q(s,a)] - \alpha \mathbb{E}_{(s,a) \sim \mathcal{D}}[Q(s,a)] - \mathcal{R}(\mu) \\ + \mathbb{E}_{(s,a,s') \sim \mathcal{D}}\Big[\big(Q(s,a) - (r(s,a) + \mathbb{E}_\pi[Q(s',a')])\big)^2\Big],$$

with $\hat{Q}^\pi = \arg\min_Q \mathcal{L}_{\text{CQL}}$ and a regularizer $\mathcal{R}(\mu)$. It is no longer guaranteed that $\hat{Q}^\pi(s,a) \leq Q^\pi(s,a)$ for all (s,a), but it is guaranteed that $\mathbb{E}_{\pi(a|s)}[\hat{Q}^\pi(s,a)] \leq \mathbb{E}_{\pi(a|s)}[Q^\pi(s,a)]$ for all $s \in \mathcal{D}$.

A common choice is maximum entropy regularization, $\mathcal{R} = \mathbb{E}_{s\sim\mathcal{D}}[\mathcal{H}(\mu(\cdot \mid s))]$. The optimal choice is then $\mu(a \mid s) \propto \exp(Q(s,a))$, and the regularized maximum equals $\log \sum_a \exp(Q(s,a))$: for discrete actions, just calculate it directly; for continuous actions, use importance sampling to estimate $\mathbb{E}_{a\sim\mu(a|s)}[Q(s,a)]$.

Algorithm 16.1: Conservative Q-learning (CQL)

repeat
  update $\hat{Q}^\pi$ with respect to $\mathcal{L}_{\text{CQL}}(\hat{Q}^\pi)$ using $\mathcal{D}$
  update policy π: if actions are discrete, $\pi(a \mid s) = 1$ if $a = \arg\max_a \hat{Q}(s,a)$ and 0 otherwise; if continuous, $\theta \leftarrow \theta + \alpha\nabla_\theta \sum_i \mathbb{E}_{a\sim\pi_\theta(a|s_i)}[\hat{Q}(s_i,a)]$
until done

16.4 Model-based offline RL

Model-based RL fits $\hat{p}(s_{t+1} \mid s_t, a_t)$ and trains $\pi(a_t \mid s_t)$ on model rollouts, collecting more real data to correct the model. What goes wrong when we cannot collect more data? The model answers "what if" questions; when the policy steers rollouts into out-of-distribution states, the model's predictions there are invalid, and the policy exploits them (just like the $\arg\max_x f_\theta(x)$ problem).

MOPO. The solution is to "punish" the policy for exploiting:

$$\tilde{r}(s,a) = r(s,a) - \lambda u(s,a),$$

with an uncertainty penalty $u(s,a)$, and then use any existing model-based RL algorithm.

Theorem 16.2 [MOPO]**.** *Assume that the value function can be represented and that the model error is bounded above by $u(s,a)$. Let $\eta_M(\pi)$ be the true return of π in the true MDP M and $\epsilon_u(\pi) = \mathbb{E}_{(s,a)\sim\rho^\pi_{\hat{T}}}[u(s,a)]$ the expected penalty along the model's rollouts of π. Then the policy $\hat{\pi}$ learned by MOPO satisfies*

$$\eta_M(\hat{\pi}) \geq \sup_\pi \{\eta_M(\pi) - 2\lambda\epsilon_u(\pi)\}.$$

In particular, for all $\delta \geq \delta_{\min}$, $\eta_M(\hat{\pi}) \geq \eta_M(\pi^\delta) - 2\lambda\delta$, where $\pi^\delta = \arg\max_{\pi:\epsilon_u(\pi)\leq\delta} \eta_M(\pi)$.

Some implications: $\eta_M(\hat{\pi}) \geq \eta_M(\pi^B) - 2\lambda\epsilon_u(\pi^B)$, so MOPO improves over the behavior policy (whose model error is small); and $\eta_M(\hat{\pi}) \geq \eta_M(\pi^\star) - 2\lambda\epsilon_u(\pi^\star)$, which quantifies the "optimality gap" in terms of model error.

COMBO. Just as CQL minimizes the Q-value of policy actions, minimize the Q-value of state-action tuples from the model:

$$\hat{Q}^{k+1} \leftarrow \arg\min_Q \beta\left(\mathbb{E}_{s,a\sim\rho(s,a)}[Q(s,a)] - \mathbb{E}_{s,a\sim\mathcal{D}}[Q(s,a)]\right) + \frac{1}{2}\mathbb{E}_{s,a,s'\sim d_f}\left[\left(Q(s,a) - \hat{\mathcal{B}}^\pi\hat{Q}^k(s,a)\right)^2\right],$$

where $\rho(s, a)$ is the distribution of model state-action tuples and d_f mixes real and model data. Intuition: if the model produces something that looks clearly different from real data, it is easy for the Q-function to make it look bad.

Trajectory Transformer. Train a joint state-action model $p_\beta(\tau) = p_\beta(s_1, a_1, \ldots, s_T, a_T)$ with a big expressive model, a Transformer over the sequence of discretized state and action dimensions $s_{1,1}, \ldots, s_{1,d_s}, a_{1,1}, \ldots, a_{1,d_a}, s_{2,1}, \ldots$; it makes accurate predictions for hundreds of steps. For control, use beam search with $\sum_t r(s_t, a_t)$ instead of probability: given the current sequence, sample next tokens from the model, store the top K with highest cumulative reward, and move on to the next token. This works because generating high-probability trajectories avoids out-of-distribution states and actions, and a really big model works well offline (lots of compute, and it captures complex behavior policies).

16.5 Which algorithm, and why offline RL

Setting	Method	Tradeoffs
only train offline	CQL	just one hyperparameter; well understood and widely tested
	IQL	more flexible (offline + online); more hyperparameters
train offline, then finetune online	AWAC	widely used and well tested
	IQL	seems to perform much better
a good way to train models in the domain	COMBO	similar properties to CQL, but benefits from models; a good model is not always easy to train
	Trajectory Transformer	very powerful and effective models; extremely expensive to train and evaluate

The standard real-world RL process is: instrument the task so RL can run (safety mechanisms, autonomous collection, rewards, resets), wait a long time for online RL, change the algorithm in some small way and wait again, and finally throw it all away and start over for the next task. The offline RL process is: collect an initial dataset (human-provided, a scripted controller, a baseline policy, or all of the above), train a policy offline, change the algorithm in some small way and retrain, collect more data and add it to a growing dataset, and keep the dataset to use again for the next project. For example, data from months of multi-task robotic grasping was reused, without any reward function, to train goal-conditioned Q-functions with a conservative offline method (goals given as images), which then work well as unsupervised pretraining for tasks with a reward and limited data; and 40 hours of robot navigation data, collected once, was later reused to build a goal-conditioned navigation system and then an exploration system.

Open problems. There is a gap between current offline RL algorithms and "the dream": collect a dataset using any policy or mixture of policies, run offline RL on it to learn a policy, and deploy

the policy in the real world. Closing it needs an offline RL workflow (supervised learning has the train/test split; offline RL has no equivalent yet), statistical guarantees (the biggest challenge being distributional shift and counterfactuals), and scalable methods for large-scale applications such as dialogue systems and data-driven navigation and driving.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 16 (Fall 2023).

Lecture 17: Reinforcement Learning Theory Basics

17.1 What questions do we ask?

There are lots of questions in RL theory. A few common ones: if I use this algorithm with N samples and k iterations, how good is the result? For Q-learning, for example, we might show $\|\hat{Q}_k - Q^\star\| \le \epsilon$ with probability at least $1-\delta$ if $N \ge f(\epsilon, \delta)$, or $\|Q^{\pi_k} - Q^\star\| \le \epsilon$, which is not the same thing: Q^{π_k} is the *true* Q-function of the policy at iteration k. This lecture focuses on questions of this type. Another kind asks how high the regret of an exploration algorithm is, for example $\mathrm{Reg}(T) \le O\big(\sqrt{T \cdot N \cdot \log \frac{NT}{\delta}}\big) + \delta T$.

Assumptions. Effective analysis is very hard in RL without strong assumptions; the trick is to make assumptions that admit interesting conclusions without divorcing us (too much) from reality. Exploration greatly complicates performance: how likely are we to find (potentially sparse) rewards? Guarantees typically address worst-case performance, and worst-case exploration is extremely hard; the goal there is to show that an exploration method (for example counts) is $\mathrm{Poly}(|S|, |A|, 1/(1-\gamma))$. For learning, we "abstract away" exploration and ask how many samples we need to learn a model or value function that results in good performance. The generative model assumption is that we can sample from $P(s' \mid s, a)$ for *any* (s, a); oracle exploration means that for every (s, a) we sample $s' \sim P(s' \mid s, a)$ N times.

What is the point? Proving that RL algorithms work perfectly every time is usually not possible with current deep RL methods, which are often not even guaranteed to converge. The goal is to understand how errors are affected by problem parameters: do larger discounts work better than smaller ones? To halve the error, do we need 2× the samples, 4×, or something else? We use precise theory to get imprecise qualitative conclusions under strong assumptions, and try to make the assumptions reasonable enough that the conclusions are likely (but not guaranteed) to apply to real problems. Claims of "provable guarantees" for RL algorithms should not be taken too seriously: the assumptions are always unrealistic, and theory is at best a rough guide.

17.2 Sample complexity of a simple model-based algorithm

With oracle exploration, consider the simple "model-based" algorithm:

1. estimate $\hat{P}(s' \mid s, a) = \#(s, a, s')/N$;
2. given π, use $\hat{P}$ to estimate $\hat{Q}^\pi$.

We ask how close $\hat{Q}^\pi$ is to Q^π, $\|Q^\pi(s,a) - \hat{Q}^\pi(s,a)\|_\infty = \max_{s,a} |Q^\pi(s,a) - \hat{Q}^\pi(s,a)| \le \epsilon$ with probability at least $1-\delta$ if $N \ge f(\epsilon, \delta)$ (the ∞-norm is good to use if we want worst-case

performance); how close the optimal Q-function $\hat{Q}^\star$ learned under $\hat{P}$ is to $Q^\star$; and how good the resulting argmax policy $\hat{\pi}$ is, $\|Q^\star - Q^{\hat{\pi}}\|_\infty \le \epsilon$.

Concentration inequalities answer how close a learned function is to the true one in terms of the number of samples.

Lemma 17.1 [Hoeffding's inequality]. *Let $X_1, \dots, X_n$ be i.i.d. random variables with mean μ and $X_i \in [b_-, b_+]$ with probability 1, and let $\bar{X}_n = n^{-1}\sum_{i=1}^n X_i$. Then*

$$P(\bar{X}_n \ge \mu + \epsilon) \le e^{-2n\epsilon^2/(b_+ - b_-)^2}, \qquad P(\bar{X}_n \le \mu - \epsilon) \le e^{-2n\epsilon^2/(b_+ - b_-)^2}.$$

So estimating μ with n samples is off by more than ϵ with probability at most $\delta = 2e^{-2n\epsilon^2/(b_+ - b_-)^2}$. Solving for ϵ: with probability at least $1 - \delta$,

$$|\bar{X}_n - \mu| \le \frac{b_+ - b_-}{\sqrt{2n}}\sqrt{\log\frac{2}{\delta}}, \qquad \text{equivalently} \qquad n \ge \frac{(b_+ - b_-)^2}{2\epsilon^2}\log\frac{2}{\delta}$$

suffices for error ϵ. The error scales as $1/\sqrt{n}$.

Proposition 17.2 [Concentration for discrete distributions]. *Let z take values in $\{1, \dots, d\}$ with distribution $\vec{q} = [\Pr(z = j)]_{j=1}^d$, and let $[\hat{q}]_j = \sum_{i=1}^N \mathbf{1}[z_i = j]/N$ from N i.i.d. samples. For all $\epsilon > 0$,*

$$\Pr\big(\|\hat{q} - \vec{q}\|_2 \ge 1/\sqrt{N} + \epsilon\big) \le e^{-N\epsilon^2}, \qquad \textit{which implies} \qquad \Pr\big(\|\hat{q} - \vec{q}\|_1 \ge \sqrt{d}(1/\sqrt{N} + \epsilon)\big) \le e^{-N\epsilon^2}.$$

Applied to $\hat{P}(s' \mid s, a)$ with $d = |S|$ and $\delta = e^{-N\epsilon^2}$, that is $\epsilon = \sqrt{\log(1/\delta)/N}$: with probability $1 - \delta$,

$$\|\hat{P}(s' \mid s, a) - P(s' \mid s, a)\|_1 \le \sqrt{|S|}\Big(\frac{1}{\sqrt{N}} + \epsilon\Big) = \sqrt{\frac{|S|}{N}} + \sqrt{\frac{|S|\log 1/\delta}{N}} \le c\sqrt{\frac{|S|\log 1/\delta}{N}}.$$

Relating P to Q^π. In vector form, with $Q^\pi, r \in \mathbb{R}^{|S||A|}$, $P \in \mathbb{R}^{|S||A|\times|S|}$, $V^\pi \in \mathbb{R}^{|S|}$, and the policy matrix $\Pi \in \mathbb{R}^{|S|\times|S||A|}$ so that $V^\pi = \Pi Q^\pi$,

$$Q^\pi(s,a) = r(s,a) + \gamma\sum_{s'} P(s' \mid s, a)V^\pi(s') \quad \Longrightarrow \quad Q^\pi = r + \gamma P V^\pi = r + \gamma P^\pi Q^\pi, \qquad P^\pi = P\Pi,$$

so $(I - \gamma P^\pi)Q^\pi = r$ and $Q^\pi = (I - \gamma P^\pi)^{-1}r$. Likewise $\hat{Q}^\pi = (I - \gamma\hat{P}^\pi)^{-1}r$.

Lemma 17.3 [Simulation lemma]. *$Q^\pi - \hat{Q}^\pi = \gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})V^\pi$: an evaluation operator applied to the difference in probabilities times the true value.*

Proof. Using $r = (I - \gamma P^\pi)Q^\pi$,

$$\begin{aligned} Q^\pi - \hat{Q}^\pi &= (I - \gamma\hat{P}^\pi)^{-1}(I - \gamma\hat{P}^\pi)Q^\pi - (I - \gamma\hat{P}^\pi)^{-1}(I - \gamma P^\pi)Q^\pi \\ &= \gamma(I - \gamma\hat{P}^\pi)^{-1}(P^\pi - \hat{P}^\pi)Q^\pi = \gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})\Pi Q^\pi = \gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})V^\pi. \square \end{aligned}$$

Lemma 17.4. *For any P^π and any $v \in \mathbb{R}^{|S||A|}$, $\|(I - \gamma P^\pi)^{-1}v\|_\infty \le \|v\|_\infty/(1-\gamma)$: the "Q-function" corresponding to "reward" v is at most $1/(1-\gamma)$ times larger, just as $\sum_{t=0}^\infty \gamma^t c = c/(1-\gamma)$.*

Proof. Let $w = (I - \gamma P^\pi)^{-1}v$. By the triangle inequality $\|a - b\| \ge \|a\| - \|b\|$ and $\|P^\pi\|_\infty \le 1$,

$$\|v\|_\infty = \|(I - \gamma P^\pi)w\|_\infty \ge \|w\|_\infty - \gamma\|P^\pi w\|_\infty \ge \|w\|_\infty - \gamma\|w\|_\infty = (1-\gamma)\|w\|_\infty. \qquad \square$$

Theorem 17.5 [Model-based policy evaluation]. *With $R_{\max} = 1$, oracle exploration with N samples per (s, a), and probability at least $1 - \delta$,*

$$\|Q^\pi - \hat{Q}^\pi\|_\infty \le \frac{\gamma}{(1-\gamma)^2} c_2 \sqrt{\frac{|S| \log 1/\delta}{N}}.$$

Proof. By Lemmas 17.3 and 17.4,

$$\begin{aligned} \|Q^\pi - \hat{Q}^\pi\|_\infty = \|\gamma(I - \gamma\hat{P}^\pi)^{-1}(P - \hat{P})V^\pi\|_\infty &\le \frac{\gamma}{1-\gamma}\|(P - \hat{P})V^\pi\|_\infty \\ &\le \frac{\gamma}{1-\gamma}\left(\max_{s,a}\|P(\cdot \mid s,a) - \hat{P}(\cdot \mid s,a)\|_1\right)\|V^\pi\|_\infty. \end{aligned}$$

Since $\sum_{t=0}^\infty \gamma^t r_t \le \sum_{t=0}^\infty \gamma^t R_{\max} = R_{\max}/(1-\gamma)$, $\|V^\pi\|_\infty \le 1/(1-\gamma)$. The concentration bound above, with a union bound over the (s, a) pairs absorbed into the constant, finishes the proof. $\square$

What does this mean? More samples mean lower error, at rate $1/\sqrt{N}$; and the error grows *quadratically* in the horizon $1/(1-\gamma)$, because each backup "accumulates" error. Some simple implications, with ϵ the bound of Theorem 17.5: using $|\sup_x f(x) - \sup_x g(x)| \le \sup_x |f(x) - g(x)|$,

$$\|Q^\star - \hat{Q}^\star\|_\infty = \left\|\sup_\pi Q^\pi - \sup_\pi \hat{Q}^\pi\right\|_\infty \le \sup_\pi \|Q^\pi - \hat{Q}^\pi\|_\infty \le \epsilon,$$

and for the policy $\hat{\pi}^\star$ that is optimal under $\hat{P}$ (so $\hat{Q}^{\hat{\pi}^\star} = \hat{Q}^\star$),

$$\|Q^\star - Q^{\hat{\pi}^\star}\|_\infty = \|Q^\star - \hat{Q}^{\hat{\pi}^\star} + \hat{Q}^{\hat{\pi}^\star} - Q^{\hat{\pi}^\star}\|_\infty \le \|Q^\star - \hat{Q}^{\hat{\pi}^\star}\|_\infty + \|Q^{\hat{\pi}^\star} - \hat{Q}^{\hat{\pi}^\star}\|_\infty \le 2\epsilon,$$

where the second term compares the true and estimated Q-functions of the same policy.

17.3 Analyzing fitted Q-iteration

With the Bellman operator $TQ = r + \gamma P \max_a Q$, exact Q-iteration is $Q_{k+1} \leftarrow TQ_k$. An abstract model of approximate fitted Q-iteration is

$$\hat{Q}_{k+1} \leftarrow \arg\min_{\hat{Q}} \|\hat{Q} - \hat{T}\hat{Q}_k\|,$$

with the approximate Bellman operator $\hat{T}Q = \hat{r} + \gamma \hat{P} \max_a Q$, where $\hat{r}(s,a) = \frac{1}{N(s,a)} \sum_i \delta((s_i, a_i) = (s,a)) r_i$ and $\hat{P}(s' \mid s,a) = N(s,a,s')/N(s,a)$. These are not models; they are the effect of averaging together transitions in the data. Which norm? There is no convergence with $\|\cdot\|_2$ (Lecture 7), so assume $\|\cdot\|_\infty$. As $k \to \infty$, what is $\lim_k \|\hat{Q}_k - Q^\star\|_\infty$? Errors come from two sources: SAMPLING ERROR, $T \neq \hat{T}$, and APPROXIMATION ERROR, $\hat{Q}_{k+1} \neq \hat{T}\hat{Q}_k$.

Sampling error. Compare $\hat{T}\hat{Q}_k$ with $T\hat{Q}_k$:

$$\begin{aligned}|\hat{T}Q(s,a) - TQ(s,a)| &= \left|\hat{r}(s,a) - r(s,a) + \gamma\left(\mathbb{E}_{\hat{P}(s'|s,a)}[\max_{a'} Q(s',a')] - \mathbb{E}_{P(s'|s,a)}[\max_{a'} Q(s',a')]\right)\right| \\ &\leq |\hat{r}(s,a) - r(s,a)| + \gamma\left|\mathbb{E}_{\hat{P}(s'|s,a)}[\max_{a'} Q(s',a')] - \mathbb{E}_{P(s'|s,a)}[\max_{a'} Q(s',a')]\right|.\end{aligned}$$

The first term is the estimation error of a continuous random variable, so Hoeffding's inequality applies directly: $|\hat{r}(s,a) - r(s,a)| \leq 2R_{\max}\sqrt{\log(1/\delta)/2N}$. The second is

$$\begin{aligned}\left|\sum_{s'}\left(\hat{P}(s' \mid s,a) - P(s' \mid s,a)\right)\max_{a'} Q(s',a')\right| &\leq \sum_{s'} |\hat{P}(s' \mid s,a) - P(s' \mid s,a)| \max_{s',a'} |Q(s',a')| \\ &= \|\hat{P}(\cdot \mid s,a) - P(\cdot \mid s,a)\|_1 \|Q\|_\infty \leq c\|Q\|_\infty \sqrt{\frac{|S| \log 1/\delta}{N}}\end{aligned}$$

by Proposition 17.2. Using a union bound over all (s,a),

$$\|\hat{T}Q - TQ\|_\infty \leq 2R_{\max} c_1 \sqrt{\frac{\log(|S||A|/\delta)}{2N}} + c_2\|Q\|_\infty \sqrt{\frac{|S| \log(|S||A|/\delta)}{N}}.$$

Approximation error. Assume $\|\hat{Q}_{k+1} - T\hat{Q}_k\|_\infty \leq \epsilon_k$. This is a strong assumption; we analyze the exact backup operator T for now and account for the approximate one below. Since $Q^\star$ is a fixed point of T and T is a γ-contraction,

$$\begin{aligned}\|\hat{Q}_k - Q^\star\|_\infty &= \|(\hat{Q}_k - T\hat{Q}_{k-1}) + (T\hat{Q}_{k-1} - TQ^\star)\|_\infty \leq \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty + \|T\hat{Q}_{k-1} - TQ^\star\|_\infty \\ &\leq \epsilon_{k-1} + \gamma\|\hat{Q}_{k-1} - Q^\star\|_\infty.\end{aligned}$$

Unrolling,

$$\|\hat{Q}_k - Q^\star\|_\infty \leq \epsilon_{k-1} + \gamma\epsilon_{k-2} + \gamma^2\epsilon_{k-3} + \cdots \leq \sum_{i=0}^{k-1} \gamma^i \epsilon_{k-i-1} + \gamma^k \|\hat{Q}_0 - Q^\star\|_\infty,$$

so

$$\lim_{k\to\infty} \|\hat{Q}_k - Q^\star\|_\infty \le \sum_{i=0}^{\infty} \gamma^i \max_k \epsilon_k = \frac{1}{1-\gamma}\|\epsilon\|_\infty :$$

the approximation error scales with the "horizon".

Putting it together. The per-iteration error ϵ_k is how much $\hat{Q}_{k+1}$ differs from $T\hat{Q}_k$, due to both sampling and approximation:

$$\|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty \le \underbrace{\|\hat{Q}_k - \hat{T}\hat{Q}_{k-1}\|_\infty}_{\text{approximation}} + \underbrace{\|\hat{T}\hat{Q}_{k-1} - T\hat{Q}_{k-1}\|_\infty}_{\text{sampling}},$$

and $\lim_k \|\hat{Q}_k - Q^\star\|_\infty \le \frac{1}{1-\gamma} \max_k \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty$. In the sampling term, $\|Q\|_\infty$ is itself $O(R_{\max}/(1-\gamma))$. So error compounds with the horizon, over iterations and due to sampling. These results needed strong (∞-norm) assumptions; more advanced results can be derived with p-norms under some distribution, $\|\hat{Q}_k - Q^\star\|_{p,\mu} = \left(\mathbb{E}_{(s,a)\sim\mu(s,a)}[|\hat{Q}_k(s,a) - Q^\star(s,a)|^p]\right)^{1/p}$.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 17 (Fall 2023).

Lecture 18: Variational Inference and Generative Models

18.1 Latent variable models

A probabilistic model is a distribution $p(x)$, or a conditional distribution $p(y \mid x)$ such as a policy $\pi_\theta(a \mid s)$. A latent variable model introduces an unobserved variable z:

$$p(x) = \sum_z p(x \mid z)p(z), \qquad p(y \mid x) = \sum_z p(y \mid x, z)p(z),$$

for example a mixture model, where z indexes the mixture element (as in the mixture-of-Gaussians policies of Lecture 2). In general,

$$p(x) = \int p(x \mid z)p(z)\, dz,$$

where both factors are "easy" distributions, for example a Gaussian prior $p(z)$ and a conditional Gaussian $p(x \mid z) = \mathcal{N}(\mu_{\text{nn}}(z), \sigma_{\text{nn}}(z))$ given by a neural network, while $p(x)$ itself can be very complex and multimodal.

In RL, conditional latent variable models give multimodal policies ($z \sim \mathcal{N}(0, I)$ fed into $p(y \mid x, z)$), and latent variable models are used in model-based RL with observations o_t emitted from latent states x_t (Lecture 11), where the latent space has structure: the model of $p(x_t)$ actually models $p(x_{t+1} \mid x_t)$ and $p(x_1)$. Later we will use RL and control together with variational inference to model human behavior, and generative models with variational inference for exploration.

Training. Given data $\mathcal{D} = \{x_1, \ldots, x_N\}$ and a model $p_\theta(x)$, the maximum likelihood fit

$$\theta \leftarrow \arg\max_\theta \frac{1}{N} \sum_i \log p_\theta(x_i) = \arg\max_\theta \frac{1}{N} \sum_i \log\Big(\int p_\theta(x_i \mid z)p(z)\, dz \Big)$$

is completely intractable. An alternative is the *expected* log-likelihood

$$\theta \leftarrow \arg\max_\theta \frac{1}{N} \sum_i \mathbb{E}_{z \sim p(z \mid x_i)} \big[\log p_\theta(x_i, z) \big] :$$

intuitively, "guess" the most likely z given x_i and pretend it is the right one; since there are many possible values of z, use the distribution $p(z \mid x_i)$. But how do we calculate $p(z \mid x_i)$?

18.2 Variational inference

Approximate $p(z \mid x_i)$ with a simpler distribution, for example $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$. This yields a bound on $\log p(x_i)$:

$$\begin{aligned}\log p(x_i) &= \log \int_z p(x_i \mid z)p(z) = \log \int_z p(x_i \mid z)p(z)\frac{q_i(z)}{q_i(z)} = \log \mathbb{E}_{z\sim q_i(z)}\Big[\frac{p(x_i \mid z)p(z)}{q_i(z)}\Big] \\ &\geq \mathbb{E}_{z\sim q_i(z)}\Big[\log \frac{p(x_i \mid z)p(z)}{q_i(z)}\Big] = \mathbb{E}_{z\sim q_i(z)}\big[\log p(x_i \mid z) + \log p(z)\big] - \mathbb{E}_{z\sim q_i(z)}[\log q_i(z)],\end{aligned}$$

by Jensen's inequality $\log \mathbb{E}[y] \geq \mathbb{E}[\log y]$. Maximizing this maximizes (a lower bound on) $\log p(x_i)$.

Definition 18.1 [Entropy and KL divergence]. The entropy of p is $\mathcal{H}(p) = -\mathbb{E}_{x\sim p(x)}[\log p(x)] = -\int_x p(x)\log p(x)\,dx$: how random the variable is, or how large its log probability is in expectation under itself (low for a narrow distribution, high for a broad one). The KL divergence is

$$D_{\mathrm{KL}}(q\|p) = \mathbb{E}_{x\sim q(x)}\Big[\log \frac{q(x)}{p(x)}\Big] = \mathbb{E}_{x\sim q(x)}[\log q(x)] - \mathbb{E}_{x\sim q(x)}[\log p(x)] = -\mathbb{E}_{x\sim q(x)}[\log p(x)] - \mathcal{H}(q) :$$

how different two distributions are, or how small the expected log probability of one distribution under another is, minus entropy.

The bound is therefore the evidence lower bound

$$\log p(x_i) \geq \mathcal{L}_i(p, q_i) = \mathbb{E}_{z\sim q_i(z)}\big[\log p(x_i \mid z) + \log p(z)\big] + \mathcal{H}(q_i).$$

Maximizing the first term puts q_i at the peak of $p(x_i, z)$; the entropy term makes it as wide as possible, so q_i covers the mass of $p(x_i, z)$. (This is also why the KL divergence includes the entropy.)

What makes a good $q_i(z)$? Intuitively it should approximate $p(z \mid x_i)$, compared in terms of KL divergence:

$$\begin{aligned}D_{\mathrm{KL}}(q_i(z)\|p(z \mid x_i)) &= \mathbb{E}_{z\sim q_i(z)}\Big[\log \frac{q_i(z)}{p(z \mid x_i)}\Big] = \mathbb{E}_{z\sim q_i(z)}\Big[\log \frac{q_i(z)p(x_i)}{p(x_i, z)}\Big] \\ &= -\mathbb{E}_{z\sim q_i(z)}\big[\log p(x_i \mid z) + \log p(z)\big] - \mathcal{H}(q_i) + \log p(x_i) \\ &= -\mathcal{L}_i(p, q_i) + \log p(x_i).\end{aligned}$$

So $\log p(x_i) = D_{\mathrm{KL}}(q_i(z)\|p(z \mid x_i)) + \mathcal{L}_i(p, q_i) \geq \mathcal{L}_i(p, q_i)$. Since $\log p(x_i)$ is independent of q_i, maximizing $\mathcal{L}_i(p, q_i)$ with respect to q_i minimizes the KL divergence, and the bound is tight when $q_i(z) = p(z \mid x_i)$.

How do we use this? Replace $\theta \leftarrow \arg\max_\theta \frac{1}{N}\sum_i \log p_\theta(x_i)$ by $\theta \leftarrow \arg\max_\theta \frac{1}{N}\sum_i \mathcal{L}_i(p, q_i)$:

Algorithm 18.1: Variational inference for latent variable models

for each x_i (or mini-batch) **do**
  calculate $\nabla_\theta \mathcal{L}_i(p, q_i)$: sample $z \sim q_i(z)$, $\nabla_\theta \mathcal{L}_i(p, q_i) \approx \nabla_\theta \log p_\theta(x_i \mid z)$
  $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}_i(p, q_i)$
  update q_i to maximize $\mathcal{L}_i(p, q_i)$ (for $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$, gradient ascent on μ_i, σ_i)

What is the problem? The number of parameters is $|\theta| + (|\mu_i| + |\sigma_i|) \times N$, growing with the dataset.

18.3 Amortized variational inference

Since $q_i(z)$ should approximate $p(z \mid x_i)$, learn a *network* $q_i(z) = q(z \mid x_i) \approx p(z \mid x_i)$: an INFERENCE NETWORK $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, alongside the generative network $p_\theta(x \mid z)$. Then

$$\log p(x_i) \geq \mathcal{L}(p_\theta(x_i \mid z), q_\phi(z \mid x_i)) = \mathbb{E}_{z \sim q_\phi(z \mid x_i)}\big[\log p_\theta(x_i \mid z) + \log p(z)\big] + \mathcal{H}(q_\phi(z \mid x_i)),$$

and for each x_i (or mini-batch): sample $z \sim q_\phi(z \mid x_i)$, set $\nabla_\theta \mathcal{L} \approx \nabla_\theta \log p_\theta(x_i \mid z)$, and update $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$ and $\phi \leftarrow \phi + \alpha \nabla_\phi \mathcal{L}$.

How do we calculate $\nabla_\phi \mathcal{L}$? The entropy of a Gaussian has a closed form. The first part has the form $J(\phi) = \mathbb{E}_{z \sim q_\phi(z \mid x_i)}[r(x_i, z)]$ with $r(x_i, z) = \log p_\theta(x_i \mid z) + \log p(z)$, so we can just use the policy gradient:

$$\nabla J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi \log q_\phi(z_j \mid x_i)\, r(x_i, z_j).$$

What is wrong with this gradient? It has high variance, like any policy gradient.

The reparameterization trick. Is there a better way? Write $z = \mu_\phi(x) + \epsilon \sigma_\phi(x)$ with $\epsilon \sim \mathcal{N}(0, 1)$ independent of ϕ. Then

$$J(\phi) = \mathbb{E}_{z \sim q_\phi(z \mid x_i)}[r(x_i, z)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0,1)}\big[r(x_i, \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i))\big],$$

and to estimate $\nabla_\phi J(\phi)$, sample $\epsilon_1, \ldots, \epsilon_M$ from $\mathcal{N}(0, 1)$ (a single sample works well) and use

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi r\big(x_i, \mu_\phi(x_i) + \epsilon_j \sigma_\phi(x_i)\big),$$

which most autodiff software computes automatically.

Another way to look at it. Splitting the bound,

$$\begin{aligned}\mathcal{L}_i &= \mathbb{E}_{z\sim q_\phi(z|x_i)}[\log p_\theta(x_i \mid z)] + \underbrace{\mathbb{E}_{z\sim q_\phi(z|x_i)}[\log p(z)] + \mathcal{H}(q_\phi(z \mid x_i))}_{-D_{\mathrm{KL}}(q_\phi(z|x_i)\|p(z))} \\ &= \mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}\big[\log p_\theta(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i))\big] - D_{\mathrm{KL}}(q_\phi(z \mid x_i)\|p(z)) \\ &\approx \log p_\theta(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i)) - D_{\mathrm{KL}}(q_\phi(z \mid x_i)\|p(z)),\end{aligned}$$

where the KL term often has a convenient analytical form (for example between Gaussians). The whole objective is a differentiable computation graph (Figure 18).

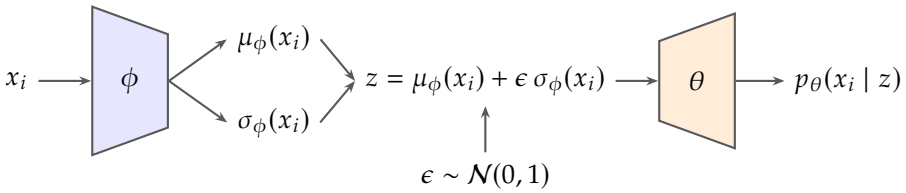


Figure 18: The reparameterized encoder and decoder: the noise ϵ enters as an input, so gradients flow to ϕ through z.

Reparameterization trick versus policy gradient. The policy gradient $\frac{1}{M}\sum_j \nabla_\phi \log q_\phi(z_j \mid x_i) r(x_i, z_j)$ can handle both discrete and continuous latent variables but has high variance, requiring multiple samples and small learning rates. The reparameterization trick $\frac{1}{M}\sum_j \nabla_\phi r(x_i, \mu_\phi(x_i) + \epsilon_j\sigma_\phi(x_i))$ handles only continuous latent variables, but is very simple to implement and has low variance.

18.4 Variational inference in deep RL

Definition 18.2 [Variational autoencoder]. The variational autoencoder (VAE) has an encoder $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, a decoder $p_\theta(x \mid z) = \mathcal{N}(\mu_\theta(z), \sigma_\theta(z))$, and prior $p(z) = \mathcal{N}(0, I)$, trained by

$$\max_{\theta,\phi} \frac{1}{N}\sum_i \log p_\theta\big(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i)\big) - D_{\mathrm{KL}}(q_\phi(z \mid x_i)\|p(z)).$$

To sample from $p(x) = \int p(x \mid z)p(z)\,dz$, draw $z \sim p(z)$ and then $x \sim p(x \mid z)$. Why does this work? In $\mathcal{L}_i = \mathbb{E}_{z\sim q_\phi(z|x_i)}[\log p_\theta(x_i \mid z)] - D_{\mathrm{KL}}(q_\phi(z \mid x_i)\|p(z))$, the KL term keeps the encoder's posteriors close to the prior, so samples from the prior land where the decoder has been trained.

Representation learning. Let z be a representation of the state s: train a VAE on the states in a replay buffer $\mathcal{R}$, then run RL using z as the state instead of s. This is a good idea because VAE latents tend to capture meaningful factors of variation in a compact form (learned representations of chairs and faces can separate factors such as rotation and lighting). A sample algorithm, which also provides a great way to use prior data:

1. collect transition (s, a, s', r) and add it to $\mathcal{R}$;
2. update $p_\theta(s \mid z)$ and $q_\phi(z \mid s)$ with a batch from $\mathcal{R}$;
3. update $Q(z, a)$ with a batch from $\mathcal{R}$.

Conditional models. Just like before, only now generating y_i and conditioning everything on x_i:

$$\mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z \mid x_i, y_i)} \big[\log p_\theta(y_i \mid x_i, z) + \log p(z \mid x_i)\big] + \mathcal{H}(q_\phi(z \mid x_i, y_i)),$$

where the prior $p(z)$ can optionally depend on x. At test time, sample $z \sim p(z \mid x_i)$ and $y \sim p(y \mid x_i, z)$. This gives multimodal imitation learning with policies $p(a \mid s, z)$, $z \sim \mathcal{N}(0, I)$, as in learning latent plans from play data and in fine-grained bimanual manipulation with a Transformer encoder producing a style variable z.

State space models. With observations $o_1, \ldots, o_T$ (playing the role of x) and latent states $z_1, \ldots, z_T$ (playing the role of z), and actions treated as observed inputs:

- the prior is $p(z) = p(z_1) \prod_t p(z_{t+1} \mid z_t, a_t)$, with $p(z_1) = \mathcal{N}(0, I)$ and learned dynamics;
- the decoder is $p_\theta(o \mid z) = \prod_t p(o_t \mid z_t)$, applied to each latent state separately;
- the encoder is $q_\phi(z \mid o) = \prod_t q_\phi(z_t \mid o_{1:t})$, a sequence model (LSTM or Transformer) over the observations, since a single image need not reveal the state.

Such models allow representation learning for model-based RL: learn a state space model and plan in the latent space (Embed to Control, SOLAR, learning latent dynamics for planning from pixels), or learn a state space model and run RL in the latent space (stochastic latent actor-critic, and learning behaviors by latent imagination).

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 18 (Fall 2023).

Lecture 19: Connection between Inference and Control

19.1 Optimal control as a model of human behavior

Optimal control is used to explain human and animal motion (running gaits, walking paths, arm movements, route choices): find the reward $r(s_t, a_t)$ such that

$$a_1, \dots, a_T = \arg\max_{a_1,\dots,a_T} \sum_{t=1}^{T} r(s_t, a_t), \quad s_{t+1} = f(s_t, a_t),$$

or $\pi = \arg\max_\pi \mathbb{E}_{s_{t+1}\sim p(s_{t+1}|s_t,a_t),\, a_t\sim\pi(a_t|s_t)}[r(s_t, a_t)]$, explains the data. But what if the data is not optimal? A monkey reaching for a target takes many different paths: some mistakes matter more than others, behavior is *stochastic*, but good behavior is still the most likely.

Definition 19.1 [Graphical model of decision making]. Introduce binary optimality variables O_t with

$$p(O_t \mid s_t, a_t) = \exp\big(r(s_t, a_t)\big)$$

(rewards are assumed nonpositive so this is a probability). With $\tau = (s_{1:T}, a_{1:T})$,

$$p(\tau \mid O_{1:T}) = \frac{p(\tau, O_{1:T})}{p(O_{1:T})} \propto p(\tau) \prod_t \exp\big(r(s_t, a_t)\big) = p(\tau) \exp\Big(\sum_t r(s_t, a_t)\Big).$$

This makes no assumption of optimal behavior: the most likely trajectories are optimal, but suboptimal ones have nonzero probability, exponentially less likely the worse they are (Figure 19). It is interesting because it can model suboptimal behavior (important for inverse RL), lets us apply inference algorithms to solve control and planning problems, and explains why stochastic behavior might be preferred (useful for exploration and transfer learning).

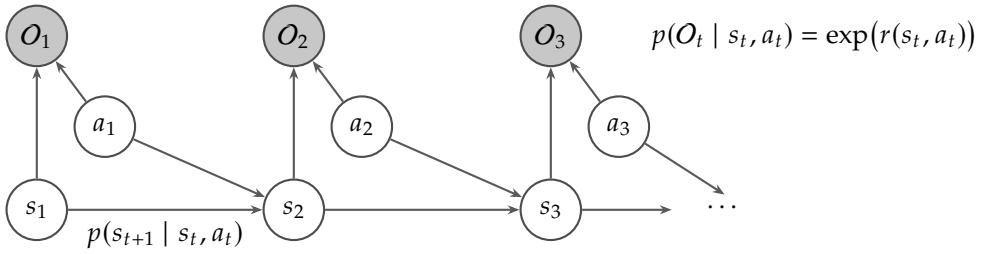


Figure 19: The graphical model of decision making with optimality variables O_t (shaded, observed).

Inference in this model is planning. With $p(O_t \mid s_t, a_t) \propto \exp(r(s_t, a_t))$ and dynamics $p(s_{t+1} \mid s_t, a_t)$, inference has three steps:

1. compute backward messages $\beta_t(s_t, a_t) = p(\mathcal{O}_{t:T} \mid s_t, a_t)$;
2. compute the policy $p(a_t \mid s_t, \mathcal{O}_{1:T})$;
3. compute forward messages $\alpha_t(s_t) = p(s_t \mid \mathcal{O}_{1:t-1})$.

19.2 Control as inference

Backward messages. Since $\mathcal{O}_{t+1:T}$ is independent of (s_t, a_t) given s_{t+1},

$$\begin{aligned}\beta_t(s_t, a_t) = p(\mathcal{O}_{t:T} \mid s_t, a_t) &= \int p(\mathcal{O}_{t:T}, s_{t+1} \mid s_t, a_t)\, ds_{t+1} \\ &= \int p(\mathcal{O}_{t+1:T} \mid s_{t+1})\, p(s_{t+1} \mid s_t, a_t)\, p(\mathcal{O}_t \mid s_t, a_t)\, ds_{t+1},\end{aligned}$$

and $p(\mathcal{O}_{t+1:T} \mid s_{t+1}) = \int p(\mathcal{O}_{t+1:T} \mid s_{t+1}, a_{t+1})\, p(a_{t+1} \mid s_{t+1})\, da_{t+1}$, where $p(a_{t+1} \mid s_{t+1})$ says which actions are likely *a priori*; assume it is uniform for now. Hence, for $t = T - 1$ down to 1,

$$\beta_t(s_t, a_t) = p(\mathcal{O}_t \mid s_t, a_t)\, \mathbb{E}_{s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)}\big[\beta_{t+1}(s_{t+1})\big], \qquad \beta_t(s_t) = \mathbb{E}_{a_t \sim p(a_t \mid s_t)}\big[\beta_t(s_t, a_t)\big].$$

$\beta_t(s_t, a_t)$ is the probability that we can be optimal at steps t through T given that we take a_t in s_t, computed recursively from $t = T$ to $t = 1$.

A closer look at the backward pass. Let $V_t(s_t) = \log \beta_t(s_t)$ and $Q_t(s_t, a_t) = \log \beta_t(s_t, a_t)$: the log of β_t is "Q-function-like". Then

$$V_t(s_t) = \log \int \exp\big(Q_t(s_t, a_t)\big)\, da_t, \qquad Q_t(s_t, a_t) = r(s_t, a_t) + \log \mathbb{E}\big[\exp(V_{t+1}(s_{t+1}))\big].$$

$V_t(s_t) \to \max_{a_t} Q_t(s_t, a_t)$ as $Q_t(s_t, a_t)$ gets bigger: a "soft" max. With deterministic transitions, $Q_t(s_t, a_t) = r(s_t, a_t) + V_{t+1}(s_{t+1})$, just like value iteration ($Q(s, a) \leftarrow r(s, a) + \gamma \mathbb{E}[V(s')]$, $V(s) \leftarrow \max_a Q(s, a)$). With stochastic transitions, $\log \mathbb{E}[\exp(V_{t+1}(s_{t+1}))]$ is an "optimistic" transition, which is not a good idea; we return to this below.

The action prior. If $p(a_t \mid s_t)$ is not uniform, then $V(s_t) = \log \int \exp\big(Q(s_t, a_t) + \log p(a_t \mid s_t)\big)\, da_t$. Letting $\tilde{Q}(s_t, a_t) = r(s_t, a_t) + \log p(a_t \mid s_t) + \log \mathbb{E}[\exp(V(s_{t+1}))]$ gives $V(s_t) = \log \int \exp(\tilde{Q}(s_t, a_t))\, da_t$: we can always fold the action prior into the reward, so a uniform action prior can be assumed without loss of generality.

Policy computation. With $\beta_t(s_t) = p(\mathcal{O}_{t:T} \mid s_t)$, and since actions at time t do not depend on $\mathcal{O}_{1:t-1}$ given s_t, the policy $p(a_t \mid s_t, \mathcal{O}_{1:T})$ is

$$\pi(a_t \mid s_t) = p(a_t \mid s_t, \mathcal{O}_{t:T}) = \frac{p(\mathcal{O}_{t:T} \mid a_t, s_t)\, p(a_t, s_t)}{p(\mathcal{O}_{t:T} \mid s_t)\, p(s_t)} = \frac{\beta_t(s_t, a_t)}{\beta_t(s_t)}\, p(a_t \mid s_t) \propto \frac{\beta_t(s_t, a_t)}{\beta_t(s_t)}$$

with a uniform action prior. In terms of the soft value functions,

$$\pi(a_t \mid s_t) = \exp\big(Q_t(s_t, a_t) - V_t(s_t)\big) = \exp\big(A_t(s_t, a_t)\big),$$

and with a temperature, $\pi(a_t \mid s_t) = \exp\big(\frac{1}{\alpha} Q_t(s_t, a_t) - \frac{1}{\alpha} V_t(s_t)\big) = \exp\big(\frac{1}{\alpha} A_t(s_t, a_t)\big)$. Better actions are more probable, ties are broken randomly, the policy is analogous to Boltzmann exploration, and it approaches the greedy policy as the temperature decreases.

Forward messages. $\alpha_1(s_1) = p(s_1)$ is usually known, and

$$\begin{aligned}\alpha_t(s_t) = p(s_t \mid O_{1:t-1}) &= \int p(s_t, s_{t-1}, a_{t-1} \mid O_{1:t-1})\, ds_{t-1}\, da_{t-1} \\ &= \int p(s_t \mid s_{t-1}, a_{t-1})\, p(a_{t-1} \mid s_{t-1}, O_{t-1})\, p(s_{t-1} \mid O_{1:t-1})\, ds_{t-1}\, da_{t-1},\end{aligned}$$

where, by Bayes' rule,

$$p(a_{t-1} \mid s_{t-1}, O_{t-1})\, p(s_{t-1} \mid O_{1:t-1}) = \frac{p(O_{t-1} \mid s_{t-1}, a_{t-1})\, p(a_{t-1} \mid s_{t-1})}{p(O_{t-1} \mid s_{t-1})} \cdot \frac{p(O_{t-1} \mid s_{t-1})\, \alpha_{t-1}(s_{t-1})}{p(O_{t-1} \mid O_{1:t-2})}.$$

The state marginals given optimality are

$$p(s_t \mid O_{1:T}) = \frac{p(s_t, O_{1:T})}{p(O_{1:T})} = \frac{p(O_{t:T} \mid s_t)\, p(s_t, O_{1:t-1})}{p(O_{1:T})} \propto \beta_t(s_t)\, \alpha_t(s_t).$$

The marginal is the intersection of the states with high probability of reaching the goal (backward messages) and the states with high probability of being reached from the initial state with high reward (forward messages): narrow near the start and the goal and widest in the middle. Human reaching movements show the same pattern, with positional variance largest in the middle of the path.

In summary: a probabilistic graphical model for optimal control in which control is inference (similar to HMMs and extended Kalman filters), very similar to dynamic programming and value iteration, but "soft".

19.3 Control as variational inference

The optimism problem. The backward pass gives $Q_t(s_t, a_t) = r(s_t, a_t) + \log \mathbb{E}[\exp(V_{t+1}(s_{t+1}))]$, an optimistic transition. Why? The inference problem is $p(s_{1:T}, a_{1:T} \mid O_{1:T})$. Marginalizing and conditioning gives the policy $p(a_t \mid s_t, O_{1:T})$: "given that you obtained high reward, what was your action probability?" That is what we want. But it also gives $p(s_{t+1} \mid s_t, a_t, O_{1:T}) \neq p(s_{t+1} \mid s_t, a_t)$: "given that you obtained high reward, what was your transition probability?" That we do not want. We should instead ask: given high reward, what was your action probability, *given that your transition probability did not change*?

Can we find another distribution $q(s_{1:T}, a_{1:T})$ that is close to $p(s_{1:T}, a_{1:T} \mid \mathcal{O}_{1:T})$ but has dynamics $p(s_{t+1} \mid s_t, a_t)$? With $x = \mathcal{O}_{1:T}$ and $z = (s_{1:T}, a_{1:T})$, find $q(z)$ to approximate $p(z \mid x)$: variational inference (Lecture 18). Let

$$q(s_{1:T}, a_{1:T}) = p(s_1) \prod_t p(s_{t+1} \mid s_t, a_t)\, q(a_t \mid s_t),$$

with the same dynamics and initial state as p; the only new thing is $q(a_t \mid s_t)$.

Proposition 19.2 [Variational lower bound]. *For this q,*

$$\log p(\mathcal{O}_{1:T}) \geq \sum_t \mathbb{E}_{(s_t, a_t) \sim q}\left[r(s_t, a_t) + \mathcal{H}(q(a_t \mid s_t))\right] :$$

maximize reward and maximize action entropy.

Proof. By the bound $\log p(x) \geq \mathbb{E}_{z \sim q(z)}[\log p(x, z) - \log q(z)]$ (the second term is the entropy $\mathcal{H}(q)$),

$$\log p(\mathcal{O}_{1:T}) \geq \mathbb{E}_{(s_{1:T}, a_{1:T}) \sim q}\Bigg[\log p(s_1) + \sum_{t=1}^{T} \log p(s_{t+1} \mid s_t, a_t) + \sum_{t=1}^{T} \log p(\mathcal{O}_t \mid s_t, a_t) \\ - \log p(s_1) - \sum_{t=1}^{T} \log p(s_{t+1} \mid s_t, a_t) - \sum_{t=1}^{T} \log q(a_t \mid s_t)\Bigg].$$

The initial-state and dynamics terms cancel, leaving $\mathbb{E}_q\left[\sum_t r(s_t, a_t) - \log q(a_t \mid s_t)\right]$, which equals the claimed sum. □

Optimizing the bound. Base case: solve for $q(a_T \mid s_T)$,

$$q(a_T \mid s_T) = \arg\max \mathbb{E}_{s_T \sim q(s_T)}\Big[\mathbb{E}_{a_T \sim q(a_T \mid s_T)}\left[r(s_T, a_T) - \log q(a_T \mid s_T)\right]\Big],$$

which is optimized when $q(a_T \mid s_T) \propto \exp(r(s_T, a_T))$:

$$q(a_T \mid s_T) = \frac{\exp(r(s_T, a_T))}{\int \exp(r(s_T, a))\, da} = \exp\big(Q(s_T, a_T) - V(s_T)\big), \qquad V(s_T) = \log \int \exp\big(Q(s_T, a_T)\big)\, da_T,$$

with $Q(s_T, a_T) = r(s_T, a_T)$, and then the optimal value of the base-case objective is

$$\mathbb{E}_{s_T \sim q(s_T)}\left[\mathbb{E}_{a_T \sim q(a_T \mid s_T)}[r(s_T, a_T) - \log q(a_T \mid s_T)]\right] = \mathbb{E}_{s_T \sim q(s_T)}[V(s_T)].$$

Recursively,

$$q(a_t \mid s_t) = \arg\max \mathbb{E}_{s_t \sim q(s_t)}\Big[\mathbb{E}_{a_t \sim q(a_t|s_t)}\big[r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t)}[V(s_{t+1})]\big] + \mathcal{H}(q(a_t \mid s_t))\Big]$$
$$= \arg\max \mathbb{E}_{s_t \sim q(s_t)}\Big[\mathbb{E}_{a_t \sim q(a_t|s_t)}\big[Q(s_t, a_t) - \log q(a_t \mid s_t)\big]\Big],$$

optimized when $q(a_t \mid s_t) \propto \exp(Q(s_t, a_t))$, so $q(a_t \mid s_t) = \exp(Q(s_t, a_t) - V(s_t))$ with $V_t(s_t) = \log \int \exp(Q_t(s_t, a_t))\, da_t$, and now

$$Q_t(s_t, a_t) = r(s_t, a_t) + \mathbb{E}\big[V_{t+1}(s_{t+1})\big],$$

a *regular* Bellman backup, not optimistic.

Algorithm 19.1: Soft value iteration

repeat
  set $Q(s, a) \leftarrow r(s, a) + \gamma \mathbb{E}[V(s')]$
  set $V(s) \leftarrow \text{soft}\max_a Q(s, a) = \log \int \exp\big(Q(s, a)\big)\, da$
until converged

This is value iteration with the max replaced by a soft max. Variants are discounted soft optimal control, with $Q_t(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}[V_{t+1}(s_{t+1})]$, and an explicit temperature, with $V_t(s_t) = \alpha \log \int \exp\big(\frac{1}{\alpha} Q_t(s_t, a_t)\big)\, da_t$.

19.4 Algorithms for RL as inference

Q-learning with soft optimality. Standard Q-learning updates $\phi \leftarrow \phi + \alpha \nabla_\phi Q_\phi(s, a)\big(r(s, a) + \gamma V(s') - Q_\phi(s, a)\big)$ with target value $V(s') = \max_{a'} Q_\phi(s', a')$. Soft Q-learning uses the same update with

$$V(s') = \text{soft}\max_{a'} Q_\phi(s', a') = \log \int \exp\big(Q_\phi(s', a')\big)\, da',$$

and acts with $\pi(a \mid s) = \exp\big(Q_\phi(s, a) - V(s)\big) = \exp\big(A(s, a)\big)$.

Algorithm 19.2: Soft Q-learning

repeat
  take some action a_i and observe (s_i, a_i, s_i', r_i), add it to $\mathcal{R}$
  sample mini-batch $\{s_j, a_j, s_j', r_j\}$ from $\mathcal{R}$ uniformly
  compute $y_j = r_j + \gamma \,\text{soft}\max_{a_j'} Q_{\phi'}(s_j', a_j')$ using target network $Q_{\phi'}$
  $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)\big(Q_\phi(s_j, a_j) - y_j\big)$
  update ϕ': copy ϕ every N steps, or Polyak average $\phi' \leftarrow \tau\phi' + (1 - \tau)\phi$
until done

Policy gradient with soft optimality. $\pi(a \mid s) = \exp(Q_\phi(s,a) - V(s))$ optimizes

$$\sum_t \mathbb{E}_{\pi(s_t,a_t)}[r(s_t,a_t)] + \mathbb{E}_{\pi(s_t)}\big[\mathcal{H}(\pi(a_t \mid s_t))\big].$$

Intuition: $\pi(a \mid s) \propto \exp(Q_\phi(s,a))$ when π minimizes $D_{\mathrm{KL}}\big(\pi(a \mid s) \| \frac{1}{Z}\exp(Q(s,a))\big)$, and $D_{\mathrm{KL}}\big(\pi(a \mid s) \| \frac{1}{Z}\exp(Q(s,a))\big) = -\mathbb{E}_{\pi(a|s)}[Q(s,a)] - \mathcal{H}(\pi) + \log Z$. This is often called ENTROPY-REGULARIZED policy gradient; it combats premature entropy collapse, and it turns out to be closely related to soft Q-learning.

Policy gradient versus Q-learning. With

$$J(\theta) = \sum_t \mathbb{E}_{\pi(s_t,a_t)}[r(s_t,a_t)] + \mathbb{E}_{\pi(s_t)}[\mathcal{H}(\pi(a \mid s_t))] = \sum_t \mathbb{E}_{\pi(s_t,a_t)}[r(s_t,a_t) - \log \pi(a_t \mid s_t)],$$

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi(a_t \mid s_t)\Big(r(s_t,a_t) + \Big(\sum_{t'=t+1}^{T} r(s_{t'},a_{t'}) - \log \pi(a_{t'} \mid s_{t'})\Big) - \log \pi(a_t \mid s_t) - 1\Big),$$

where the -1 (from differentiating $\log \pi$ inside the expectation) can be ignored as a baseline, and the inner sum approximates $Q(s_{t+1}, a_{t+1})$. Recalling $\log \pi(a_t \mid s_t) = Q(s_t,a_t) - V(s_t)$,

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_i \sum_t \big(\nabla_\theta Q(s_t,a_t) - \nabla_\theta V(s_t)\big)\big(r(s_t,a_t) + Q(s_{t+1},a_{t+1}) - Q(s_t,a_t) + V(s_t)\big),$$

where the final $V(s_t)$ acts as a baseline and can be dropped. Compare the soft Q-learning update, a descent (rather than ascent) direction,

$$-\frac{1}{N}\sum_i \sum_t \nabla_\theta Q(s_t,a_t)\big(r(s_t,a_t) + \text{soft}\max_{a_{t+1}} Q(s_{t+1},a_{t+1}) - Q(s_t,a_t)\big) :$$

the two differ only in the $\nabla_\theta V(s_t)$ term and in using the soft max (an off-policy correction) instead of the sampled $Q(s_{t+1}, a_{t+1})$.

Benefits of soft optimality. It improves exploration and prevents entropy collapse; makes it easier to specialize (finetune) policies for more specific tasks; gives a principled approach to breaking ties; gives better robustness, due to wider coverage of states; can reduce to hard optimality as the reward magnitude increases; and is a good model of human behavior (Lecture 20).

19.5 Example methods

Stochastic energy-based policies. Deterministic policies commit to one hypothesis, for example one way around an obstacle in a maze, when both look equally good early on. An energy-based policy $\pi(a \mid s) \propto \exp(Q(s,a))$, with $\pi(a_t \mid s_t) = \exp(Q_t(s_t,a_t) - V_t(s_t))$, $Q_t(s_t,a_t) = r(s_t,a_t) + \mathbb{E}[V_{t+1}(s_{t+1})]$, and $V_t(s_t) = \log \int \exp(Q_t(s_t,a_t))\, da_t$, tracks both hypotheses, since the multimodal $Q(s,a)$ gives probability to every high-value mode. Such policies also provide pretraining: a policy trained with soft Q-learning to maximize speed in any direction learns diverse running behaviors and fine-tunes faster on specific mazes and hallways than random or DDPG initialization.

Algorithm 19.3: Soft actor-critic

repeat
  Q-function update: evaluate the current policy, $Q(s,a) \leftarrow r(s,a) + \mathbb{E}_{s' \sim p_s,\, a' \sim \pi}\left[Q(s',a') - \log \pi(a' \mid s')\right]$, which converges to Q^π (update messages)
  policy update: $\pi_{\text{new}} = \arg\min_{\pi'} D_{\text{KL}}\left(\pi'(\cdot \mid s) \,\|\, \frac{1}{Z} \exp Q^{\pi_{\text{old}}}(s,\cdot)\right)$ (fit the variational distribution; in practice one gradient step)
  interact with the world and collect more data
until done

Soft actor-critic has trained real robots, for example stacking Lego blocks with a manipulator in about two hours and learning to walk with a quadruped directly in the real world.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 19 (Fall 2023).

Lecture 20: Inverse Reinforcement Learning

20.1 Why learn rewards?

So far we have manually designed a reward function to define a task. What if we want to learn the reward function from observing an expert, and then use reinforcement learning? We apply the approximate optimality model of Lecture 19, but now learn the reward.

From the imitation learning perspective: standard imitation learning copies the actions performed by the expert, with no reasoning about the outcomes of actions; human imitation learning copies the *intent* of the expert and might take very different actions. From the reinforcement learning perspective: in a game the reward is the score, but for driving in traffic, what is the reward?

Definition 20.1 [Inverse reinforcement learning]. Inverse reinforcement learning (IRL) infers reward functions from demonstrations. Forward RL is given states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, (sometimes) transitions $p(s' \mid s, a)$, and a reward function $r(s, a)$, and learns $\pi^\star(a \mid s)$. IRL is given $\mathcal{S}$, $\mathcal{A}$, (sometimes) $p(s' \mid s, a)$, and samples $\{\tau_i\}$ from $\pi^\star(\tau)$, and learns $r_\psi(s, a)$ with reward parameters ψ, and then uses it to learn $\pi^\star(a \mid s)$.

By itself this is an underspecified problem: many reward functions can explain the same behavior (a path through a grid is optimal for a reward at its endpoint, for a penalty everywhere else, and for many others). The reward may be linear, $r_\psi(s, a) = \sum_i \psi_i f_i(s, a) = \psi^T f(s, a)$ with features f, or a neural network with parameters ψ.

Feature matching IRL. With a linear reward, if the features f are important, match their expectations: with π^{r_ψ} the optimal policy for r_ψ, pick ψ such that

$$\mathbb{E}_{\pi^{r_\psi}}[f(s, a)] = \mathbb{E}_{\pi^\star}[f(s, a)],$$

where the left side is the state-action marginal under π^{r_ψ} and the right side uses the unknown optimal policy, approximated with expert samples. This is still ambiguous. The maximum margin principle picks

$$\max_{\psi, m} m \quad \text{s.t.} \quad \psi^T \mathbb{E}_{\pi^\star}[f(s, a)] \geq \max_{\pi \in \Pi} \psi^T \mathbb{E}_{\pi}[f(s, a)] + m,$$

but needs to somehow "weight" by the similarity between $\pi^\star$ and π. Using the "SVM trick", this becomes

$$\min_{\psi} \frac{1}{2}\|\psi\|^2 \quad \text{s.t.} \quad \psi^T \mathbb{E}_{\pi^\star}[f(s, a)] \geq \max_{\pi \in \Pi} \psi^T \mathbb{E}_{\pi}[f(s, a)] + D(\pi, \pi^\star),$$

with D, for example, the difference in feature expectations. Issues: maximizing the margin is a bit arbitrary; there is no clear model of expert suboptimality (slack variables can be added); and it is a messy constrained optimization problem, not great for deep learning.

20.2 Learning the reward function

Use the graphical model of Lecture 19 with $p(O_t \mid s_t, a_t, \psi) = \exp(r_\psi(s_t, a_t))$, which makes no assumption of optimal behavior. Then

$$p(\tau \mid O_{1:T}, \psi) \propto p(\tau) \exp\Big(\sum_t r_\psi(s_t, a_t)\Big),$$

where $p(\tau)$ can be ignored since it is independent of ψ. Given samples $\{\tau_i\}$ from $\pi^\star(\tau)$, maximum likelihood learning is

$$\max_\psi \frac{1}{N} \sum_{i=1}^N \log p(\tau_i \mid O_{1:T}, \psi) = \max_\psi \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \log Z,$$

with $r_\psi(\tau) = \sum_t r_\psi(s_t, a_t)$. The hard part is the PARTITION FUNCTION $Z = \int p(\tau) \exp(r_\psi(\tau))\, d\tau$.

Proposition 20.2 [IRL GRADIENT**].**

$$\nabla_\psi \mathcal{L} = \mathbb{E}_{\tau \sim \pi^\star(\tau)}\big[\nabla_\psi r_\psi(\tau)\big] - \mathbb{E}_{\tau \sim p(\tau \mid O_{1:T}, \psi)}\big[\nabla_\psi r_\psi(\tau)\big].$$

Proof. Differentiating,

$$\nabla_\psi \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_\psi r_\psi(\tau_i) - \frac{1}{Z} \int p(\tau) \exp\big(r_\psi(\tau)\big) \nabla_\psi r_\psi(\tau)\, d\tau,$$

and $\frac{1}{Z} p(\tau) \exp(r_\psi(\tau)) = p(\tau \mid O_{1:T}, \psi)$. □

The first term is estimated with expert samples; the second is the expectation under the soft optimal policy for the current reward. The gradient is zero when the two match.

Estimating the expectation. The second term is

$$\mathbb{E}_{\tau \sim p(\tau \mid O_{1:T}, \psi)}\Big[\nabla_\psi \sum_{t=1}^T r_\psi(s_t, a_t)\Big] = \sum_{t=1}^T \mathbb{E}_{(s_t, a_t) \sim p(s_t, a_t \mid O_{1:T}, \psi)}\big[\nabla_\psi r_\psi(s_t, a_t)\big],$$

and from Lecture 19, $p(a_t \mid s_t, O_{1:T}, \psi)\, p(s_t \mid O_{1:T}, \psi) = \frac{\beta(s_t, a_t)}{\beta(s_t)} \cdot p(s_t \mid O_{1:T}, \psi) \propto \beta(s_t, a_t)\, \alpha(s_t)$, the product of the backward and forward messages. Letting $\mu_t(s_t, a_t) \propto \beta(s_t, a_t)\alpha(s_t)$,

$$\sum_{t=1}^T \iint \mu_t(s_t, a_t) \nabla_\psi r_\psi(s_t, a_t)\, ds_t\, da_t = \sum_{t=1}^T \vec{\mu}_t^T \nabla_\psi \vec{r}_\psi,$$

where μ_t is the state-action visitation probability for each (s_t, a_t).

Algorithm 20.1: MaxEnt IRL

repeat
  given ψ, compute the backward message $\beta(s_t, a_t)$ (Lecture 19)
  given ψ, compute the forward message $\alpha(s_t)$ (Lecture 19)
  compute $\mu_t(s_t, a_t) \propto \beta(s_t, a_t)\alpha(s_t)$
  evaluate $\nabla_\psi \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\psi r_\psi(s_{i,t}, a_{i,t}) - \sum_{t=1}^{T}\iint \mu_t(s_t, a_t)\nabla_\psi r_\psi(s_t, a_t)\, ds_t\, da_t$
  $\psi \leftarrow \psi + \eta\nabla_\psi \mathcal{L}$
until converged

Why MaxEnt? In the case $r_\psi(s_t, a_t) = \psi^T f(s_t, a_t)$ one can show that it optimizes

$$\max_\psi \mathcal{H}(\pi^{r_\psi}) \quad \text{s.t.} \quad \mathbb{E}_{\pi^{r_\psi}}[f] = \mathbb{E}_{\pi^\star}[f],$$

where π^{r_ψ} is the optimal max-ent policy under r_ψ and the unknown expert policy is estimated with samples: the policy is as random as possible while matching features. Maximum entropy IRL was used, for example, to model drivers' route choices on a road network, with features such as road type, speed, number of lanes, and turn type.

20.3 Approximations in high dimensions

MaxEnt IRL so far requires solving for the (soft) optimal policy in the inner loop, and enumerating all state-action tuples for visitation frequency and gradient. Practical problems need large and continuous state and action spaces, states obtained via sampling only, and unknown dynamics.

Unknown dynamics and large spaces. Assume we do not know the dynamics but can sample, as in standard RL. Idea: learn $p(a_t \mid s_t, O_{1:T}, \psi)$ with any max-ent RL algorithm, maximizing $J(\theta) = \sum_t \mathbb{E}_{\pi(s_t,a_t)}[r_\psi(s_t, a_t)] + \mathbb{E}_{\pi(s_t)}[\mathcal{H}(\pi(a \mid s_t))]$, then run this policy to sample $\{\tau_j\}$:

$$\nabla_\psi \mathcal{L} \approx \frac{1}{N}\sum_{i=1}^{N}\nabla_\psi r_\psi(\tau_i) - \frac{1}{M}\sum_{j=1}^{M}\nabla_\psi r_\psi(\tau_j),$$

a sum over expert samples minus a sum over policy samples.

More efficient sample-based updates. Fully learning the policy in every iteration looks expensive. What if we use "lazy" policy optimization, only *improving* $p(a_t \mid s_t, O_{1:T}, \psi)$ a little each time? The estimator is then biased, since the samples come from the wrong distribution. One solution is importance sampling:

$$\nabla_\psi \mathcal{L} \approx \frac{1}{N}\sum_{i=1}^{N}\nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j}\sum_{j=1}^{M} w_j \nabla_\psi r_\psi(\tau_j), \qquad w_j = \frac{p(\tau)\exp(r_\psi(\tau_j))}{\pi(\tau_j)} = \frac{\exp\left(\sum_t r_\psi(s_t, a_t)\right)}{\prod_t \pi(a_t \mid s_t)},$$

where the initial-state and dynamics factors cancel. Each policy update with respect to r_ψ brings us closer to the target distribution.

Guided cost learning. Starting from an initial policy π: generate policy samples from π; update the reward r_ψ (a neural network) using the samples and human demonstrations with the importance weighted gradient above; update π with respect to the reward; repeat. The result is both a policy and a reward, learned for real robot manipulation tasks from human demonstrations.

20.4 IRL and GANs

This looks a bit like a game (Figure 20). The reward update makes demos more likely and samples less likely; the policy update, $\nabla_\theta \mathcal{L} \approx \frac{1}{M}\sum_{j=1}^{M} \nabla_\theta \log \pi_\theta(\tau_j) r_\psi(\tau_j)$, changes the policy to make it *harder* to distinguish from demos.

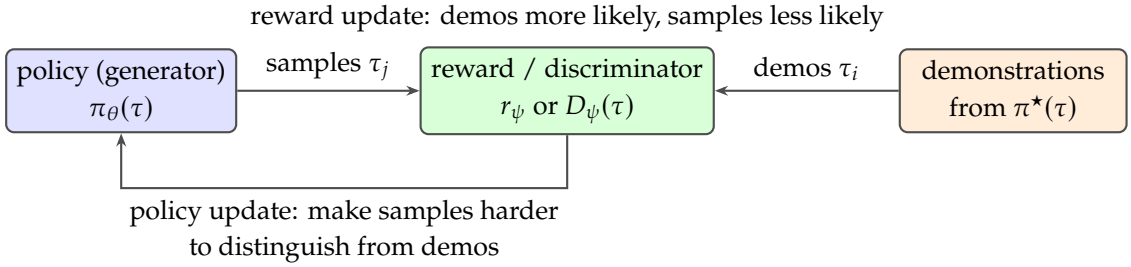


Figure 20: Inverse RL as a game between a policy (generator) and a reward or discriminator.

A GENERATIVE ADVERSARIAL NETWORK has a generator $p_\theta(x \mid z)$ producing samples from $p_\theta(x)$, data samples from $p^\star(x)$, and a discriminator $D_\psi(x) = p_\psi(\text{real} \mid x)$:

$$\psi = \arg\max_\psi \frac{1}{N}\sum_{x\sim p^\star} \log D_\psi(x) + \frac{1}{M}\sum_{x\sim p_\theta} \log\big(1 - D_\psi(x)\big), \qquad \theta \leftarrow \arg\max_\theta \mathbb{E}_{x\sim p_\theta} \log D_\psi(x).$$

Inverse RL as a GAN. The best discriminator is $D^\star(x) = p^\star(x)/(p_\theta(x) + p^\star(x))$. For IRL the optimal policy approaches $\pi_\theta(\tau) \propto p(\tau)\exp(r_\psi(\tau))$, so choose the parameterization

$$D_\psi(\tau) = \frac{p(\tau)\frac{1}{Z}\exp(r(\tau))}{p_\theta(\tau) + p(\tau)\frac{1}{Z}\exp(r(\tau))} = \frac{\frac{1}{Z}\exp(r(\tau))}{\prod_t \pi_\theta(a_t \mid s_t) + \frac{1}{Z}\exp(r(\tau))},$$

where $p(\tau)$ cancels, and optimize it (including Z, with the same objective as ψ) by

$$\psi \leftarrow \arg\max_\psi \mathbb{E}_{\tau\sim p^\star}[\log D_\psi(\tau)] + \mathbb{E}_{\tau\sim\pi_\theta}\big[\log(1 - D_\psi(\tau))\big].$$

Importance weights are no longer needed: they are subsumed into Z.

Generalization via inverse RL. What can we learn from a demonstration to enable better transfer? We need to decouple the goal from the dynamics: policy = reward + dynamics. A reward learned adversarially from a quadruped's demonstration can reproduce the behavior under different

conditions, such as a changed body.

Can we just use a regular discriminator? With $D_\psi(\tau)$ a standard binary neural network classifier and policy update $\nabla_\theta \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^{M} \nabla_\theta \log \pi_\theta(\tau_j) \log D_\psi(\tau_j)$ (generative adversarial imitation learning), optimization is often simpler to set up, with fewer moving parts; but the discriminator knows nothing at convergence, and the "reward" generally cannot be reoptimized. Guided cost learning learns a reward function, a distribution $p(\tau) \propto \exp(r(\tau))$ (the MaxEnt model) under which demos have maximum likelihood, with $D(\tau) = \frac{\frac{1}{Z}\exp(r(\tau))}{\frac{1}{Z}\exp(r(\tau))+\pi(\tau)}$; generative adversarial imitation learning uses a classifier $D(\tau)$, the probability that τ is a demo, with $\log D(\tau)$ as the "reward". These are actually the same thing, differing in the form of the classifier. Such adversarial imitation has been used, for example, for humanoid expert behaviors and for imitating motion-capture clips with simulated characters.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 20 (Fall 2023).

Lecture 21: RL with Sequence Models

21.1 Beyond MDPs

Most real-world problems are partially observed: the observations o_t (a camera image of a car chasing a cheetah, other drivers' intentions, a first-person view in a game, a conversation with a person) do not obey the Markov property, and the state s_t is not known. Partially observed MDPs can be weird.

- *Information-gathering actions.* In a maze seen from inside, without a view of the layout, it can be optimal to take actions that gather information about where we are rather than make progress, which has no counterpart in a fully observed MDP.
- *Stochastic optimal policies.* In Figure 21, states A and C produce the same observation and the goal is B. A memoryless deterministic policy must move the same way in A and C, so from one of them it never reaches the goal; the best memoryless policy moves left or right with probability 0.5 each. In a fully observed MDP there is always a deterministic optimal policy.

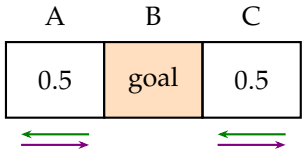


Figure 21: A and C look identical; the best memoryless policy moves left or right with probability 0.5 in both.

21.2 Which methods handle partial observability?

What does "handle" mean? Here: find the best policy in the policy class, for example the best memoryless policy $\pi_\theta(a \mid o)$.

Policy gradients. Replacing $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) A(s_{i,t}, a_{i,t})$ by the same expression with $o_{i,t}$ is fine for the policy term (its derivation used no Markov property) but takes some care for the advantage. The key point is that the advantage is a function of the state s_t and does not depend on s_{t-1}; that is why it is OK to use $r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$, since every time we see the state we expect to get the same value regardless of past states. It is *not* OK to train $\hat{V}(o_t)$, because past observations do matter for that value. So

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \Big(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid o_{i,t}) \Big) \Big(\sum_{t=1}^{T} r(o_{i,t}, a_{i,t}) \Big)$$

works, while $\frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid o_{i,t})\big(r_{i,t} + \gamma \hat{V}(o_{i,t+1}) - \hat{V}(o_{i,t})\big)$ does not. The reward-to-go version, $\frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid o_{i,t}) \sum_{t'=t}^{T} r(o_{i,t'}, a_{i,t'})$, is fine too, since the causality argument only uses the fact that future actions cannot affect past rewards.

Value-based methods do not work without the Markov property: $Q(o,a) \leftarrow r(o,a) + \gamma \max_{a'} Q(o',a')$ is wrong for the same reason. In the example above, value-based methods would find a deterministic policy and fail.

Model-based RL methods. Replacing $\hat{p}(s' \mid s,a)$ by $\hat{p}(o' \mid o,a)$ is also wrong. Suppose there are two doors, one that always opens and one that never does, and the agent cannot tell which is which. A model $\hat{p}(o' \mid o,a)$ learns $p(o' = \text{Pass} \mid o = \text{Left}, a = \text{Open}) = 0.5$, suggesting a 50% chance to open the door *each time* we try, so just keep trying. But in reality $p(o' = \text{Pass} \mid o = \text{Left}, a = \text{Open}) = 0$ if it did not open before.

21.3 State space models and history states

Learn a Markovian state space. A state space model (Lecture 18) with $p(z) = p(z_1)\prod_t p(z_{t+1} \mid z_t, a_t)$, $p_\theta(o \mid z) = \prod_t p(o_t \mid z_t)$, and encoder $q_\phi(z \mid o) = \prod_t q_\phi(z_t \mid o_{1:t})$ gives a Markovian latent state, so $Q(z,a) \leftarrow r(z,a) + \gamma \max_{a'} Q(z',a')$ is correct, where $Q(s,a)$ is unavailable and $Q(o,a)$ is wrong. This can work quite well. Why might it not be enough? Prediction can be hard, and maybe we do not need good prediction to get high rewards.

History states. Since the latent state is inferred from a history, the state is a function of the history: can we just give the history to our value function, $s_t = (o_1, o_2, \ldots, o_t)$? Does that obey the Markov property? Yes: $s_{t+1} \perp s_{t-1} \mid s_t$, because $s_{t-1} = (o_1, \ldots, o_{t-1})$ tells nothing we did not already know from $s_t = (o_1, \ldots, o_t)$. So

$$Q(o_1, \ldots, o_t, a) \leftarrow r(o,a) + \gamma \max_{a'} Q(o_1, \ldots, o_{t+1}, a')$$

is correct. How to represent it? A fixed (short) history of stacked frames fed to a network is sometimes bad; a sequence model (RNN, LSTM, Transformer) reading the whole history and outputting $Q_{a_1}, Q_{a_2}, \ldots$ is general.

A practical detail. Standard deep Q-learning collects (s,a,s'), adds it to $\mathcal{R}$, samples a batch $\{(s_i, a_i, s_i')\}_{i=1}^n$, and updates on it. With history states it collects (o_t, a_t, o_{t+1}), gets the history by concatenating $o_1, \ldots, o_{t-1}$, and samples batches of $(o_{1,i}, \ldots, o_{t,i}, a_{t,i}, o_{1,i}, \ldots, o_{t+1,i})$, which is super expensive. Can we reuse the RNN or LSTM hidden states h_t? The key idea is to store h_t in $\mathcal{R}$ (the details are a little subtle), and it is not clear how to do this with Transformers.

In summary: POMDPs are weird. Some methods "just do it", but most efficient ones do not, because they require value functions, and even those that do only get the best *memoryless* policy. We could learn a Markovian state space with models, or just use history states, which means using a sequence model to read in observation histories.

21.4 RL and language models

A language model is a Transformer (position-wise encoder, masked self-attention and nonlinear layers repeated N times, position-wise softmax) trained with supervised learning to predict each next token. We can also train it with RL if what we want is to maximize some reward function rather than just represent the data distribution. This raises questions: what is the (PO)MDP, what is the reward, and what algorithm should we use?

A basic formulation. The prompt (context, prefix) is the state s, and the completion is the action a. For "what is capital of France?", with completion "Paris ⟨eos⟩", $\pi_\theta(a \mid s) = p(x_5 \mid x_{1:4})\, p(x_6 \mid x_{1:4}, x_5)$, and the goal is $\mathbb{E}_{\pi_\theta(a|s)}[r(s,a)]$: a basic one-step RL problem, a bandit (for now). Then $\nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta \log p(x_5 \mid x_{1:4}) + \nabla_\theta \log p(x_6 \mid x_{1:4}, x_5)$ and

$$\nabla_\theta \mathbb{E}_{\pi_\theta(a|s)}[r(s,a)] = \mathbb{E}_{\pi_\theta(a|s)}\left[\nabla_\theta \log \pi_\theta(a \mid s) r(s,a)\right] \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i \mid s) r(s,a_i)$$

with samples from $\pi_\theta(a \mid s)$ (REINFORCE-style), or

$$\approx \frac{1}{N} \sum_i \frac{\pi_\theta(a_i \mid s)}{\bar{\pi}(a_i \mid s)} \nabla_\theta \log \pi_\theta(a_i \mid s) r(s,a_i) = \hat{\nabla}(\theta, \bar{\pi}, \{a_i\})$$

with samples from $\bar{\pi}(a \mid s)$ (importance-weighted, as in PPO). The latter is preferred because it allows several gradient steps per batch of samples, and evaluating the reward can be expensive:

Algorithm 21.1: Policy gradient for language models

```
repeat
    sample a batch B = {a_i}, a_i ~ π_θ(a | s)
    evaluate r(s, a_i) for each a_i ∈ B
    π̄ ← π_θ
    for K times do
        sample a minibatch M ⊂ B
        θ ← θ + α ∇̂(θ, π̄, M)
until done
```

Learned rewards. What is $r(s,a_i)$? It can itself be a neural network: a reward model that scores "Paris" at +1.0, "A city called Paris" at +0.9, "I dunno..." at −0.1, "London" at −1.0, and an insulting answer at −10.0.

21.5 RL from human feedback

How do we train the reward model $r_\psi(s,a)$? We could ask people to label answers with numbers and fit them by supervised learning, but how do people know these numbers? It is easier to compare.

Definition 21.1 [Rewards from preferences]**.** Given (s, a_1, a_2), predict how likely a person is to prefer a_1 over a_2:

$$p(a_1 \text{ preferred over } a_2) = \frac{\exp\big(r_\psi(s, a_1)\big)}{\exp\big(r_\psi(s, a_1)\big) + \exp\big(r_\psi(s, a_2)\big)},$$

and maximize the likelihood of the human labels with respect to the parameters ψ of r.

Algorithm 21.2: RL from human feedback

```
run supervised training (or finetuning) to get an initial π_θ(a | s)
repeat
    for each s sample K answers a_k ~ π(a | s), add to D = {(s_i, a_{i,1}, ..., a_{i,K})}
    get humans to label which a_{i,k} they prefer for each s_i
    train r_ψ using the labeled dataset D
    update π_θ using RL with reward r_ψ(s, a)
until done
```

Some issues:

- Human preferences are expensive. Most preference data comes from the initial supervised model, and each RL iteration typically adds a smaller set; there are many RL iterations (including generating new samples from the policy) per round of preference gathering. This is *model-based* RL (the reward model is the model), and offline model-based RL if preferences are collected only once.
- So the problem is overoptimization: the policy exploits the reward model, as in model-based RL. The usual fix is to penalize divergence from the original supervised policy π_β:

$$\mathbb{E}_{\pi_\theta(a|s)}[r(s,a)] - \beta D_{\mathrm{KL}}(\pi_\theta \| \pi_\beta) = \mathbb{E}_{\pi_\theta(a|s)}\big[r(s,a) + \beta \log \pi_\beta(a \mid s) - \beta \log \pi_\theta(a \mid s)\big].$$

- The reward model needs to be very good, so it is typically itself a large Transformer.

21.6 Multi-step RL with language models

In a dialogue (for example a visual dialogue game, where a questioner bot asks about an image and must then pick it out), the action is what the bot says, the observation is what the human says, the state is the history $s_3 = \{o_1, a_1, o_2, a_2, o_3\}$, and the reward is the dialogue outcome. Applications include dialogue systems, assistant chat bots, tool use (for example command-line tools), and playing games. This is not RLHF: RLHF learns from human preferences with an episode of a single answer, while sequential decision making learns from the dialogue outcome with the whole dialogue as an episode.

How to train? Policy gradients require samples from the human, which could work but is expensive. Value-based methods could learn offline from data.

What is a time step? A per-utterance time step is the natural choice with short horizons, but has a huge action space. A per-token time step has simple discrete actions but very long horizons.

Value-based RL, per utterance. With $Q(s, a) \leftarrow r(s) + \gamma \max_{a'} Q(s', a')$, encode the entire dialogue history s_t and the candidate utterance a_t with Transformer sequence models (which could be pretrained language models such as BERT) and combine them into $Q(s_t, a_t)$.

Value-based RL, per token. A Transformer value function reads the history token by token and outputs, at each position, a value for every possible next token. The target for the current token adds $r(s)$ to the max over the next token's values if the agent chooses the next token, or else to the value of the next token in the dataset.

Putting it together. The usual value-based details apply (target networks, replay buffers, the double-Q trick, and so on). These methods can be used with either online or offline RL, but are particularly useful offline, which means taking care of distributional shift: a policy constraint (KL divergence on the actor), a CQL-style penalty, or an IQL-style backup; there is no single best answer yet. Examples include human-centric dialogue training with an actor-critic, a KL policy constraint, and rewards from user sentiment (per-utterance time steps); a negotiation chatbot with a Q-function trained with CQL and task rewards (per utterance); and implicit language Q-learning, combining IQL and CQL with a behavior-cloned actor for policy extraction on visual dialogue (per-token time steps).

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 21 (Fall 2023).

Lecture 22: Meta-Learning and Transfer Learning

22.1 Transfer learning

In Montezuma's Revenge (Lecture 13) we know what to do because we understand what the sprites mean: the key opens doors, ladders can be climbed, and the skull, whatever it does, cannot be good. Prior understanding of problem structure can help us solve complex tasks quickly. Can RL use the same prior knowledge? If we have solved prior tasks, we might have acquired useful knowledge for a new one, stored in a Q-function (which actions or states are good), a policy (which actions are potentially useful; some actions are never useful), a model (the laws of physics that govern the world), or features and hidden states (a good representation, which should not be underestimated).

Definition 22.1 [Transfer learning]. Transfer learning uses experience from one set of tasks for faster learning and better performance on a new task. In RL, a task is an MDP: experience in the source domain is transferred to the target domain. The number of attempts in the target domain is the number of shots: 0-shot runs a policy trained in the source domain, 1-shot tries the task once, and few-shot tries it a few times.

Transfer problems can be framed in three ways:

1. *forward transfer*: learn policies that transfer effectively, by training on a source task and then running (or finetuning) on the target task, which relies on the tasks being quite similar;
2. *multi-task transfer*: train on many tasks and transfer to a new task, sharing representations and layers across tasks; the new task must be similar to the distribution of training tasks;
3. *meta-learning*: learn to learn on many tasks, accounting for the fact that we will be adapting to a new task during training.

22.2 Forward transfer

Pretraining followed by finetuning is the most popular transfer learning method in (supervised) deep learning. In RL we are likely to face *domain shift* (representations learned in the source domain might not work well in the target domain, as from a driving simulator to real driving), a *difference in the MDP* (some things possible in the source domain are not possible in the target domain), and *finetuning issues* (finetuning may still need to explore, but the optimal policy at the end of pretraining may be deterministic).

Domain adaptation in computer vision. Train a classifier in the source domain and make an intermediate layer z invariant to the domain: a domain classifier $D_\phi(z)$ guesses the domain from z, and its

gradient is reversed into the feature extractor, so that features cannot reveal the domain. This rests on the INVARIANCE ASSUMPTION: everything that is different between domains is irrelevant. Formally, $p(x)$ is different, but there exists some $z = f(x)$ such that $p(y \mid z) = p(y \mid x)$ while $p(z)$ is the same.

Domain adaptation in RL for dynamics. Invariance is not enough when the dynamics do not match: a wall present in the real world but absent in the simulator changes which behavior is good. Instead, learn classifiers that distinguish real from simulated transitions and add a reward offset in the simulator, $\tilde{r}(s, a) = r(s, a) + \Delta r(s, a)$ with

$$\begin{aligned}\Delta r(s_t, a_t, s_{t+1}) &= \log p_{\text{target}}(s_{t+1} \mid s_t, a_t) - \log p_{\text{source}}(s_{t+1} \mid s_t, a_t) \\ &= \log p(\text{target} \mid s_t, a_t, s_{t+1}) - \log p(\text{target} \mid s_t, a_t) \\ &\quad - \log p(\text{source} \mid s_t, a_t, s_{t+1}) + \log p(\text{source} \mid s_t, a_t),\end{aligned}$$

computed from two domain classifiers by Bayes' rule. The penalty steers the policy away from transitions that are impossible in the target domain. It does not work when the target domain allows behaviors the source domain does not support at all.

What if we can also finetune? RL tasks are generally much less diverse than supervised ones, so features are less general and policies and value functions become overly specialized. And optimal policies in fully observed MDPs are deterministic, so exploration is lost at convergence, and low-entropy policies adapt very slowly to new settings. Unsupervised skill discovery (Lecture 14) and maximum entropy RL (Lecture 19) help.

Maximizing forward transfer. The basic intuition: the more varied the training domain, the more likely we are to generalize in zero shot to a slightly different domain. RANDOMIZATION of dynamics, appearance, and so on is widely used for transfer from simulation to the real world. For example, training a hopper on a single torso mass works only near that mass, while training on an ensemble of masses works across the range; the ensemble can also be adapted toward the test domain, and it holds up under some unmodeled effects. Randomized simulated textures and lighting allowed real-world flight without a single real image, and randomized dynamics allowed sim-to-real robotic control and quadruped locomotion over challenging terrain.

22.3 Multi-task transfer

Learning several tasks together can accelerate learning of all of them and provide better pretraining for downstream tasks. Multi-task RL corresponds to single-task RL in a joint MDP: pick an MDP at random in the first state (through $p(s_0)$) and proceed in it. If the policy can do multiple things in the same environment (do the dishes or the laundry), it needs a context: a CONTEXTUAL POLICY $\pi_\theta(a \mid s, \omega)$ with task descriptor ω, equivalently a policy on the augmented state $\tilde{s} = (s, \omega)$ with $\tilde{\mathcal{S}} = \mathcal{S} \times \Omega$.

Goal-conditioned policies. A policy $\pi_\theta(a \mid s, g)$ conditioned on a goal g, another state, with reward $r(s, a, g) = \delta(s = g)$ or $\delta(\|s - g\| \le \epsilon)$, needs no manually defined reward for each task and transfers

in zero shot to a new task if it is another goal. But such policies are often hard to train in practice, and not all tasks are goal-reaching tasks (for example, reaching a goal while avoiding a region).

22.4 Meta-learning

If you have learned 100 tasks already, can you figure out how to *learn* more efficiently? Then having multiple tasks is a huge advantage. Meta-learning is learning to learn; in practice it is very closely related to multi-task learning, with many formulations: learning an optimizer, learning an RNN that ingests experience, or learning a representation. It is a good idea because deep RL, especially model-free, requires a huge number of samples; a meta-learned learner can explore more intelligently, avoid actions known to be useless, and acquire the right features more quickly.

Supervised meta-learning. Supervised learning learns $f(x) \to y$; supervised meta-learning learns $f(\mathcal{D}^{\text{tr}}, x) \to y$, trained on many (few-shot) tasks, each with a training set and a test set, and tested on new tasks. The training set can be read in, for example, by an RNN over $(x_1, y_1), (x_2, y_2), \ldots$ followed by x_{test}. "Generic" learning is

$$\theta^\star = \arg\min_\theta \mathcal{L}(\theta, \mathcal{D}^{\text{tr}}) = f_{\text{learn}}(\mathcal{D}^{\text{tr}});$$

"generic" meta-learning is

$$\theta^\star = \arg\min_\theta \sum_{i=1}^{n} \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{ts}}), \qquad \phi_i = f_\theta(\mathcal{D}_i^{\text{tr}}).$$

For the RNN, $\phi_i = [h_i, \theta_p]$: the hidden state after reading the training set plus the meta-learned weights of the output network $p_{\phi_i}(y \mid x)$.

Definition 22.2 [Meta-reinforcement learning]. Reinforcement learning solves

$$\theta^\star = \arg\max_\theta \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)] = f_{\text{RL}}(\mathcal{M})$$

for an MDP $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, r\}$. Meta-reinforcement learning solves

$$\theta^\star = \arg\max_\theta \sum_{i=1}^{n} \mathbb{E}_{\pi_{\phi_i}(\tau)}[R(\tau)], \qquad \phi_i = f_\theta(\mathcal{M}_i),$$

over meta-training MDPs $\{\mathcal{M}_1, \ldots, \mathcal{M}_n\}$, assuming $\mathcal{M}_i \sim p(\mathcal{M})$. At meta-test time, sample $\mathcal{M}_{\text{test}} \sim p(\mathcal{M})$ and get $\phi_i = f_\theta(\mathcal{M}_{\text{test}})$.

Examples: household tasks for a robot, or a running robot that must run at different target velocities (0.5, 0.7, -0.2 m/s in training, a new velocity at test time).

Contextual policies and meta-learning. Meta-RL can be written as $\theta^\star = \arg\max_\theta \sum_i \mathbb{E}_{\pi_\theta}[R(\tau)]$ with

$\pi_\theta(a_t \mid s_t, s_1, a_1, r_1, \ldots, s_{t-1}, a_{t-1}, r_{t-1})$: the context is used to infer whatever we need to solve $\mathcal{M}_i$, i.e. z_t or ϕ_i (which are really the same thing). In meta-RL the context is inferred from experience in $\mathcal{M}_i$; in multi-task RL it is typically given (a stack location, a walking direction, where to hit a puck).

22.5 Meta-RL with recurrent policies

The main question is how to implement $f_\theta(\mathcal{M}_i)$. What should it do? (1) Improve the policy with experience from $\mathcal{M}_i$, $\{(s_1, a_1, s_2, r_1), \ldots, (s_T, a_T, s_{T+1}, r_T)\}$; and (2), new in RL, choose how to interact, i.e. choose a_t: meta-RL must also *choose how to explore*. So we just train an RNN policy (Figure 22), with $\phi_i = [h_i, \theta_\pi]$. Crucially, the RNN hidden state is not reset between episodes.

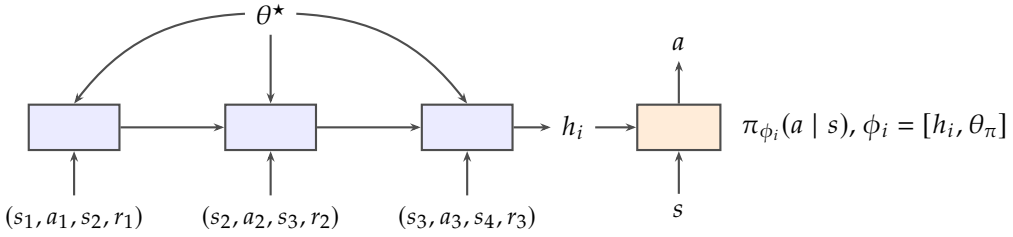


Figure 22: A recurrent meta-RL policy: the hidden state summarizes the transitions seen in the current MDP, and the meta-learned weights $\theta^\star$ are shared.

Why recurrent policies learn to explore. Meta-training optimizes $\theta^\star = \arg\max_\theta \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} r(s_t, a_t)\right]$ over the entire *meta-episode*, spanning several episodes in the same MDP. In a maze where the goal location varies across MDPs, wasted moves in the first episode are worth it if they find the goal for later episodes, so optimizing total reward over the meta-episode with an RNN policy automatically learns to explore. Architectures include standard RNNs (LSTMs), attention with temporal convolution, and parallel permutation-invariant context encoders.

22.6 Gradient-based meta-learning

Back to representations: is pretraining a type of meta-learning? Better features mean faster learning of a new task. What if $f_\theta(\mathcal{M}_i)$ is *itself* an RL algorithm? Standard RL computes $\theta^\star = \arg\max_\theta J(\theta)$ with $J(\theta) = \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)]$ by $\theta^{k+1} \leftarrow \theta^k + \alpha \nabla_{\theta^k} J(\theta^k)$. So let

$$f_\theta(\mathcal{M}_i) = \theta + \alpha \nabla_\theta J_i(\theta),$$

which requires interacting with $\mathcal{M}_i$ to estimate $\nabla_\theta \mathbb{E}_{\pi_\theta}[R(\tau)]$.

Definition 22.3 [Model-agnostic meta-learning]. Model-agnostic meta-learning (MAML) for RL replaces the single-task update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ by

$$\theta \leftarrow \theta + \beta \sum_i \nabla_\theta J_i\left[\theta + \alpha \nabla_\theta J_i(\theta)\right],$$

so that one adaptation step from θ lands near the optimum $\theta_i^\star$ of each task.

What did we just do? Supervised learning is $f(x) \to y$, supervised meta-learning is $f(\mathcal{D}^{\text{tr}}, x) \to y$, and model-agnostic meta-learning is

$$f_{\text{MAML}}(\mathcal{D}^{\text{tr}}, x) = f_{\theta'}(x), \qquad \theta' = \theta - \alpha \sum_{(x,y)\in\mathcal{D}^{\text{tr}}} \nabla_\theta \mathcal{L}(f_\theta(x), y).$$

It is just another computation graph, implementable with any autodiff package, but with a favorable inductive bias. A quadruped meta-trained with MAML on forward and backward running adapts to either direction after one gradient step. Further work covers meta-policy gradient estimators, structured exploration, and hybrid algorithms.

22.7 Meta-RL as a POMDP

A POMDP $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{E}, r\}$ has an observation space $\mathcal{O}$ and emission probability $\mathcal{E} = p(o_t \mid s_t)$; the policy must act on observations, $\pi_\theta(a \mid o)$, which typically requires either explicit state estimation, $p(s_t \mid o_{1:t})$, or policies with memory. Meta-RL is just a POMDP: with a policy $\pi_\theta(a \mid s, z)$ where z encapsulates the information the policy needs to solve the current task, define $\tilde{\mathcal{M}} = \{\tilde{\mathcal{S}}, \mathcal{A}, \tilde{\mathcal{O}}, \tilde{\mathcal{P}}, \mathcal{E}, r\}$ with $\tilde{\mathcal{S}} = \mathcal{S} \times \mathcal{Z}$, $\tilde{s} = (s, z)$, $\tilde{\mathcal{O}} = \mathcal{S}$, $\tilde{o} = s$. Learning a task means inferring z from the context $(s_1, a_1, s_2, r_1), (s_2, a_2, s_3, r_2), \ldots$, and the key idea is that solving the POMDP $\tilde{\mathcal{M}}$ is equivalent to meta-learning.

Using explicit state estimation, we need $p(z_t \mid s_{1:t}, a_{1:t}, r_{1:t})$. Explore via posterior sampling with latent context: repeatedly sample $z \sim \hat{p}(z_t \mid s_{1:t}, a_{1:t}, r_{1:t})$ from some approximate (for example variational) posterior, and act according to $\pi_\theta(a \mid s, z)$ to collect more data, acting as though z were correct. This is not optimal, since it never takes actions purely to gather information, but it is pretty good, both in theory and in practice.

Variational inference for meta-RL. Use a policy $\pi_\theta(a_t \mid s_t, z_t)$ and an inference network $q_\phi(z_t \mid s_1, a_1, r_1, \ldots, s_t, a_t, r_t)$, and solve

$$(\theta, \phi) = \arg\max_{\theta,\phi} \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}_{z\sim q_\phi, \tau\sim\pi_\theta} \left[R_i(\tau) - D_{\text{KL}}(q(z \mid \ldots) \| p(z)) \right],$$

which maximizes post-update reward (as in standard meta-RL) while staying close to the prior. This is conceptually very similar to RNN meta-RL, but with a stochastic z, which enables exploration via posterior sampling. PEARL instantiates it with a permutation-invariant encoder: each transition $(s, a, s', r)_n$ yields a factor $\Psi_\phi(z \mid c_n)$, the product gives $q_\phi(z \mid c)$, and the maximization uses soft actor-critic.

22.8 Three perspectives on meta-RL

1. *Just RNN it*: conceptually simple and relatively easy to apply, but vulnerable to meta-overfitting and challenging to optimize in practice.
2. *Bi-level optimization*, $f_\theta(\mathcal{M}_i) = \theta + \alpha \nabla_\theta J_i(\theta)$ (MAML for RL): good extrapolation ("consistent") and conceptually elegant, but complex and requires many samples.
3. *An inference problem*, $\pi_\theta(a \mid s, z)$ with $z_t \sim p(z_t \mid s_{1:t}, a_{1:t}, r_{1:t})$: simple, effective exploration via posterior sampling and an elegant reduction to solving a special POMDP, but vulnerable to meta-overfitting and challenging to optimize in practice.

They are not that different: perspective 3 is just perspective 1 with stochastic hidden variables ($\phi = z$), and perspective 2 is just a particular architecture choice for these.

Meta-RL and emergent phenomena. Meta-RL gives rise to episodic learning; model-free meta-RL gives rise to model-based adaptation; and meta-RL gives rise to causal reasoning. Humans and animals seemingly learn in a variety of ways (highly efficient but apparently model-free RL, episodic recall, model-based RL, causal inference). Perhaps each is a separate "algorithm" in the brain, but maybe these are all emergent phenomena resulting from meta-RL.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 22 (Fall 2023).

Lecture 23: Challenges and Open Problems

23.1 A brief review

The course covered learning-based control: imitation learning, inverse RL, and reinforcement learning, connected to probabilistic inference through control as inference. Within RL: "classic" model-free algorithms (policy gradients and advanced policy gradients such as TRPO and PPO; actor-critic methods, including Q-function actor-critics such as SAC; and value-based methods such as deep Q-learning); model-based control (such as LQR) and model-based RL with or without a policy; exploration; and unsupervised RL.

23.2 Challenges in deep RL

Challenges with the core algorithms are *stability* (does the algorithm converge?), *efficiency* (how long, in samples, does it take to converge?), and *generalization* (after it converges, does it generalize?). Challenges with the assumptions are whether this is even the right problem formulation, and what the source of supervision is.

Stability and hyperparameter tuning. Devising stable RL algorithms is very hard.

- Q-learning and value function estimation: fitted Q and fitted value methods with deep network function estimators are typically not contractions, hence there is no guarantee of convergence (the backup $\mathcal{B}V$ followed by projection can move away from $V^\star$, Lecture 7); and there are many parameters for stability: target network delay, replay buffer size, clipping, sensitivity to learning rates.
- Policy gradients (likelihood ratio, REINFORCE): a very high variance gradient estimator, requiring many samples and complex baselines, with parameters such as batch size, learning rate, and baseline design.
- Model-based RL: the model class and fitting method matter; optimizing the policy with respect to the model is nontrivial due to backpropagation through time; and, more subtly, the policy tends to exploit the model.

Sample complexity. Each step down the following ladder is about 10× more sample-efficient: gradient-free methods (NES, CMA), fully online methods (A3C), policy gradient methods (TRPO), replay buffer value estimation methods (Q-learning, DDPG, NAF, SAC), model-based deep RL (PETS, guided policy search), and model-based "shallow" RL (PILCO). Table 1 gives the orders of magnitude on the half-cheetah.

Method family (example)	Steps	Episodes	Real time
fully online methods	100,000,000	100,000	~ 15 days
TRPO with GAE	10,000,000	10,000	~ 1.5 days
replay buffer value estimation	1,000,000	1,000	~ 3 hours
model-based deep RL (PETS)	30,000	30	~ 5 minutes

Table 1: Approximate samples needed to learn half-cheetah running, by method family.

Evolution strategies, a gradient-free method, sit a further step above the fully online methods. On a real robot, a comparison on a log scale shows another 10× gap, with learning from about 20 minutes of experience. High sample complexity means waiting a long time for experiments, makes real-world learning difficult or impractical, precludes expensive high-fidelity simulators, and limits applicability to real-world problems.

Scaling up and generalization. Supervised deep learning (ImageNet) is large-scale, emphasizes diversity, and is evaluated on generalization. Deep RL benchmarks are small-scale, emphasize mastery, and are evaluated on performance: where is the generalization? RL has a big problem: in supervised learning, data collection is done once and training runs for many epochs; in RL, the collect-and-train loop is done many times, and in actual RL research the whole loop is rerun by the researcher many, many times. Learning to run in simulation takes 6 days of real time (if it were real time) to run on an infinite flat plane, and the real world is not so simple. Off-policy RL suggests a fix: a big dataset from past interaction, trained for many epochs, occasionally getting more data.

Single task or multi-task? Multi-task RL is single-task RL in a joint MDP (Lecture 22), which is where generalization can come from; this may require no new assumption but merits additional treatment.

Where does the supervision come from? To learn from many different tasks, we need to get those tasks from somewhere: learn objectives or rewards from demonstration (inverse RL), or generate objectives automatically. Other sources of supervision are demonstrations, language, and human preferences. Should supervision tell us *what* to do or *how* to do it?

Rethinking the problem formulation. How should we define a control problem: what is the data, what is the goal, and what is the supervision (which may not be the same as the goal)? Think about the assumptions that fit your problem setting, and do not assume that the basic RL problem is set in stone.

23.3 RL as an engineering tool

We think of RL as learning decisions from consequences. Engineering a control system classically means characterizing the system (deriving the equations of motion), simulating it, and designing a controller. With RL, anything you can simulate you can control: before, characterize, simulate, control; now, characterize, simulate, run RL. This is a powerful engineering tool, not that different from the conventional approach. Its main role is as a powerful *inversion* engine, and its main

weakness is that we still need to simulate.

23.4 RL and the real world

Moravec's paradox. Abstract thought is a new trick, so we find it hard, while perception and motor control are so well mastered that they seem easy; the hard problems are easy and the easy problems are hard. This seems like a statement about AI but is actually a statement about the physical universe: there are "easy" universes, where success means high reward (optimal control), the world is closed and the rules are known, there is lots of simulation, and the main question is whether RL can optimize really well; and "hard" universes, where success means survival (good-enough control), the world is open and everything must come from data, there is no simulation (because the rules are unknown), and the main question is whether RL can generalize and adapt.

How do we engineer a system that can deal with the unexpected, with minimal external supervision about what to do, unexpected situations requiring adaptation, solutions that must be discovered autonomously, and the need to stay alive long enough to discover them? Humans are extremely good at this, current AI systems are extremely bad at it, and RL in principle can do it while nothing else can. But this setting is rarely studied in RL research, and RL should be really good in the "hard" universes.

Questions that come up in the real world (not only for robots, which are just the most natural to think about because they are embodied like us): how do we tell RL agents what we want them to do (for example, from human preferences)? How can we learn fully autonomously in continual environments (a sequence of tasks that reset each other, or reset-free learning via multi-task learning)? How do we remain robust as the environment changes around us? What is the right way to generalize using experience and prior data? What is the right way to bootstrap exploration with prior experience (exploring from a learned behavioral prior instead of from scratch)?

Why is this interesting? It is exciting to see what solutions intelligent agents come up with, most of all when they are unexpected; this requires the world they inhabit to admit novel solutions, and so to be complicated enough. To see interesting emergent behavior we must train in environments that actually require it. RL in the real world may be difficult, but it is also rewarding.

23.5 RL as "universal" learning

Deep learning works because of big models and big *labeled* datasets. Self-supervised learning aims to reduce the supervision burden: a small labeled dataset plus a giant unlabeled "garbage" dataset (the Internet). But then where does the knowledge come from? "Classic" unsupervised learning fits $p_\theta(x)$, as large language models do (perhaps why prompting them is such an art).

Stepping back: we need machine learning to produce adaptable and complex decisions (Lecture 1). Even an image label is usually not the decision: what happens with it afterwards (tagging a user's

photo, detecting an endangered animal in a camera trap) is a decision with consequences. RL is a way to use "cheap", previously collected data: a limited amount of supervision tells us what the task is (for example a reward function), while a large, cheap dataset of previously collected data tells us how the world works (the dynamics). Big datasets from past interaction can train the best possible initial model for a new task. With naive RL this is a costly interactive process in the real world, but self-supervised learning is about using cheap data we already have lying around, which points to offline RL.

The recipe. Take a large dataset of diverse (but possibly low-quality) behavior, run offline RL with human-defined skills, goal-conditioned RL, or self-supervised skill discovery, and then learn the downstream task. For example, goal-conditioned offline RL with a conservative method based on CQL needs no reward function at all (tasks are goal images) and works very well as unsupervised pretraining before finetuning with a task reward and limited data. Offline RL can also train language models: a model-based approach to multi-turn dialogue uses a language model as a "human simulator" to generate a huge synthetic dataset that need not be good, only plausible, and RL then infers what is desirable to do to get good outcomes. RL with language models is great for tasks that are easy for them to simulate but hard for them to perform optimally: instead of emulating humans, they learn to achieve desired outcomes using their understanding of how humans behave.

23.6 Back to the bigger picture

Learning is the basis of intelligence: RL can reason about decision making, and deep models allow RL algorithms to learn and represent complex input-output mappings, solving complex problems end to end. What is missing, and where does the signal come from? Perhaps from unsupervised or self-supervised learning ("LeCun's cake"), meaning model learning (predicting the future) and generative modeling of the world, with lots to do even before accomplishing a goal; from imitation and understanding other agents (we are social animals with culture, for a reason); from the giant value backup (all it takes is one +1); or from all of the above.

How should we answer these questions? Pick the right problems: ask whether a problem has a chance of solving an important problem (optimism in the face of uncertainty is a good exploration strategy). Do not be afraid to change the problem statement, since many of these challenges will not be met by iterating on existing benchmarks. Applications matter: applying methods to realistic, challenging real-world domains teaches us a lot about what is missing, and RL has a long history of disregarding this. Think big and start small.

Source. Sergey Levine, *CS 285: Deep Reinforcement Learning*, Lecture 23 (Fall 2023).